CloudDQ AI

Serverless, AI-Assisted Data Quality Assessment and Monitoring Platform on AWS

Team name	[Team name]

#	Team member	Reg. No.	Role
1	Pooja Palani	[Reg. No.]	Backend & API — Lambda, API Gateway, DynamoDB, Bedrock invoke
2	Dewlyn Mary	[Reg. No.]	Frontend & Identity — Amplify, Cognito, S3
3	Lathika Shree	[Reg. No.]	Operations & Governance — CloudWatch, IAM, config, Bedrock access

Live application: https://main.d3ffgcfvtof2u.amplifyapp.com/
Source: github.com/Pooja-Palani/clouddq-ai

Contents

1. Title and Team
2. Objective
3. List of Modules
4. Detailed Description of the Modules
5. Architecture
6. Individual Contribution
7. Tools / Software Requirements
8. AWS Services Used
9. Screenshots of AWS Services
10. Feature Output Screenshots
11. Results, Challenges and Future Scope
12. References
13. Appendix — Complete Module Code

1. Title and Team

Project title: CloudDQ AI — Serverless, AI-Assisted Data Quality Assessment and Monitoring Platform on AWS

Team name: [Team name]

#	Team member	Reg. No.	Role
1	Pooja Palani	[Reg. No.]	Backend & API — Lambda, API Gateway, DynamoDB, Bedrock invoke
2	Dewlyn Mary	[Reg. No.]	Frontend & Identity — Amplify, Cognito, S3
3	Lathika Shree	[Reg. No.]	Operations & Governance — CloudWatch, IAM, config, Bedrock access

2. Objective

Organisations make decisions on data they rarely measure. CloudDQ AI lets a Data User upload a CSV dataset, scores it across six standard data-quality dimensions with an explainable rule engine, shows the result as a dashboard, writes a plain-language executive summary with Amazon Bedrock, and lets a Business User raise an alert that the Data User corrects and re-validates. The whole platform is serverless on AWS, keeps raw data private, and sends only aggregated metrics to the AI model.

- Measure dataset quality with a transparent, weighted six-dimension score (0–100).
- Classify health as Healthy (80–100), Warning (60–79) or Critical (<60) and decide fitness for analytics.
- Visualise dimensions, column quality, score trend, severity and issues for each dataset.
- Detect personal data (names, emails, phone numbers) in uploaded columns with Amazon Comprehend.
- Generate an executive summary and top-3 fixes with Amazon Bedrock from metrics only, never raw rows.
- Support two roles: Data Users run quality checks; Business Users review results and raise alerts.
- Close the loop with a correction workflow: alert (emailed through Amazon SNS) → acknowledge → re-upload → re-analyse → resolve.
- Run fully serverless on AWS with no stored AWS keys in the browser.

Data quality score

```
DQ = 0.25×Completeness + 0.20×Validity + 0.15×Uniqueness + 0.15×Consistency +
0.15×Accuracy + 0.10×Timeliness
Health: 80–100 HEALTHY (fit for analytics) · 60–79 WARNING · <60 CRITICAL
```

3. List of Modules

ID	Module	Purpose	AWS services
M1	Authentication & Role-Based Access	Email sign-in and two user roles enforced in both the UI and the API.	Cognito, API Gateway authorizer
M2	Frontend SaaS Application	React + TypeScript single-page app with role-aware navigation.	Amplify Hosting
M3	Dataset Upload & Repository	Secure CSV upload with presigned URLs and a per-user dataset catalogue.	S3, Lambda, DynamoDB
M4	Data Quality Engine (Six Dimensions)	Explainable profiling, scoring, rules, anomaly detection and PII detection.	Lambda, Comprehend
M5	Dashboard, Reports & Analytics	Visual quality report per dataset with export and print.	Lambda, DynamoDB, Amplify
M6	AI Executive Summary	Business-friendly explanation and top-3 recommendations from aggregated metrics.	Bedrock, Lambda
M7	Alerts & Soft Correction Workflow	Business → Data User collaboration with email notification and re-validation.	Lambda, DynamoDB, SNS
M8	API, Data Store & Cloud Deployment	One REST API, one Lambda, one table — monitored and least-privilege.	API Gateway, Lambda, DynamoDB, CloudWatch, SNS, IAM

4. Detailed Description of the Modules

M1 · Authentication & Role-Based Access

AWS services: Cognito, API Gateway authorizer

- Amazon Cognito user pool with email sign-in and a strong password policy (upper, lower, number, symbol).
- Public SPA app client (no secret) using SRP authentication through amazon-cognito-identity-js.
- Two Cognito groups, DataUser and BusinessUser; the group claim in the ID token decides the role.
- Every API call carries the ID token as Authorization: Bearer; API Gateway's Cognito authorizer rejects missing or invalid tokens with 401.
- Lambda re-checks the role for each action (e.g. only DataUser can upload; only BusinessUser can raise alerts) and returns 403 otherwise.

M2 · Frontend SaaS Application

AWS services: Amplify Hosting

- React 19, TypeScript, Vite and Tailwind CSS; charts with Recharts; routing with React Router.
- Pages: Landing, Login, Home (role workspace), Datasets repository, Upload, Dataset detail, Alerts.
- Protected routes redirect signed-out users to login; the Upload route is visible only to Data Users.
- Built and hosted by AWS Amplify from the GitHub main branch; configuration is injected at build time through VITE_ environment variables.

M3 · Dataset Upload & Repository

AWS services: S3, Lambda, DynamoDB

- The browser asks the API for a short-lived presigned S3 PUT URL (15 minutes), then uploads the CSV directly to the private bucket.
- The dataset is registered in DynamoDB (DATASET#id / META) with owner, file, S3 key, status and a status timeline.
- Uploads are keyed datasets/<user>/<dataset>/<file>.csv; the bucket blocks all public access and is encrypted (SSE-S3).
- The repository page lists the user's datasets through the GSI1 index (USER#sub) with score and health badges.

M4 · Data Quality Engine (Six Dimensions)

AWS services: Lambda, Comprehend

- Profiler infers each column's type (id, email, phone, date, numeric, categorical, string) and null rate.
- Dimension scores: Completeness (null cells), Validity (format checks), Uniqueness (duplicate keys), Consistency (case/format variants), Accuracy (range checks), Timeliness (future and stale dates).
- DQ score = 0.25·Completeness + 0.20·Validity + 0.15·Uniqueness + 0.15·Consistency + 0.15·Accuracy + 0.10·Timeliness.
- Rules with thresholds and severities (e.g. Uniqueness ≥ 98 HIGH, Validity ≥ 90 MEDIUM) turn failures into issues.

- Anomaly detection compares with the previous run: null-rate spike, volume drop, freshness delay and dimension score drop. No ML model is needed.
- Amazon Comprehend DetectPiiEntities scans up to five sample values per column and flags columns holding personal data (NAME, EMAIL, PHONE, ADDRESS…) with a confidence score; generic types such as dates are ignored.

M5 · Dashboard, Reports & Analytics

AWS services: Lambda, DynamoDB, Amplify

- Score card with health and fit-for-analytics flag; status timeline (Uploaded → Processing → Analyzed → ReportGenerated → Completed).
- Six-dimension bar chart, DQ score trend across runs, column quality heatmap, severity and rule coverage.
- Issue list with severity filter (High / Medium / Low) and audit history.
- Export the report as CSV and print it; the run history is kept on the dataset item for trend charts.

M6 · AI Executive Summary

AWS services: Bedrock, Lambda

- Lambda builds a metrics JSON (score, health, dimensions, issue counts, anomalies, top issue messages) — no raw CSV rows.
- Calls the Amazon Bedrock Converse API (Amazon Nova Lite, inference profile us.amazon.nova-lite-v1:0) and asks for JSON with an executive summary and exactly three recommendations.
- The summary is cached on the dataset per analysis run, and the UI shows exactly which metrics the model received.
- If Bedrock is unavailable, a deterministic summary is built from the same metrics so the feature degrades gracefully.

M7 · Alerts & Soft Correction Workflow

AWS services: Lambda, DynamoDB, SNS

- A Business User sends a quality alert with a note; the dataset workflow moves to AlertRaised and the Data User sees it in the inbox.
- Lambda publishes the alert to the Amazon SNS topic clouddq-alerts, which emails subscribers with the score, health, note and a link to the dataset.
- The Data User acknowledges, moves the dataset to correction, re-uploads a fixed CSV and re-runs the analysis.
- Issue lifecycle OPEN → ASSIGNED → IN_PROGRESS → RESOLVED, with reopen and a full audit history.
- An issue can only be resolved when the latest dimension score meets the rule threshold (re-validation).

M8 · API, Data Store & Cloud Deployment

AWS services: API Gateway, Lambda, DynamoDB, CloudWatch, SNS, IAM

- API Gateway REST API (stage prod) with / and /{proxy+} ANY methods routed to one Python 3.12 Lambda by Lambda proxy integration.

- DynamoDB single-table design: PK/SK plus GSI1 for a user's datasets and alerts; on-demand capacity.
- Lambda logs to CloudWatch (/aws/lambda/clouddq-api); alarm clouddq-api-errors fires on any error in 5 minutes and notifies the SNS topic.
- Three IAM users (one per team member) with role-scoped policies and an execution role clouddq-lambda-role for the function.

5. Architecture

CloudDQ AI is fully serverless. The React application is hosted on AWS Amplify. Users sign in to Amazon Cognito and receive a JWT. Every API call goes to Amazon API Gateway, whose Cognito authorizer validates the token before invoking a single AWS Lambda function. Lambda issues presigned S3 URLs, reads uploaded CSVs from Amazon S3, runs the data-quality engine, stores results in Amazon DynamoDB, asks Amazon Bedrock for an executive summary from aggregated metrics, and writes logs to Amazon CloudWatch. IAM controls the team's console access and the function's permissions.

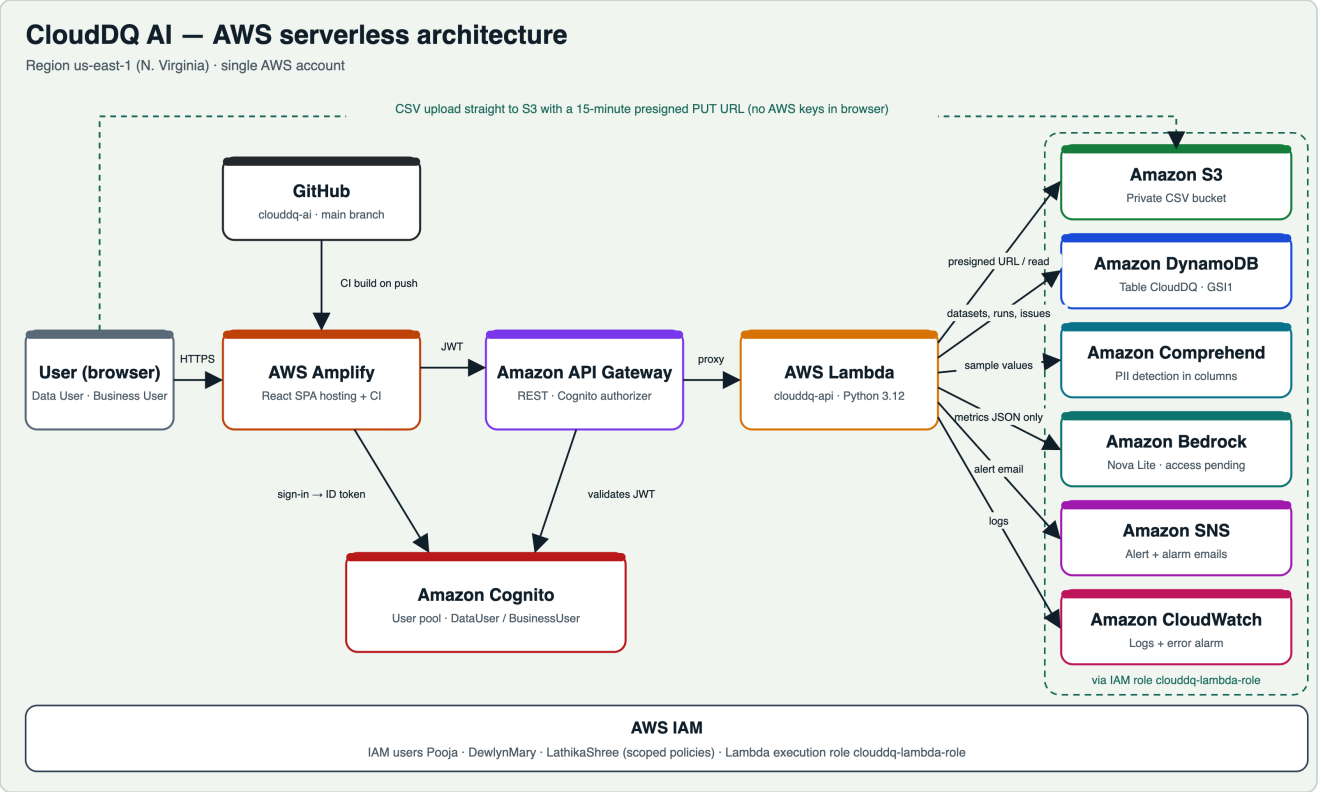


Fig. A1: CloudDQ AI AWS architecture

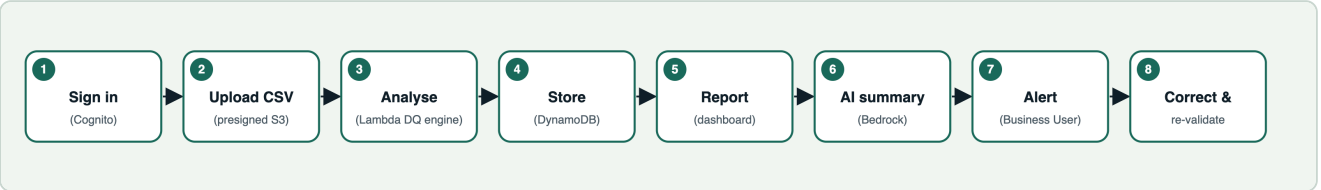


Fig. A2: End-to-end workflow

DynamoDB single-table design

Entity	PK	SK	GSI1
Dataset	`DATASET#<id>`	`META`	`USER#<sub>` / `DATASET#…`
Analysis run	`DATASET#<id>`	`RUN#<iso time>`	—
Issue	`DATASET#<id>`	`ISSUE#<id>`	—

Alert	ALERT#<id>	META	USER#<sub> / ALERT#…

REST API (JWT-protected)

Method	Path	Role
POST	/datasets/upload-url	DataUser
POST	/datasets	DataUser
GET	/datasets, /datasets/{id}	Both
POST	/datasets/{id}/analyze	DataUser
GET	/datasets/{id}/runs · /report · /export · /issues	Both
POST	/datasets/{id}/summary	Both
POST	/datasets/{id}/correction	DataUser
POST / GET	/alerts	BusinessUser / Both
POST	/issues/{id}/assign | acknowledge | resolve | reopen	Mostly DataUser

Security

- Cognito groups for role-based access, enforced again in Lambda.
- JWT validated by API Gateway on every request.
- Private, encrypted S3; uploads by 15-minute presigned PUT URLs; the browser never holds AWS keys.
- Bedrock receives aggregated metrics JSON only — raw CSV rows stay in S3.
- Separate IAM users per team member with scoped policies; no root use for daily work.

6. Individual Contribution

Member	Reg. No.	Focus	Modules
Pooja Palani	[Reg. No.]	Backend, API and AI integration	M3, M4, M6, M8
Dewlyn Mary	[Reg. No.]	Frontend, identity and storage	M1, M2, M3 (upload), M5
Lathika Shree	[Reg. No.]	Operations, monitoring and governance	M6 (model access), M7 testing, M8 (monitoring, IAM)

Pooja Palani — Backend, API and AI integration

Modules: M3, M4, M6, M8

- Created the DynamoDB table CloudDQ with the PK/SK keys and GSI1 index.
- Created the Lambda execution role and the clouddq-api function (Python 3.12, us-east-1); packaged and deployed the backend code and set environment variables.
- Built the REST API in API Gateway: Cognito authorizer, / and /{proxy+} routes, CORS, deployment to the prod stage.
- Implemented the Bedrock Converse call and the metrics-only payload for the AI summary.
- Integrated Amazon Comprehend PII detection into the analysis and added its IAM permission to the Lambda role.
- Ran the first Lambda test so CloudWatch logging started.

Dewlyn Mary — Frontend, identity and storage

Modules: M1, M2, M3 (upload), M5

- Created the Cognito user pool, public SPA app client, DataUser / BusinessUser groups and demo users.
- Created the private, encrypted S3 bucket and its CORS rule for browser uploads.
- Set up AWS Amplify Hosting from GitHub, the monorepo build specification and the VITE_ environment variables.
- Connected the frontend sign-in (Cognito) and API configuration; added the Amplify URL to Cognito and S3 CORS.

Lathika Shree — Operations, monitoring and governance

Modules: M6 (model access), M7 testing, M8 (monitoring, IAM)

- Checked Amazon Bedrock model availability (Nova Lite) and the inference profile used by Lambda.
- Verified the Amplify environment variables before release.
- Confirmed the CloudWatch log group and created the clouddq-api-errors alarm.
- Created the Amazon SNS topic clouddq-alerts for alert emails and connected the CloudWatch alarm to it.
- Reviewed IAM users and policies for least privilege.
- Tested the end-to-end alert and correction workflow with both roles.

All three — Shared

Modules: —

- Requirement analysis, architecture design, DQ dimension and rule design, testing, documentation and the review presentation.

7. Tools / Software Requirements

Category	Tools
Languages	Python 3.12 (backend), TypeScript (frontend)
Frontend	React 19, Vite 8, Tailwind CSS 4, React Router 7, Recharts 3, Axios, amazon-cognito-identity-js
Backend	boto3 / botocore (AWS SDK for Python), Python standard library (csv, re, statistics)
AWS tooling	AWS Management Console, AWS CLI v2, AWS SAM (template.yaml)
Dev tools	Cursor / VS Code, Git and GitHub, Node.js 20+, npm, Postman, Google Chrome DevTools
Testing	Python unittest/pytest for the DQ engine; local API server (backend/local_server.py)
Hardware	Any laptop with 8 GB RAM and internet access; AWS account (region us-east-1)

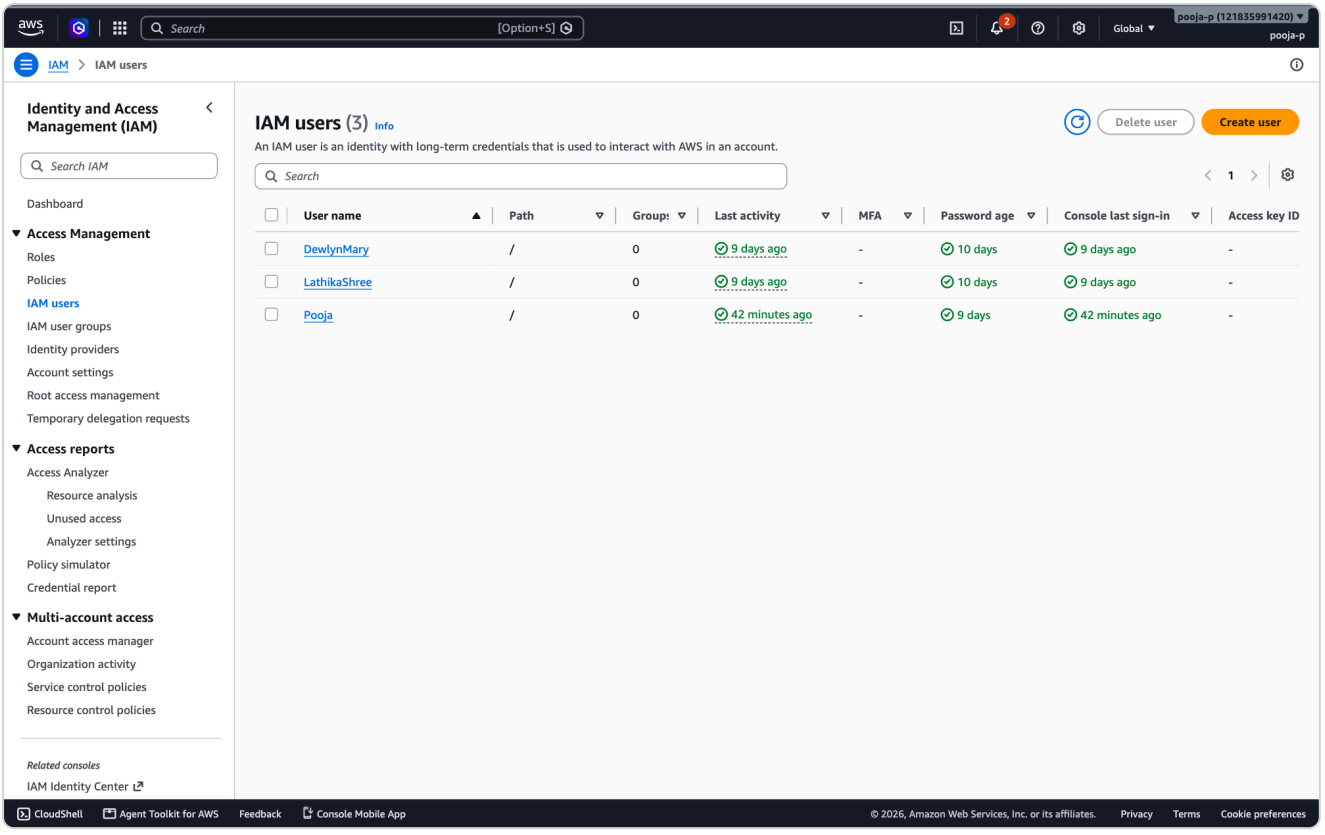
8. AWS Services Used

Ten application services plus IAM, all in region **us-east-1** (N. Virginia). Nine run end to end, including the AI service Amazon Comprehend; Amazon Bedrock is integrated in code and waiting for AWS to enable model access on the account.

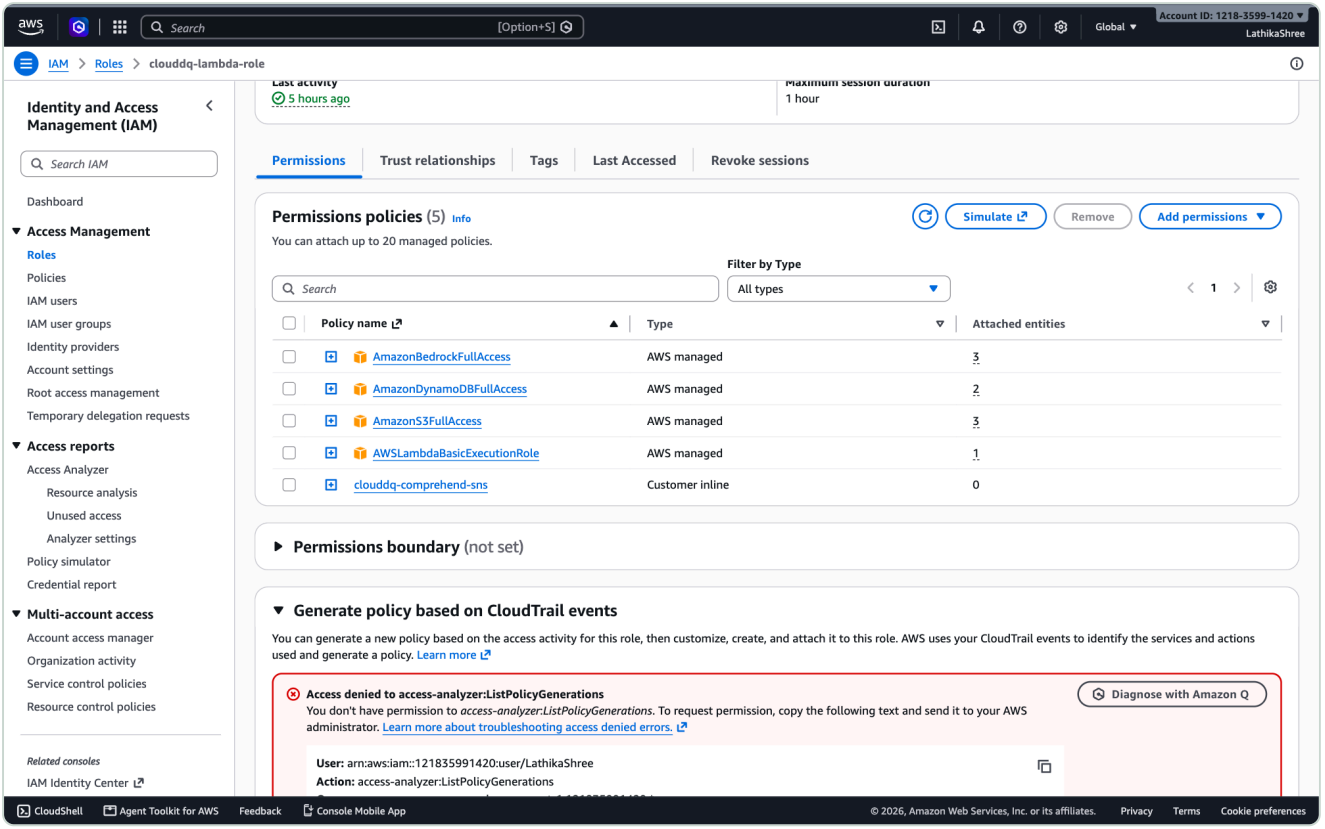
Service	Use in CloudDQ AI	Resource
1. AWS Amplify	Builds and hosts the React app from GitHub; injects VITE_ configuration at build time.	`clouddq-ai · main branch`
2. Amazon Cognito	User pool for sign-in; DataUser / BusinessUser groups; issues the JWT used by the API.	`us-east-1_DT3M9ZhMz`
3. Amazon API Gateway	REST API with a Cognito authorizer; forwards requests to Lambda (proxy).	`fdro0463k8 · stage prod`
4. AWS Lambda	Python 3.12 backend: routing, DQ engine, alerts, presigned URLs, AI integrations.	`clouddq-api`
5. Amazon S3	Private, encrypted storage for uploaded CSVs; presigned PUT uploads.	`clouddq-datasets-121835991420`
6. Amazon DynamoDB	Single table for datasets, runs, issues, alerts; GSI1 for per-user queries.	`CloudDQ`
7. Amazon Comprehend	AI service: detects personal data (PII) in sample column values during analysis.	`DetectPiiEntities`
8. Amazon SNS	Emails business alerts to the Data User and CloudWatch alarm notifications.	`clouddq-alerts`
9. Amazon CloudWatch	Lambda logs and error alarm for monitoring.	`/aws/lambda/clouddq-api`
10. Amazon Bedrock	Generative AI executive summary from aggregated metrics (Converse API); account access pending with AWS.	`us.amazon.nova-lite-v1:0`
11. AWS IAM	Team IAM users with scoped policies; Lambda execution role.	`clouddq-lambda-role`

9. Screenshots of AWS Services

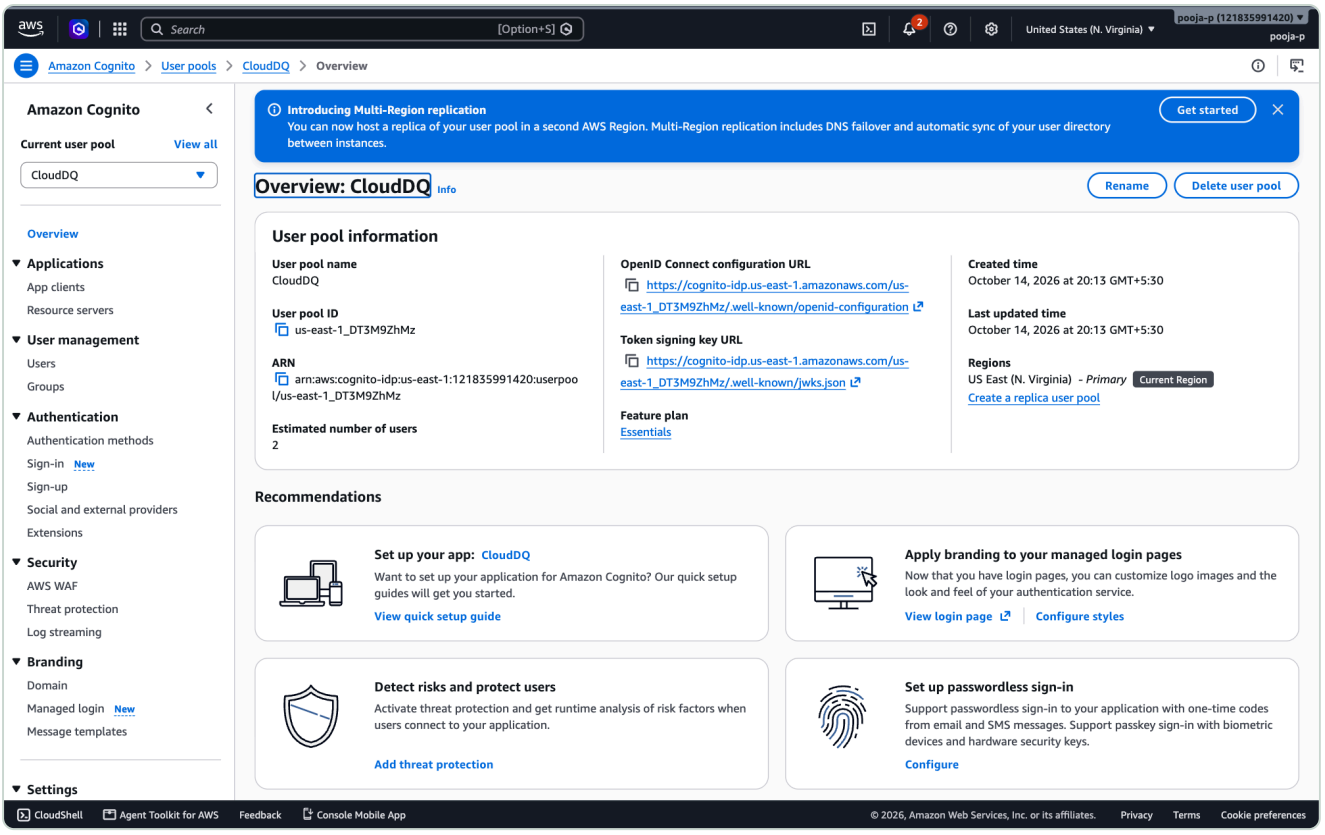
Console screenshots of each configured service.



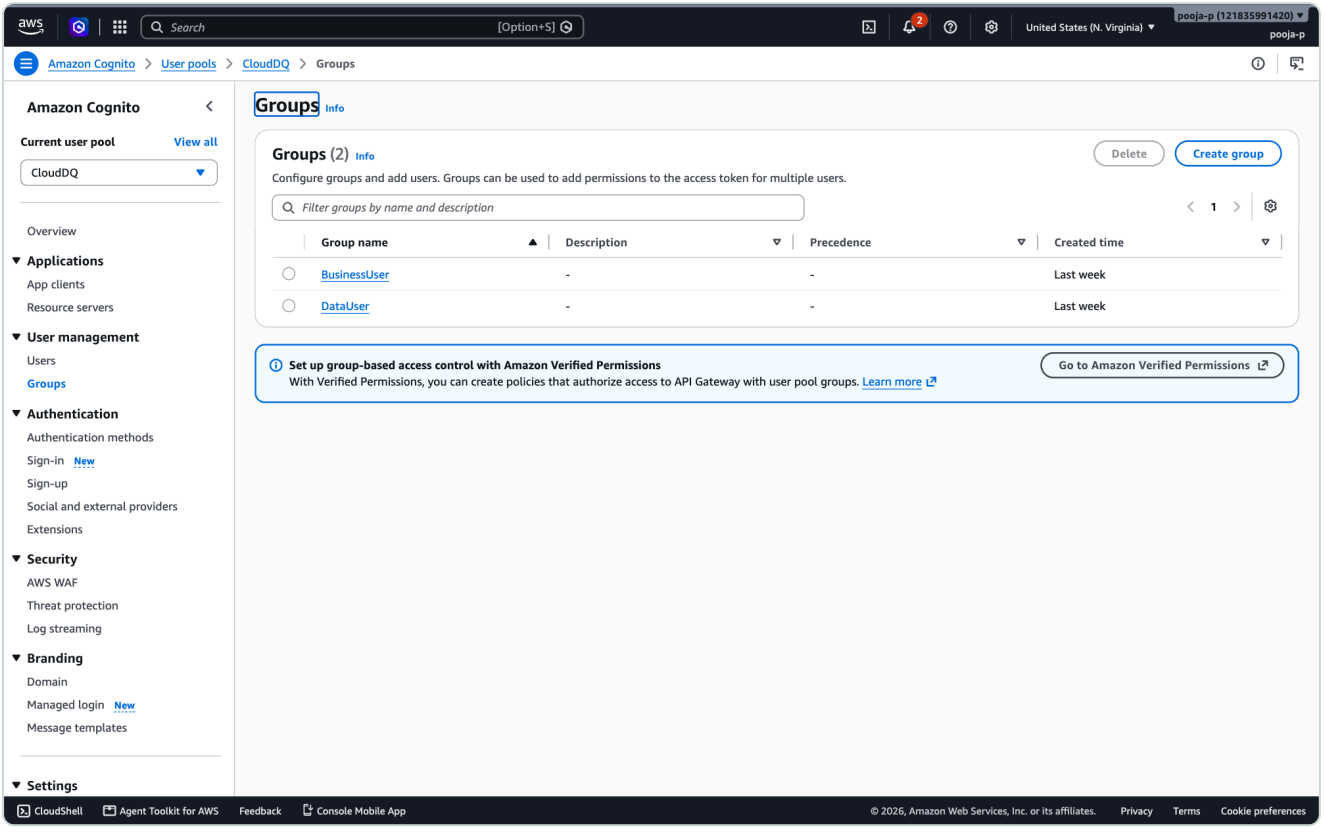
AWS IAM — IAM → Users: Pooja, DewlynMary, LathikaShree



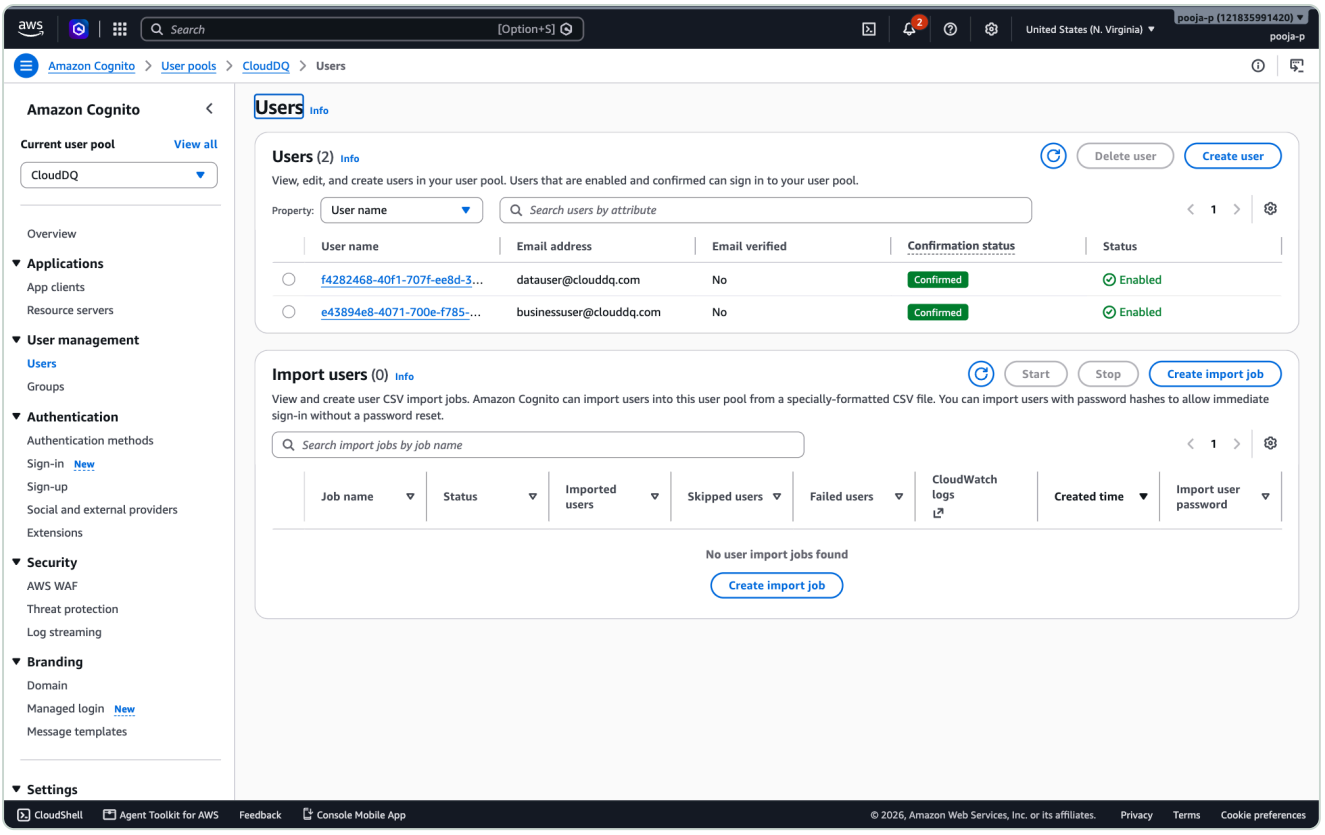
AWS IAM — Roles → clouddq-lambda-role → Permissions



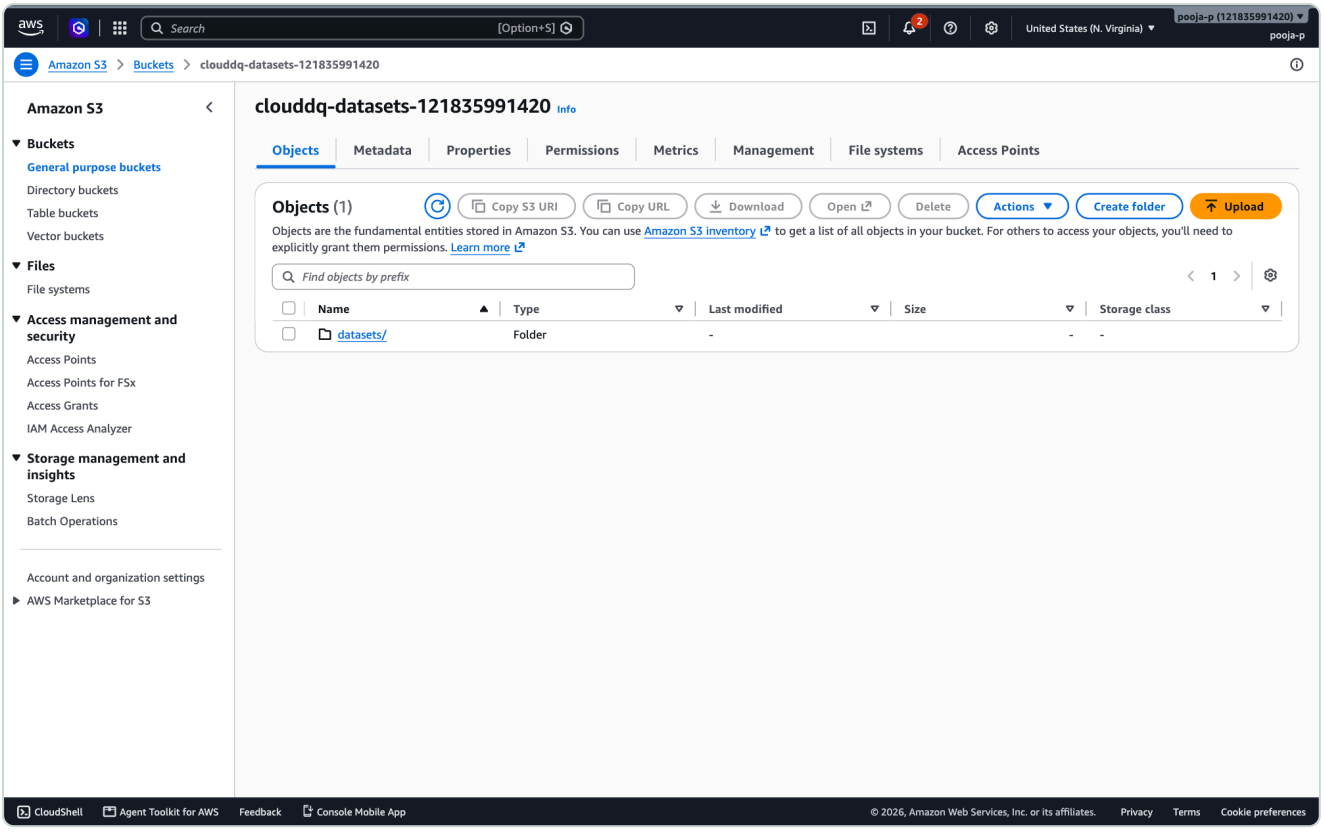
Amazon Cognito — User pool us-east-1_DT3M9ZhMz → Overview



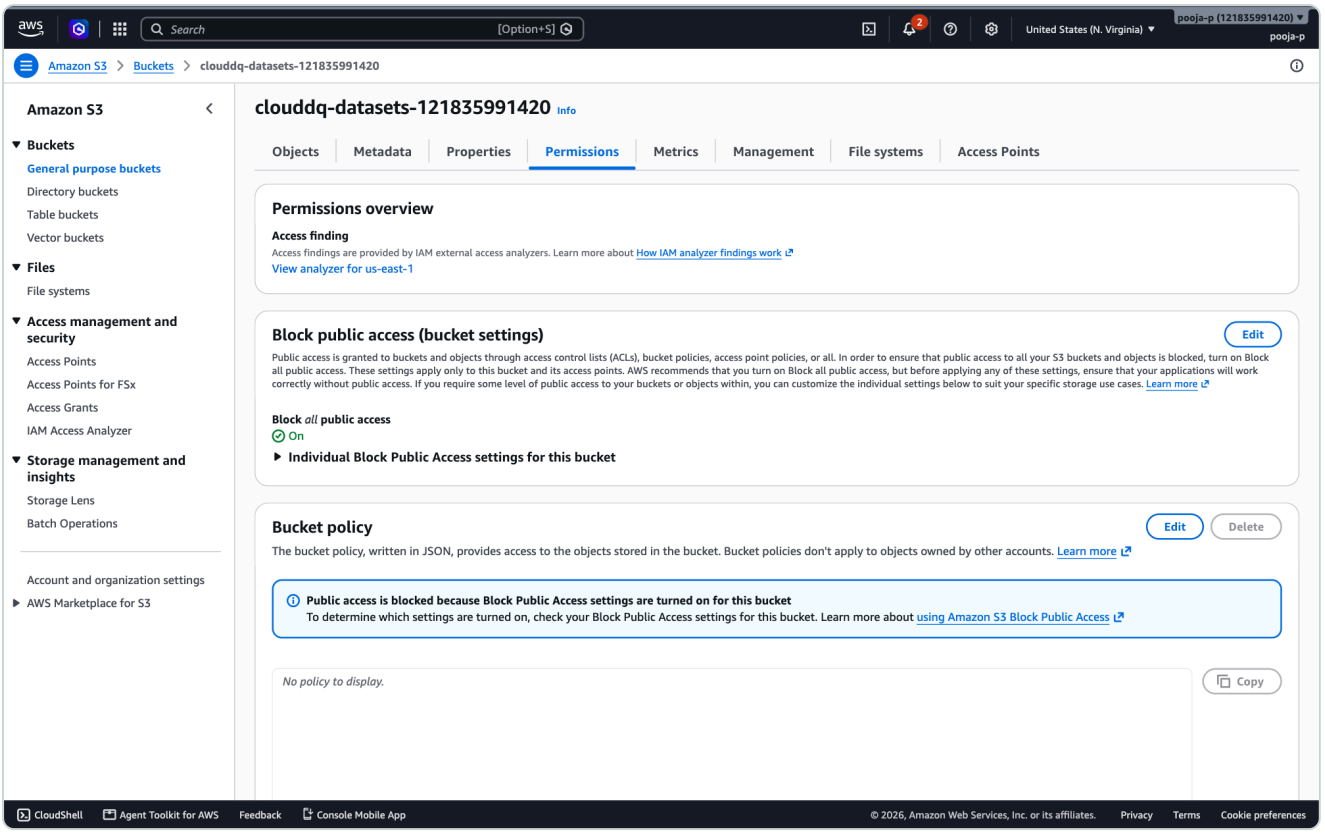
Amazon Cognito — User pool → Groups: DataUser, BusinessUser



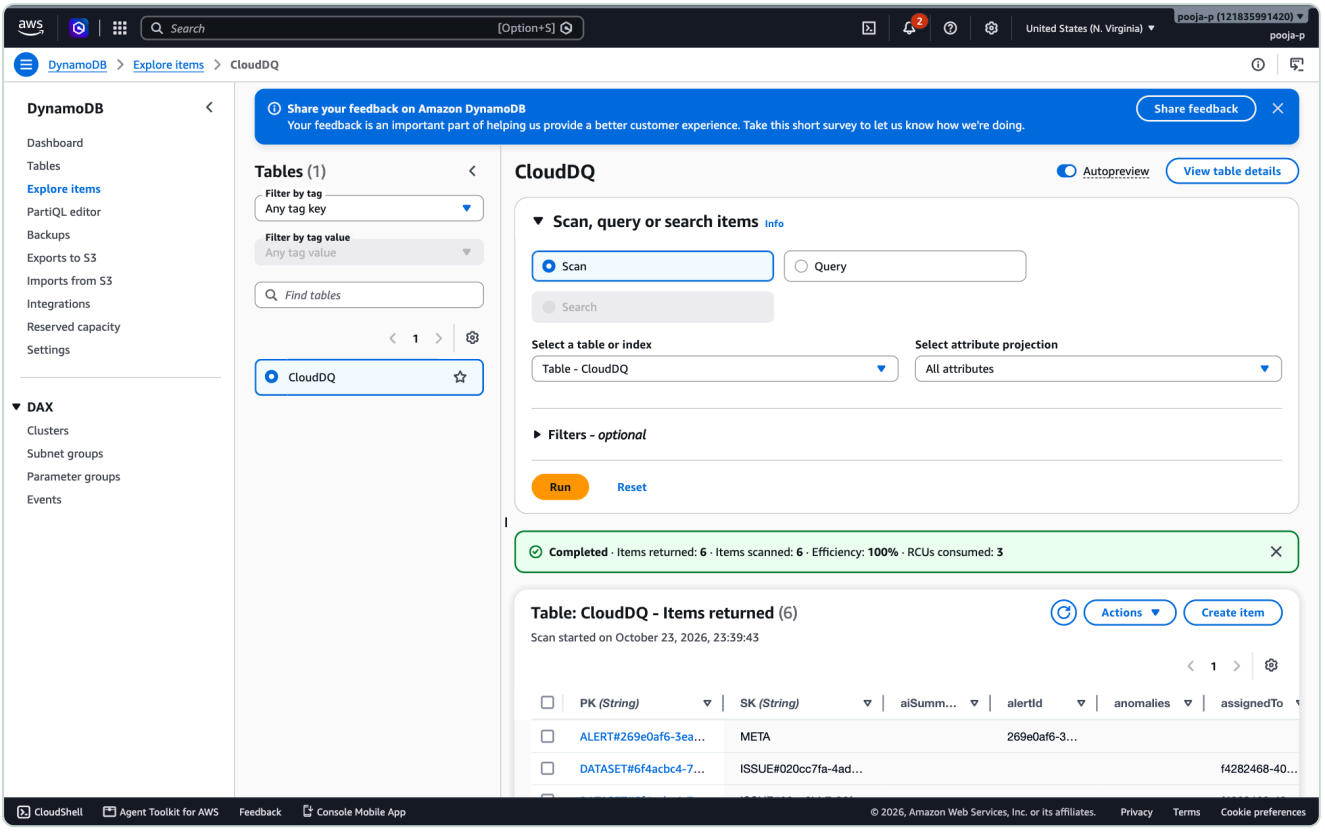
Amazon Cognito — User pool → Users (demo Data User and Business User)



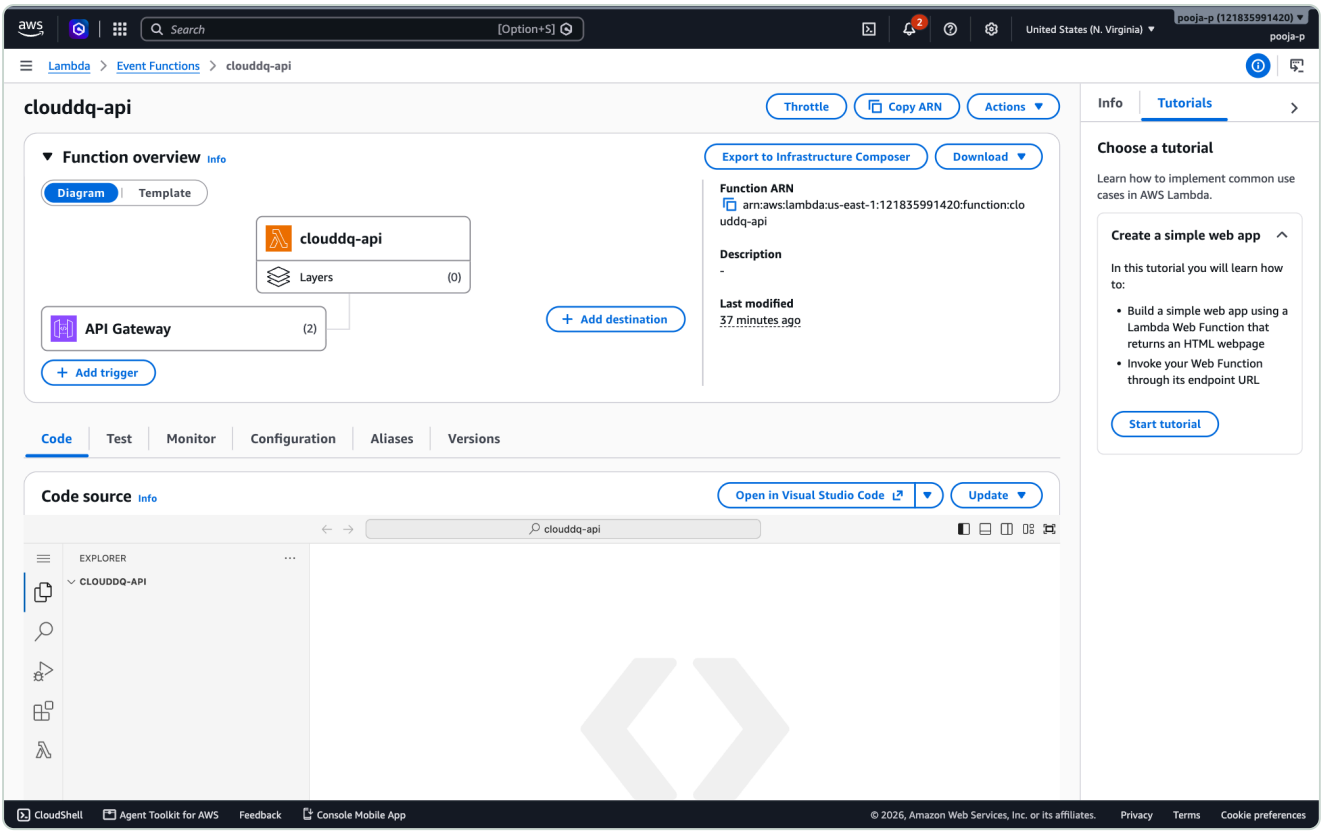
Amazon S3 — Bucket clouddq-datasets-121835991420 → Objects



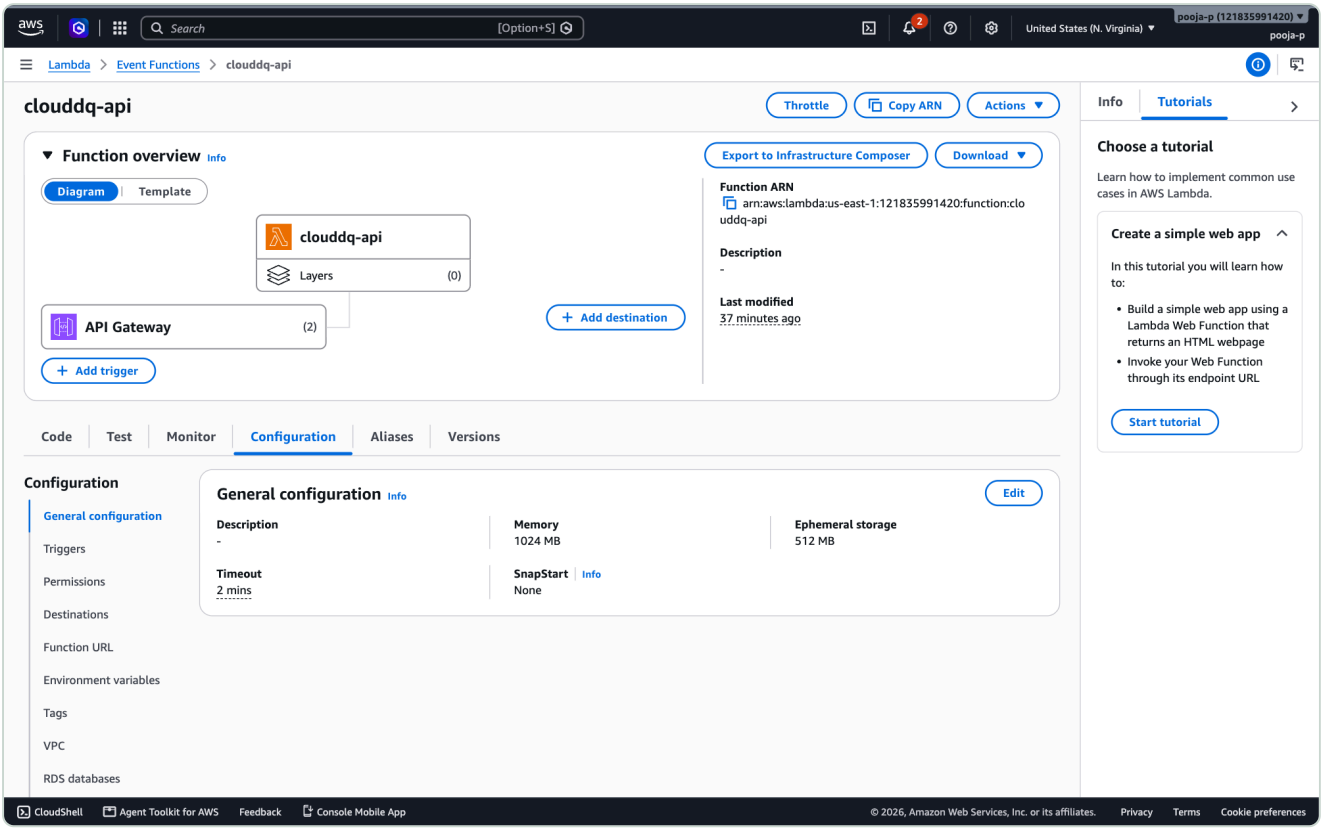
Amazon S3 — Bucket → Permissions: Block all public access On



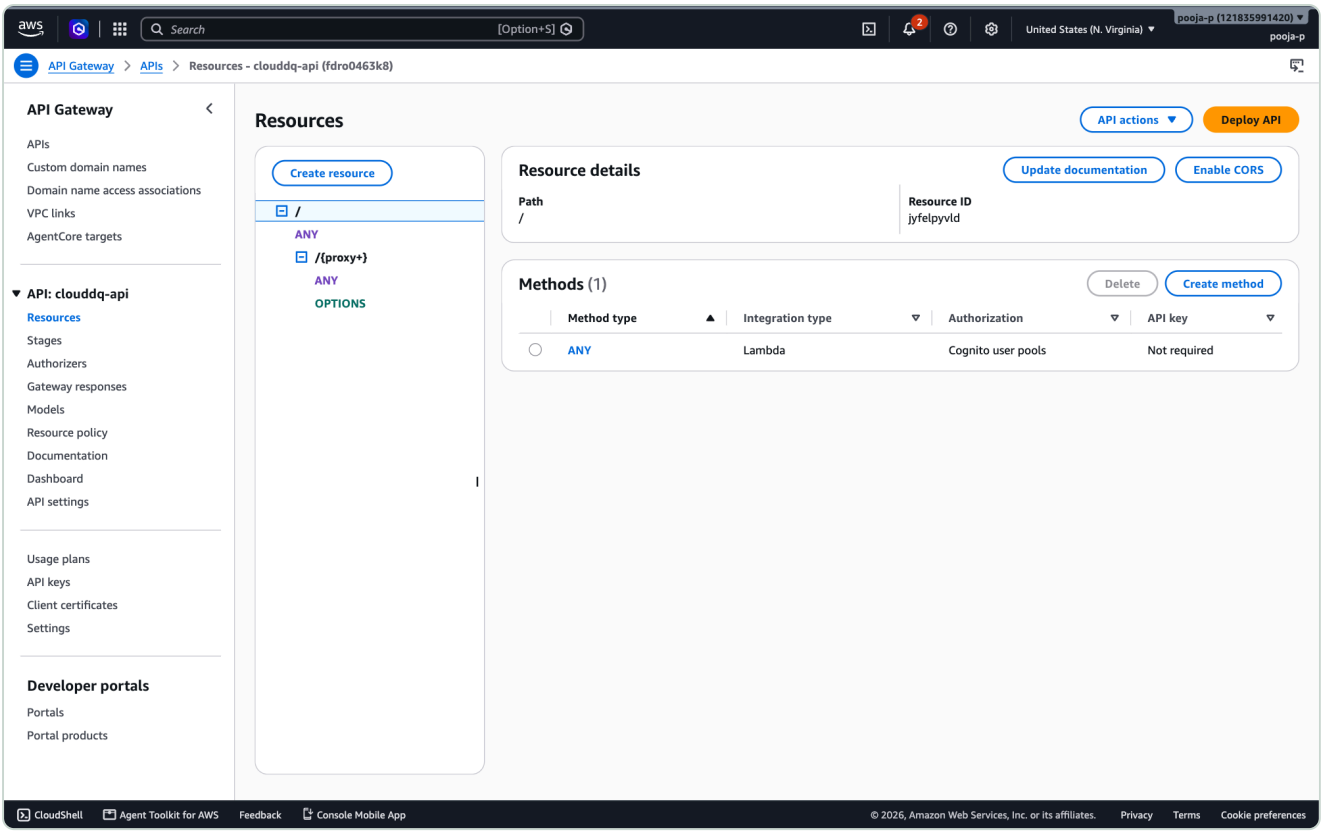
Amazon DynamoDB — Table CloudDQ → Explore items



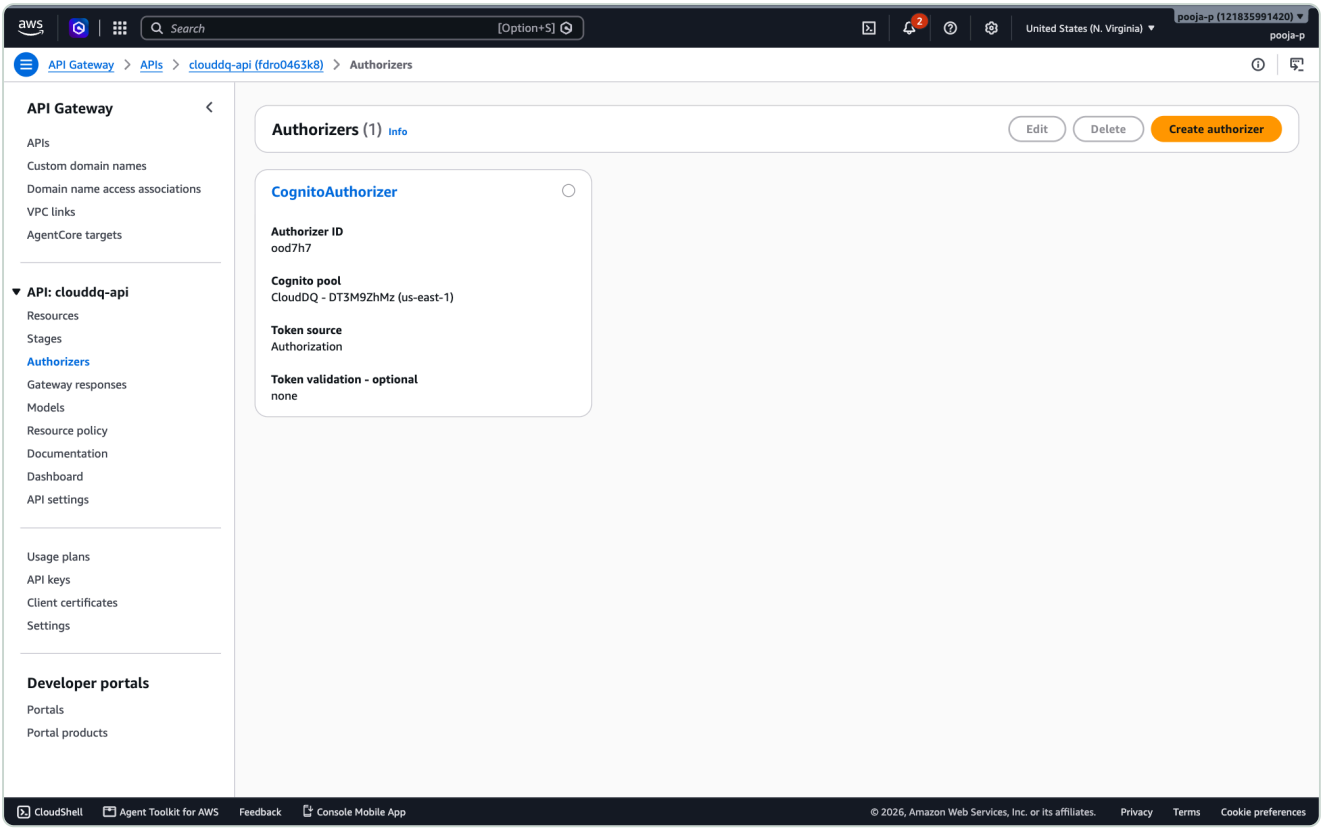
AWS Lambda — Function clouddq-api → overview with API Gateway trigger



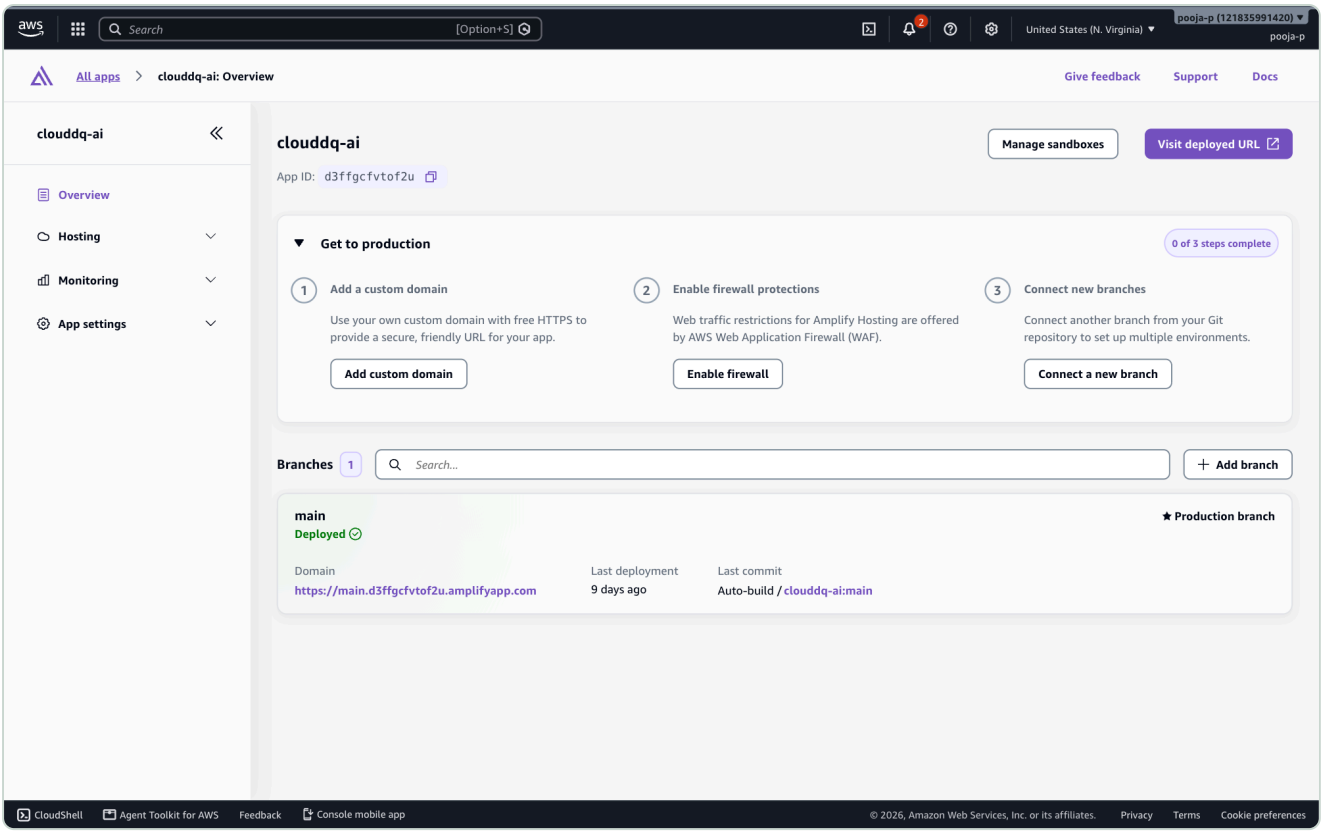
AWS Lambda — Function clouddq-api → Configuration



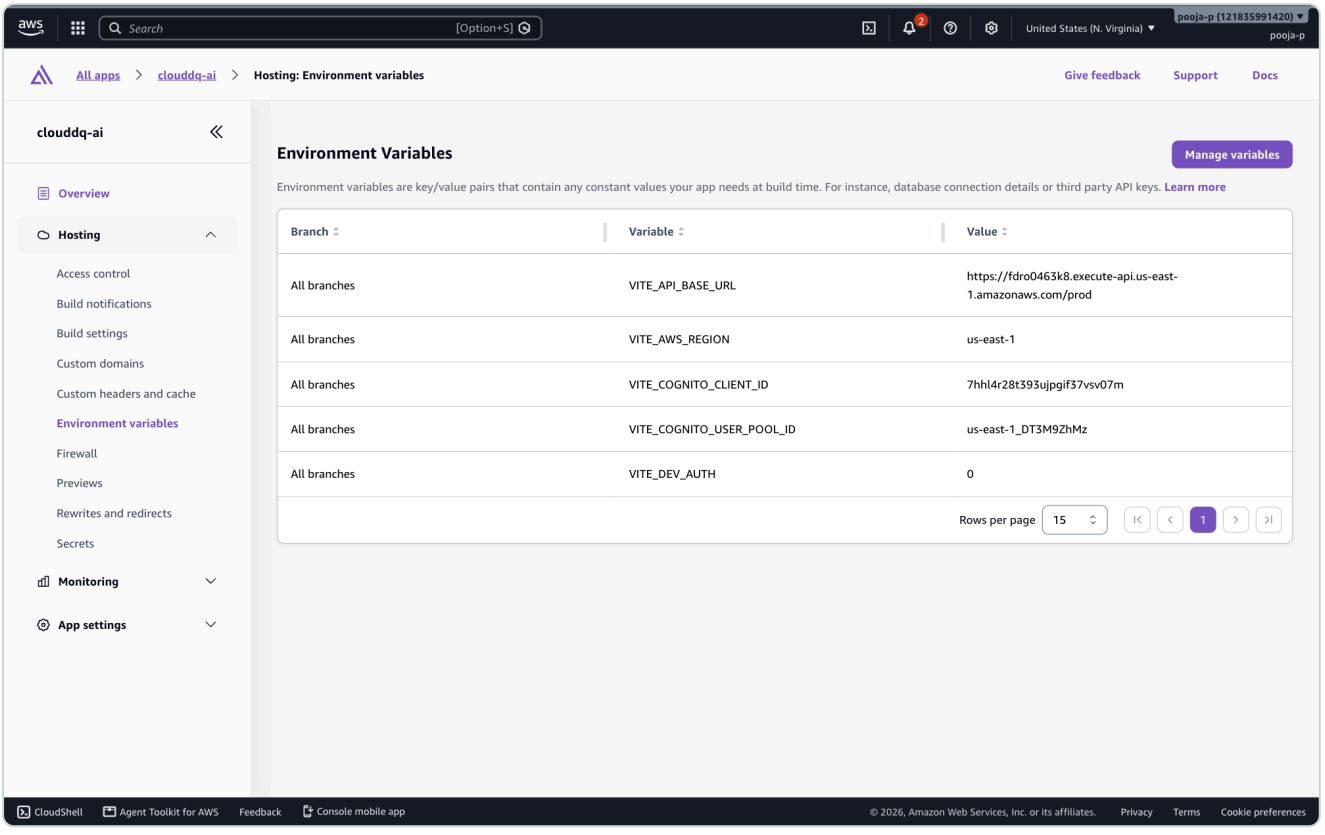
Amazon API Gateway — REST API fdro0463k8 → Resources: / and /{proxy+} ANY



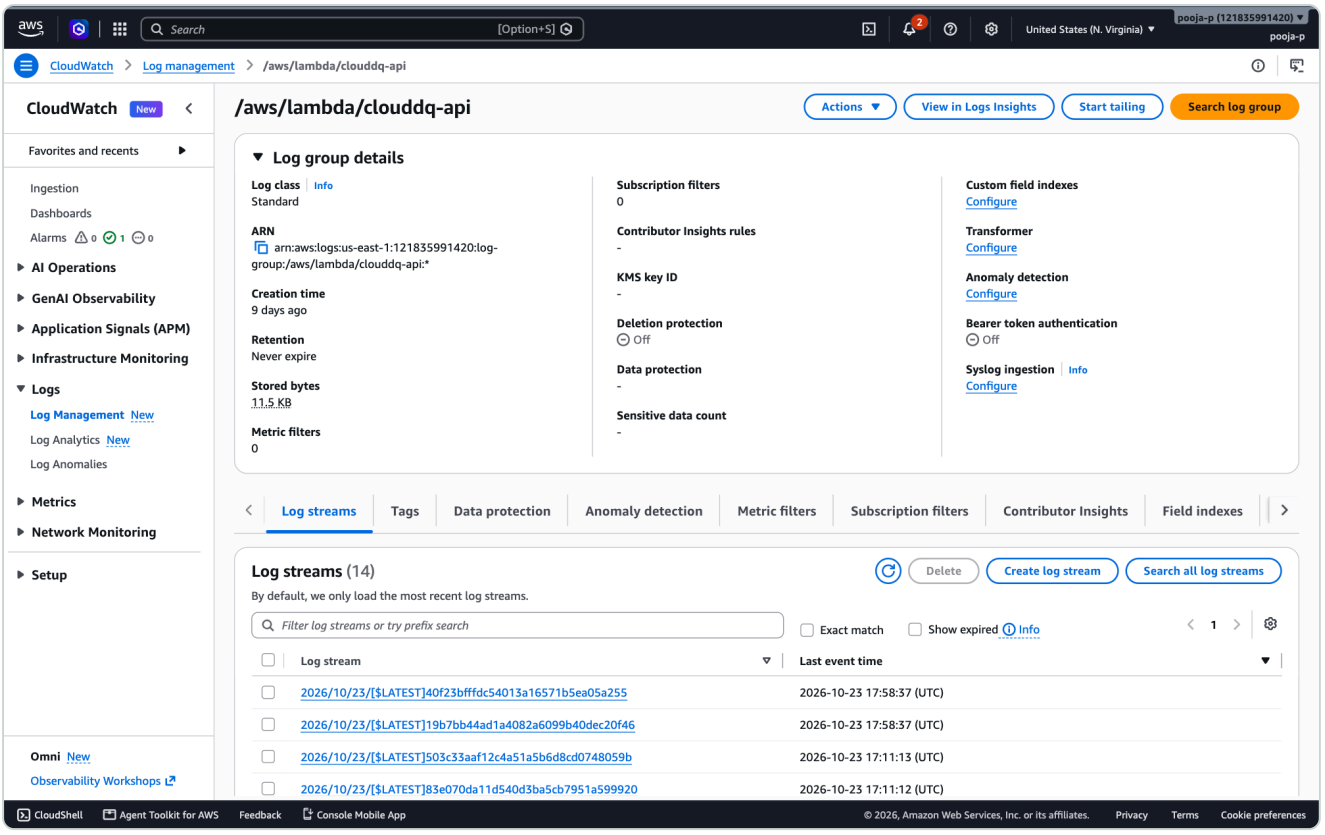
Amazon API Gateway — REST API → Authorizers: CognitoAuthorizer



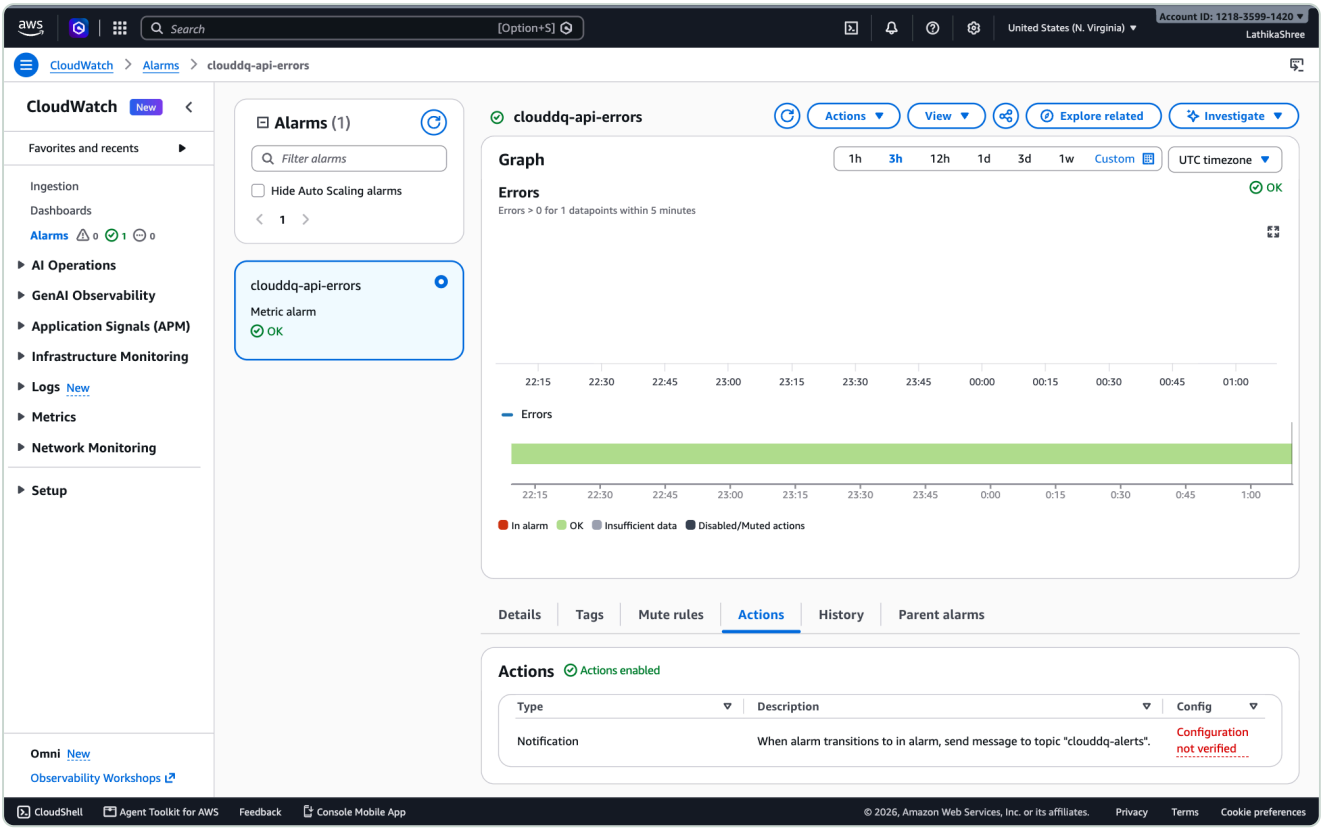
AWS Amplify — App clouddq-ai → Overview (main branch deployed)



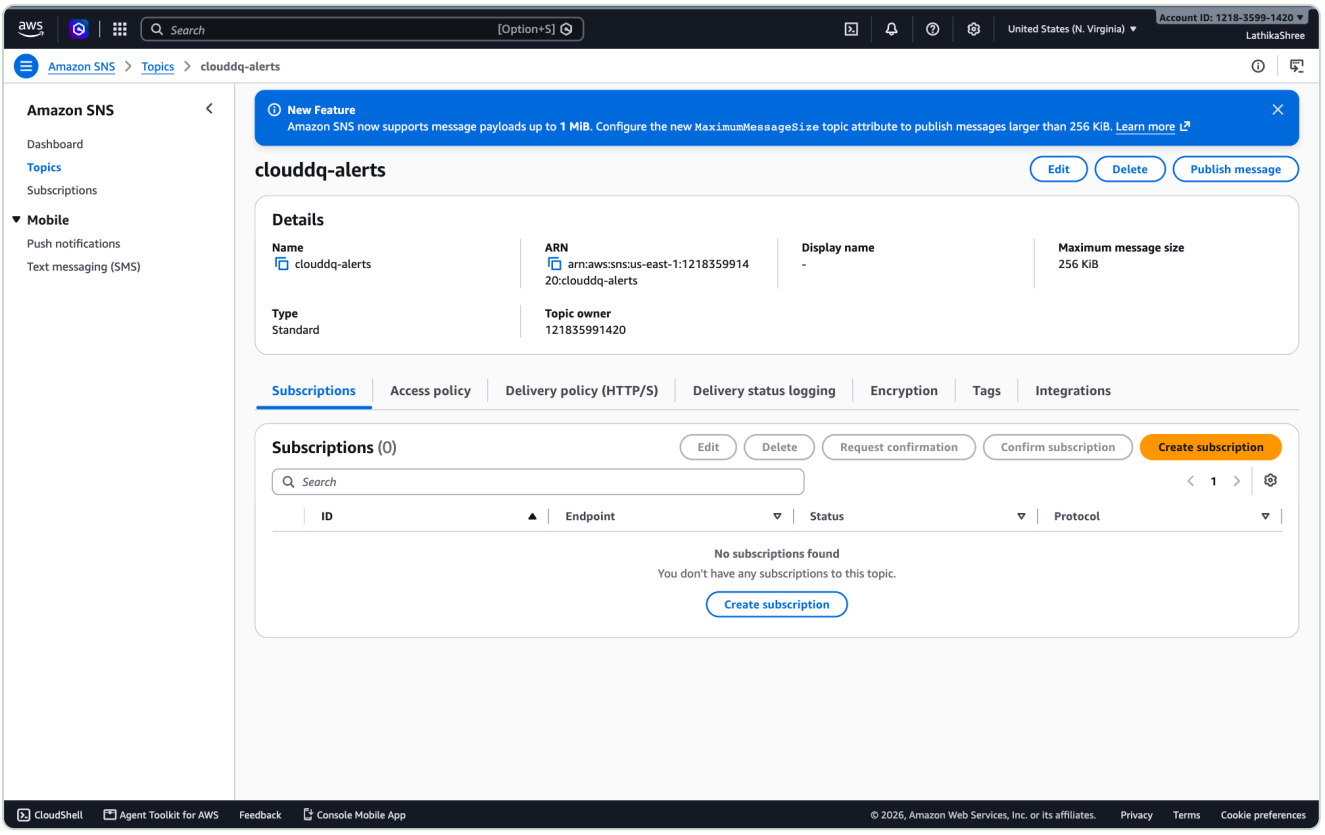
AWS Amplify — App clouddq-ai → Environment variables



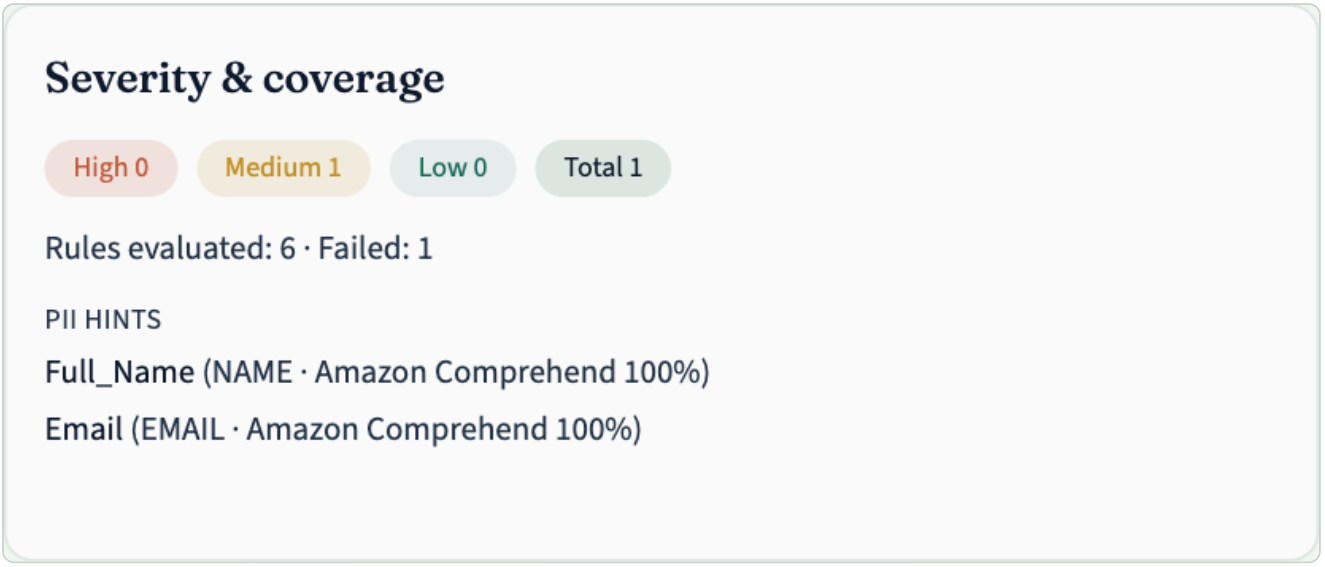
Amazon CloudWatch — Log group /aws/lambda/clouddq-api



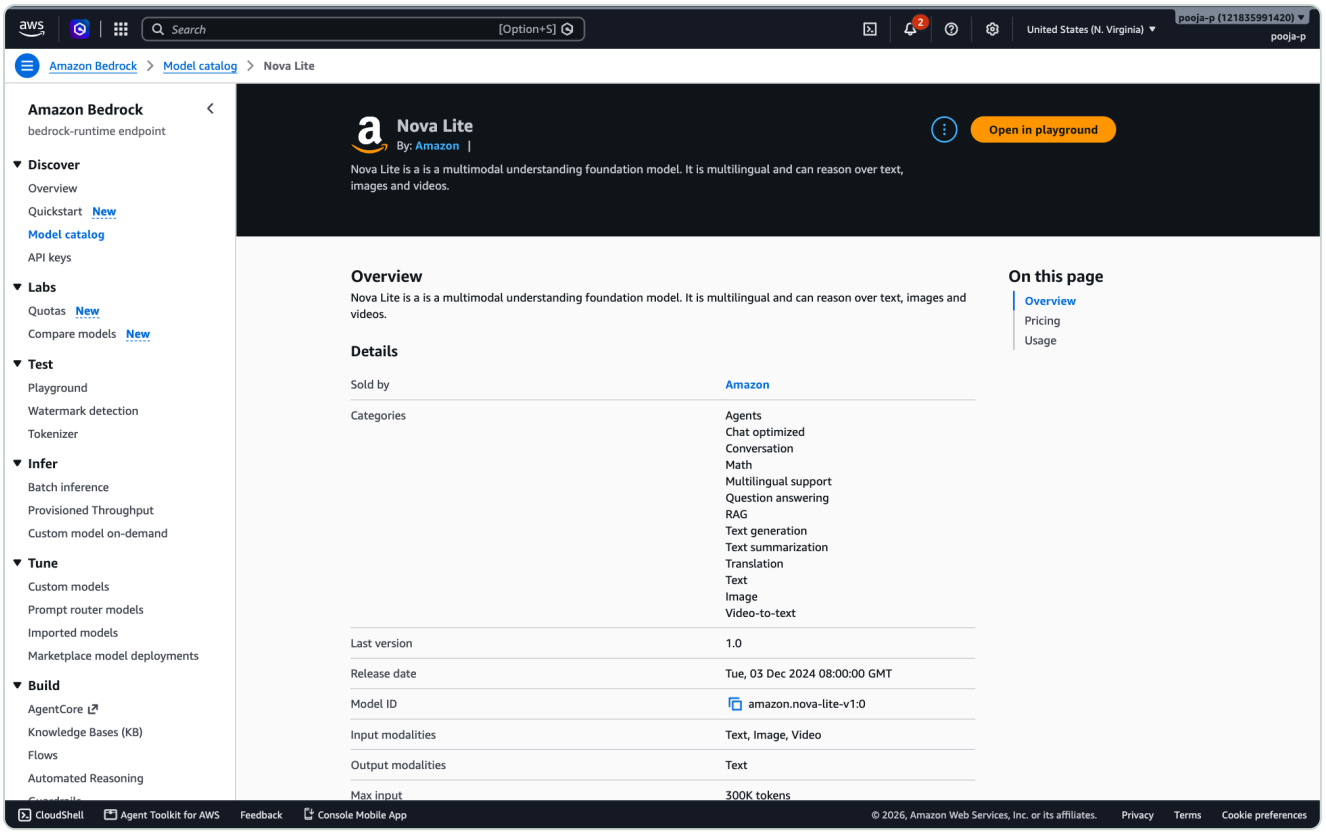
Amazon CloudWatch — Alarm clouddq-api-errors



Amazon SNS — Topic clouddq-alerts (alert emails and alarm notifications)



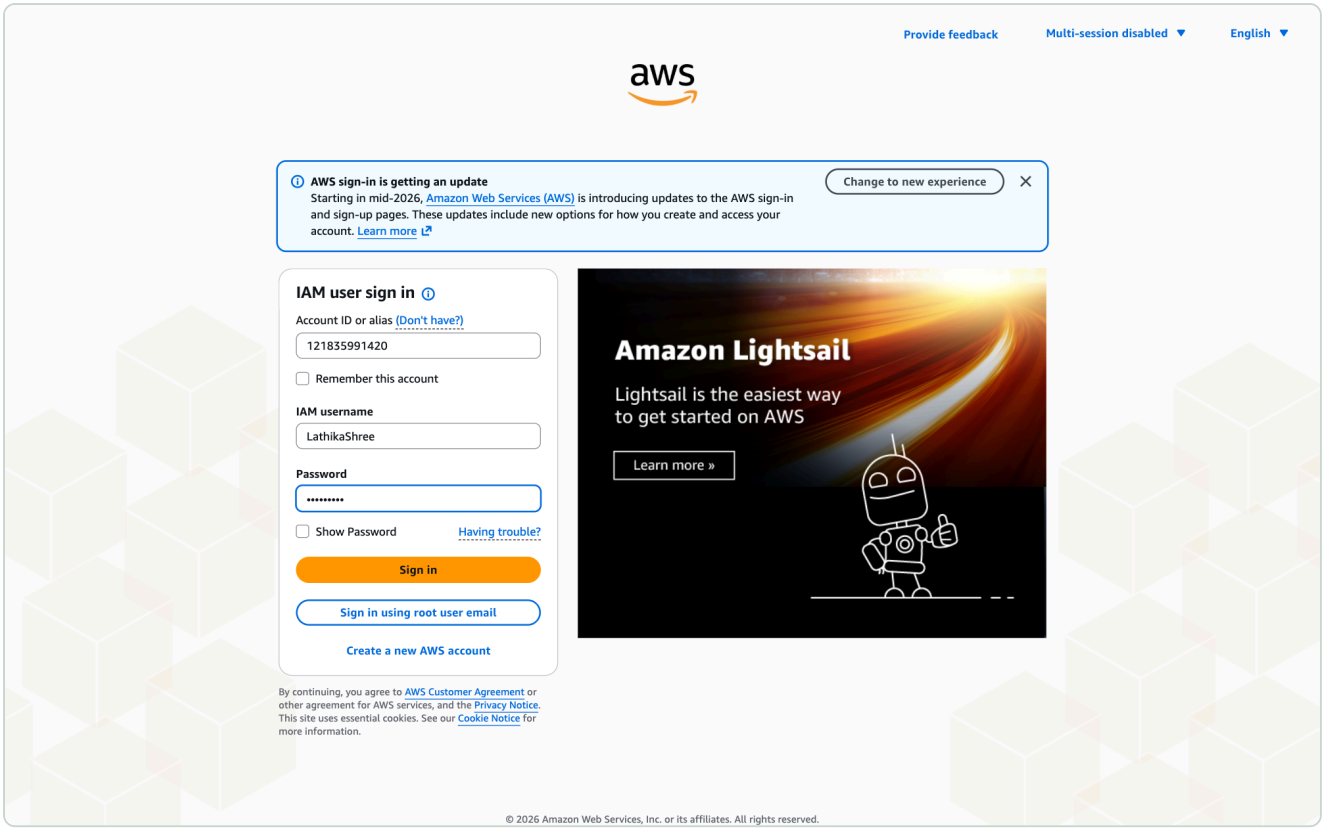
Amazon Comprehend — PII detection result inside CloudDQ (Employee Directory dataset)



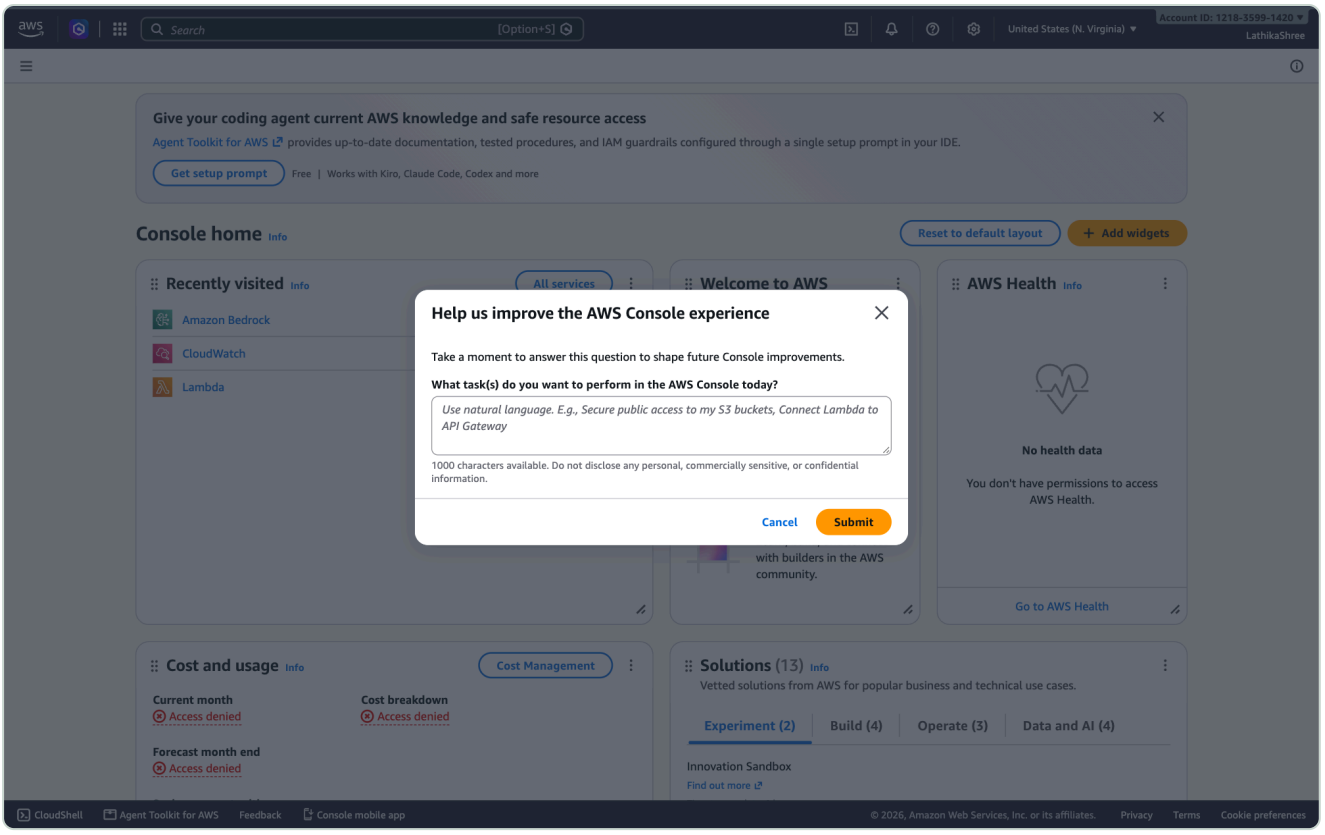
Amazon Bedrock — Model catalog → Amazon Nova Lite

Walkthrough — Lathika connects Amazon SNS to CloudWatch

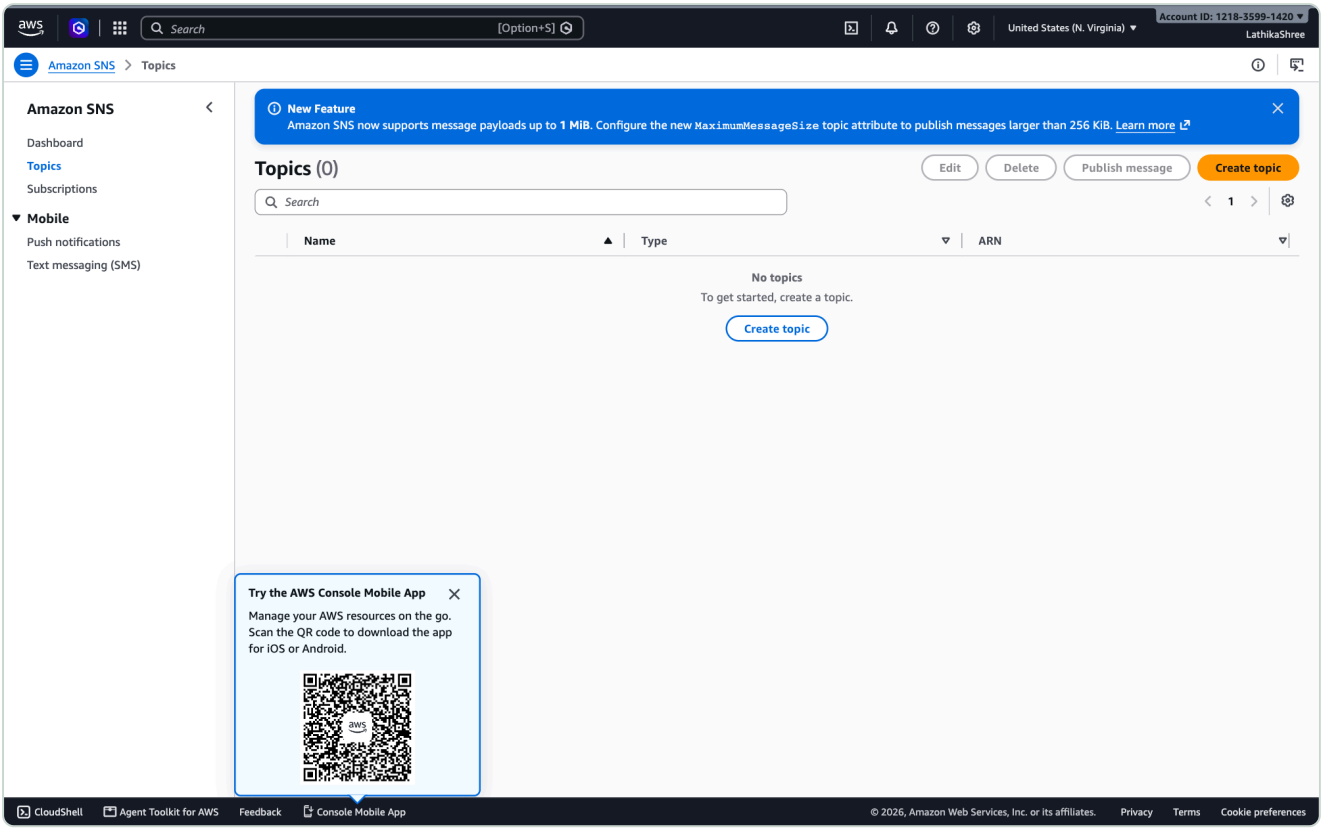
Recorded from the LathikaShree IAM user: create the SNS topic, then add it as the notification action of the clouddq-api-errors alarm. Email subscribers are added on the topic page (step 5) and must confirm the email AWS sends.



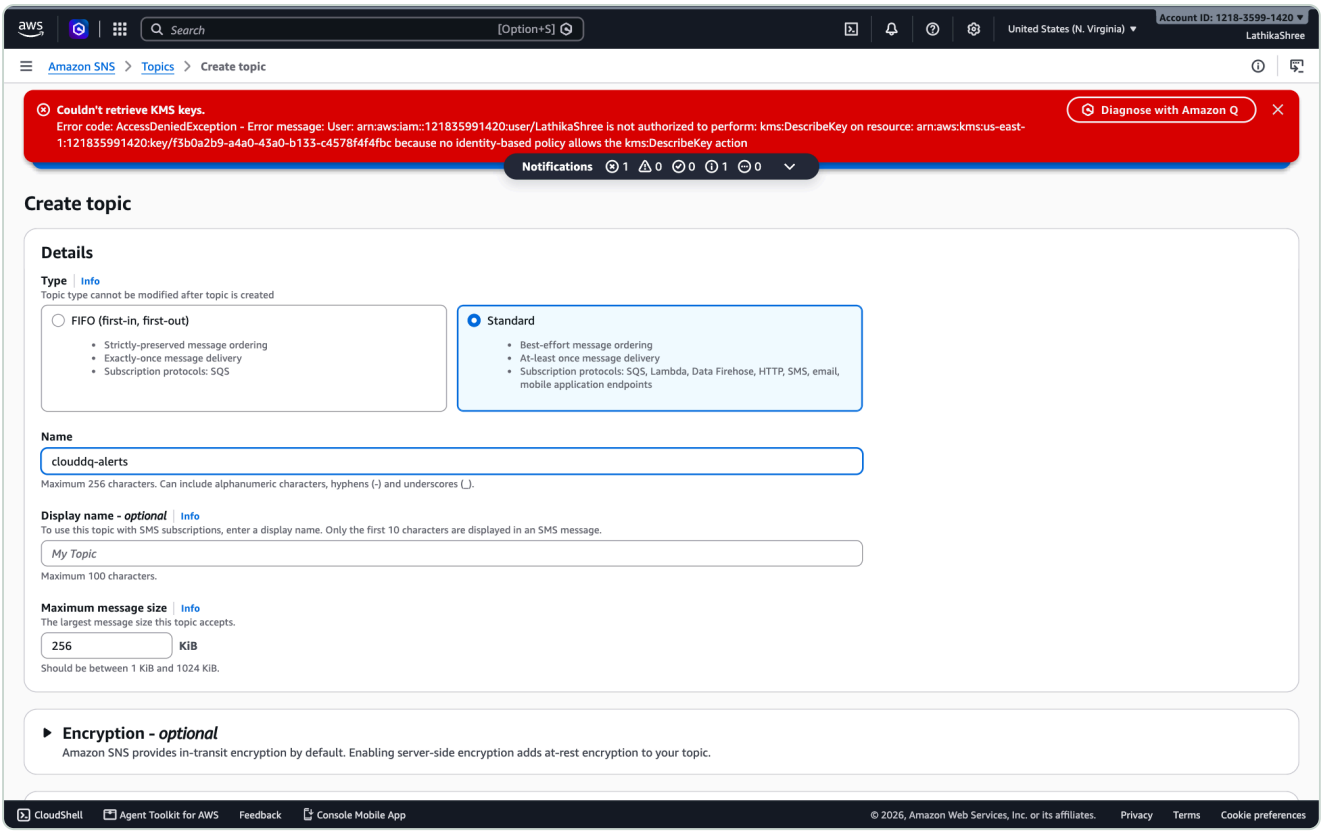
Step 1: Sign in as LathikaShree — IAM user sign-in with account 121835991420 and username LathikaShree.



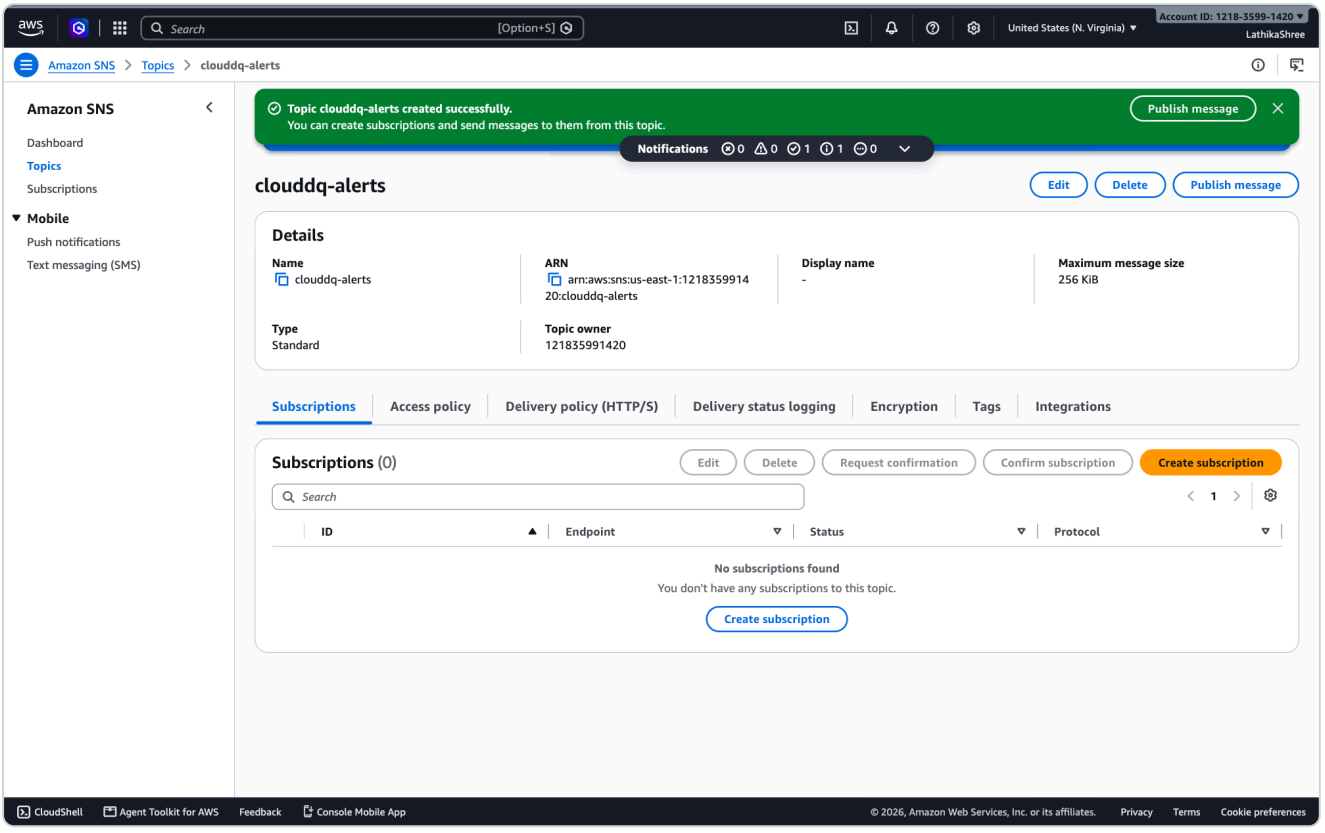
Step 2: Console home — Signed in as LathikaShree in US East (N. Virginia).



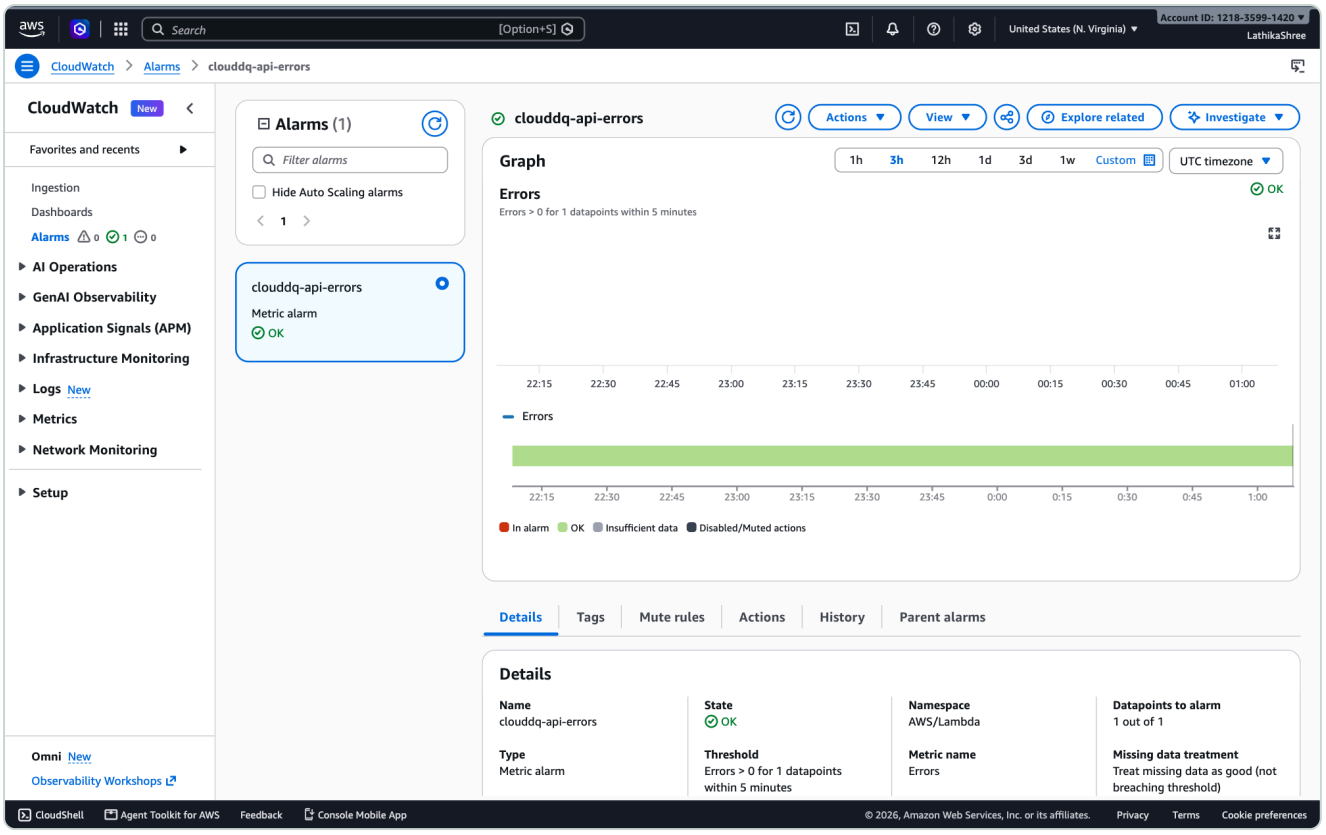
Step 3: Amazon SNS → Topics — No topics yet. Choose Create topic.



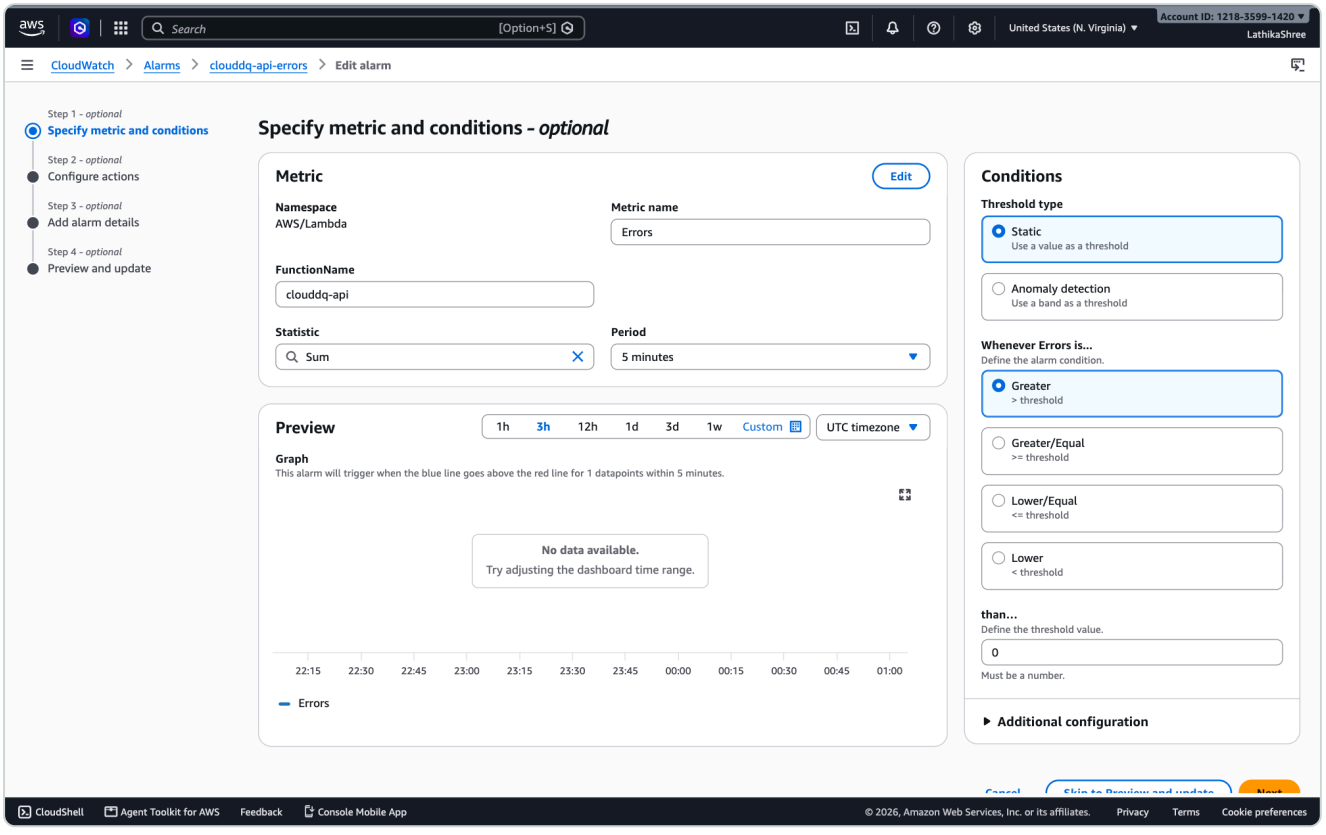
Step 4: Create topic — Type Standard, name clouddq-alerts. The red KMS banner only means Lathika can't list encryption keys; this topic uses default encryption, so it can be ignored.



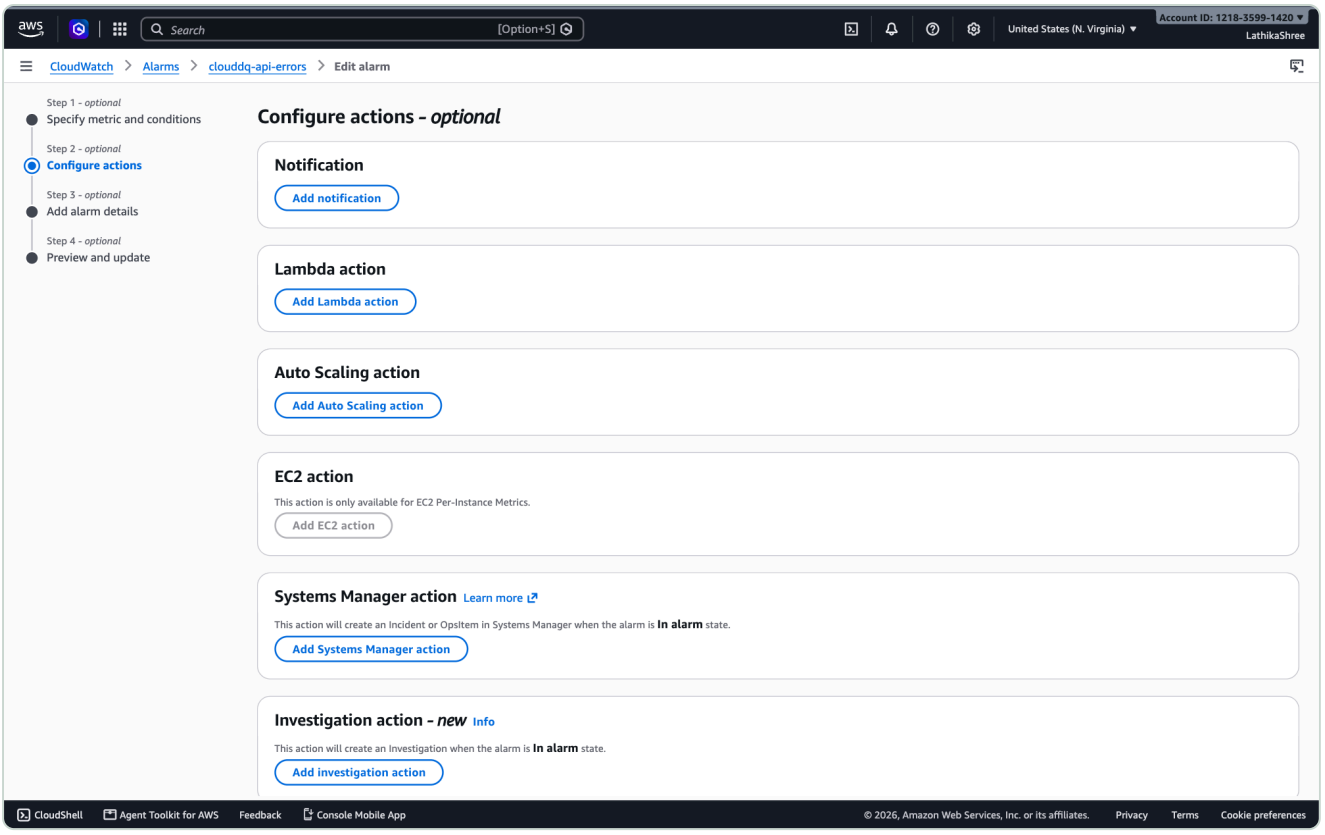
Step 5: Topic created — ARN arn:aws:sns:us-east-1:121835991420:clouddq-alerts — the same ARN Lambda uses in ALERT_TOPIC_ARN. Subscriptions are added from this page.



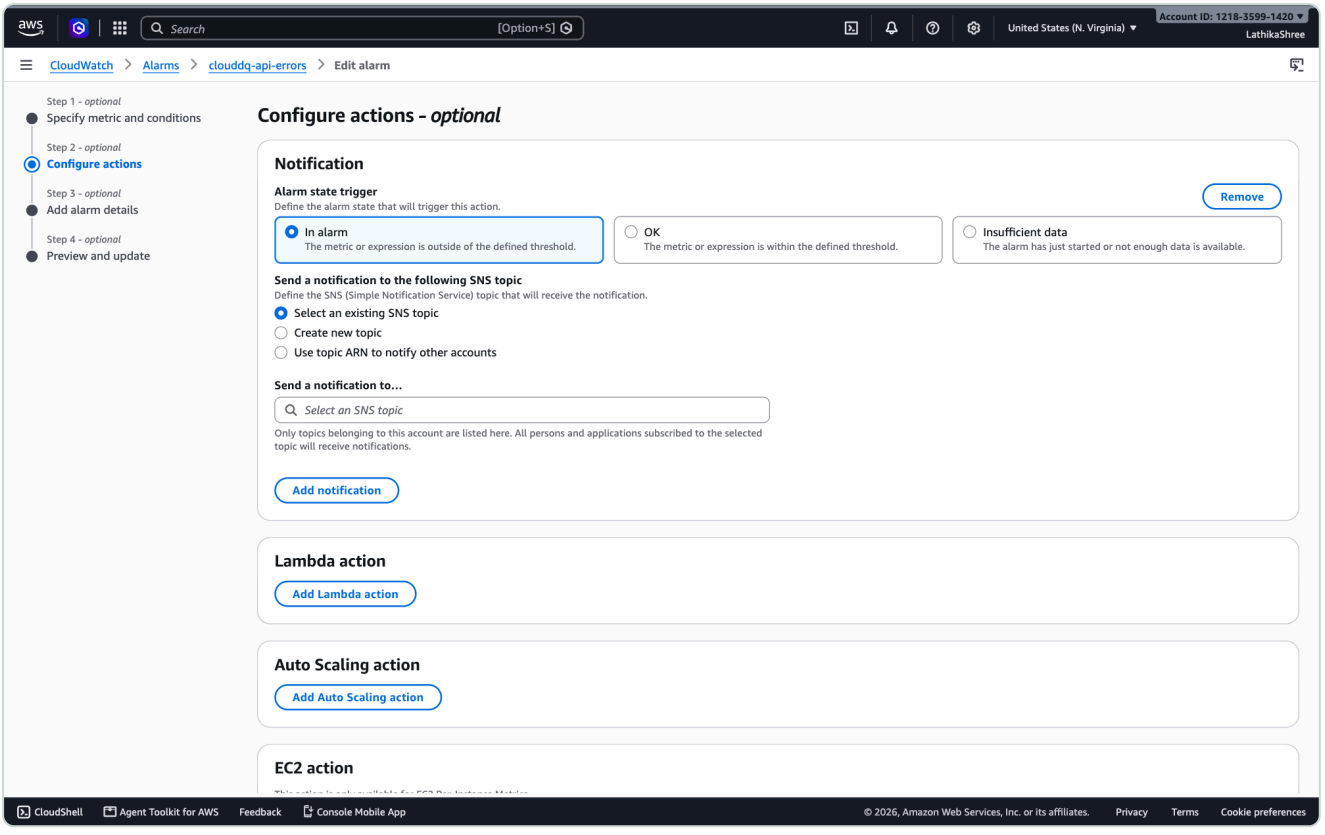
Step 6: CloudWatch → Alarms → clouddq-api-errors — Open the alarm, then Actions → Edit.



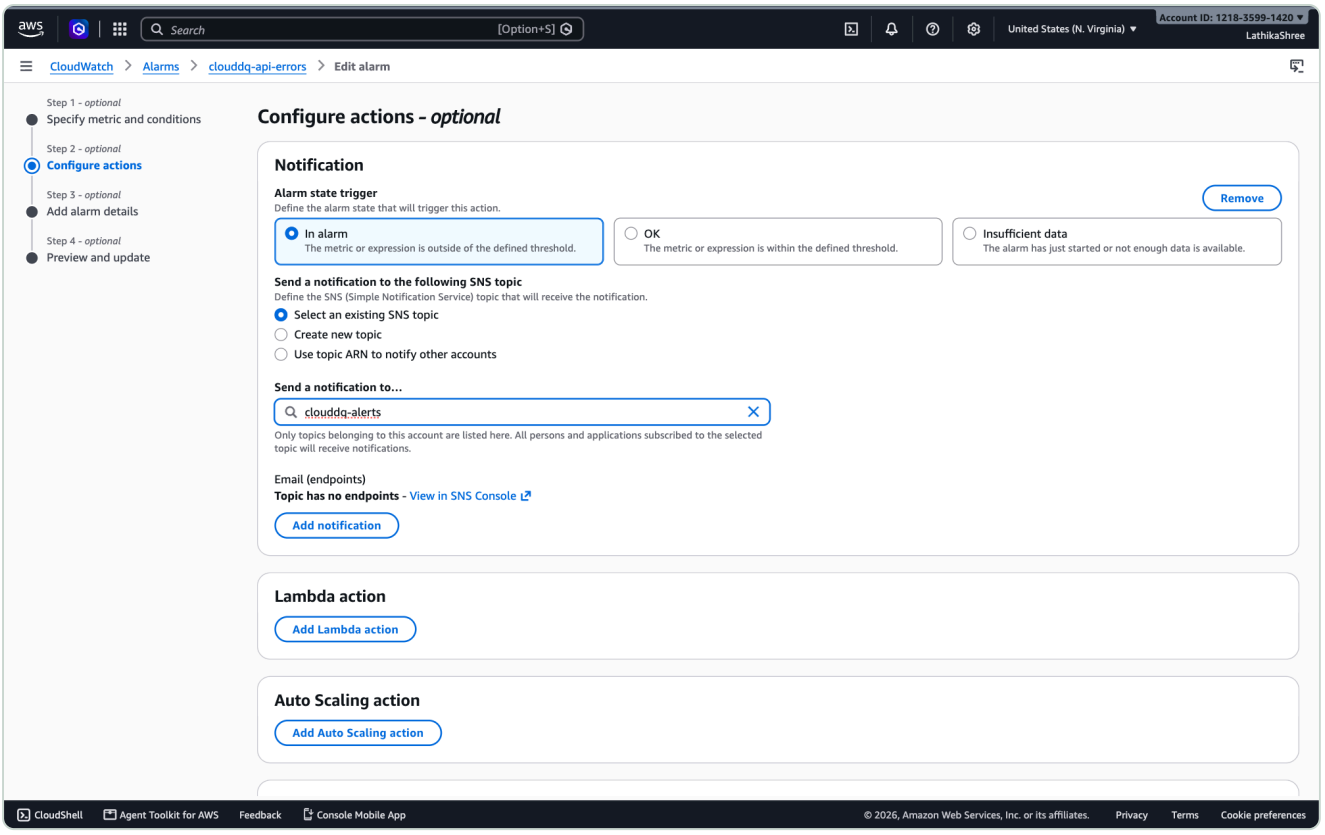
Step 7: Edit alarm — step 1 — Metric and condition stay the same (Lambda Errors > 0 in 5 minutes). Choose Next.



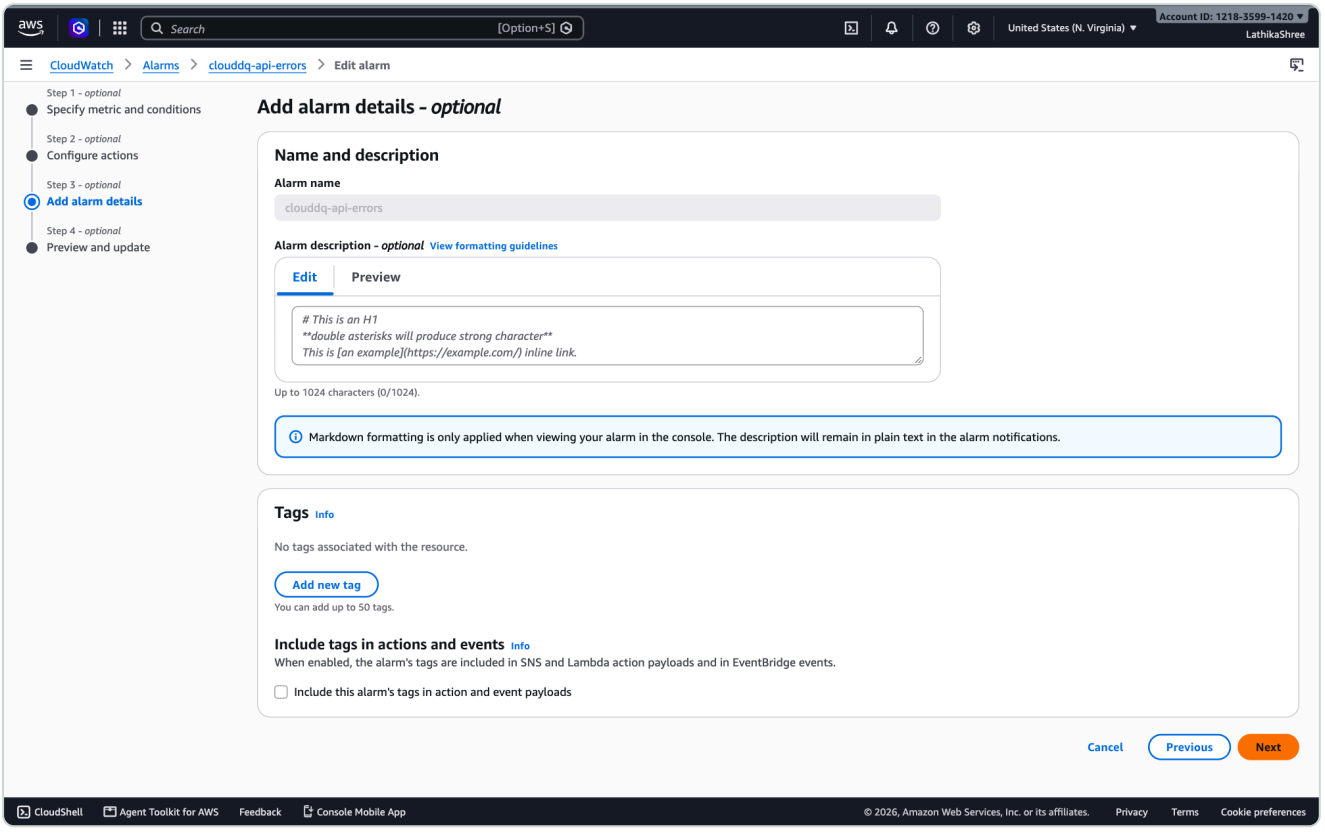
Step 8: Edit alarm — step 2 — Configure actions. Choose Add notification.



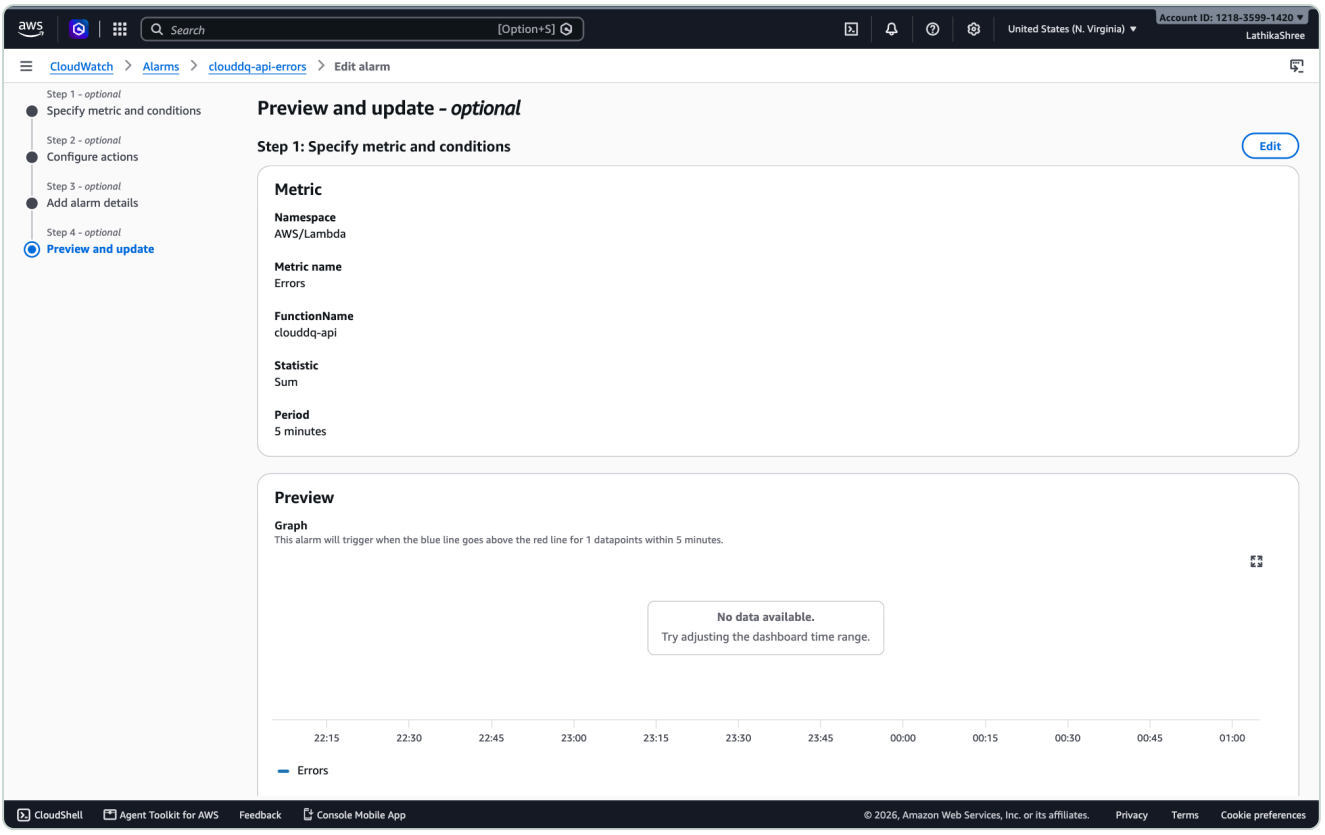
Step 9: Add notification — Trigger In alarm, Select an existing SNS topic.



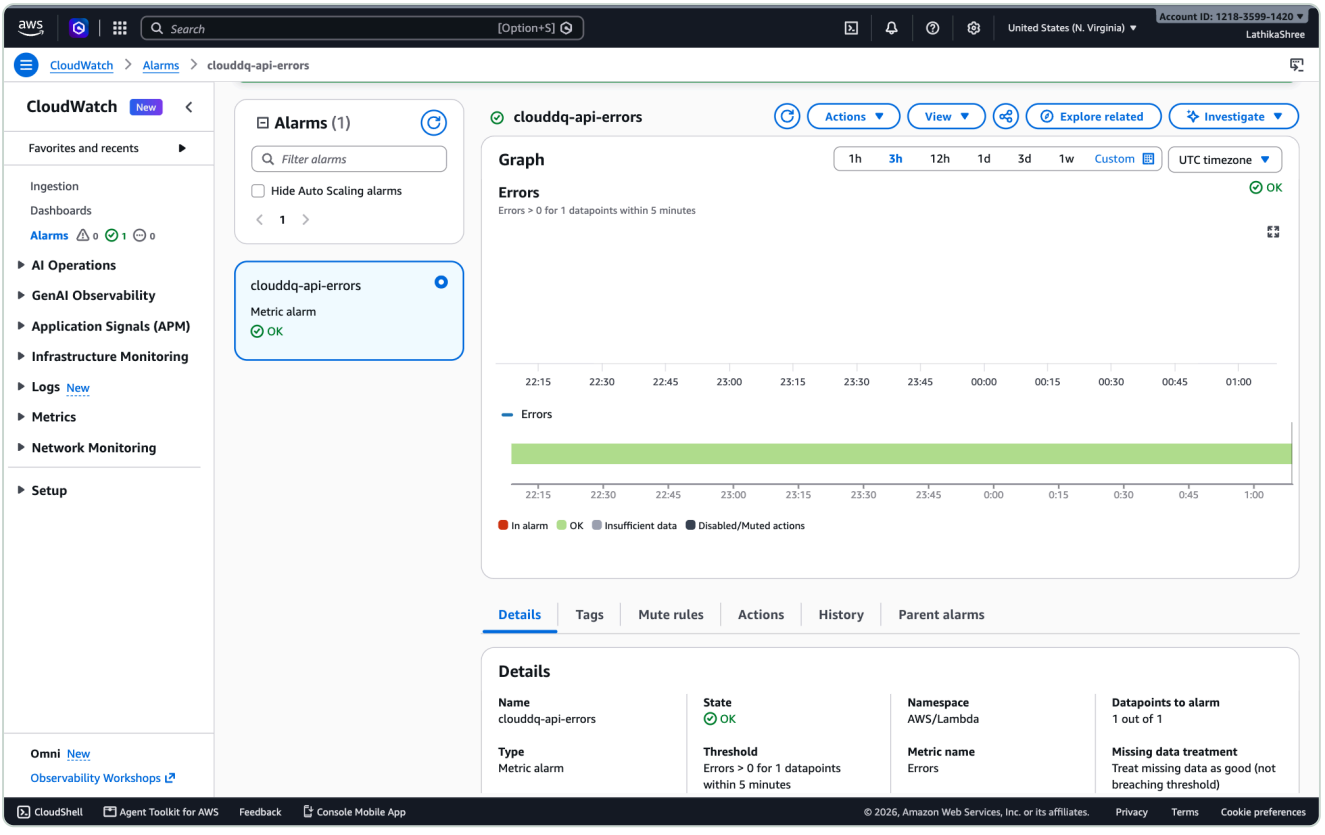
Step 10: Choose the topic — Send a notification to clouddq-alerts. Then Next.



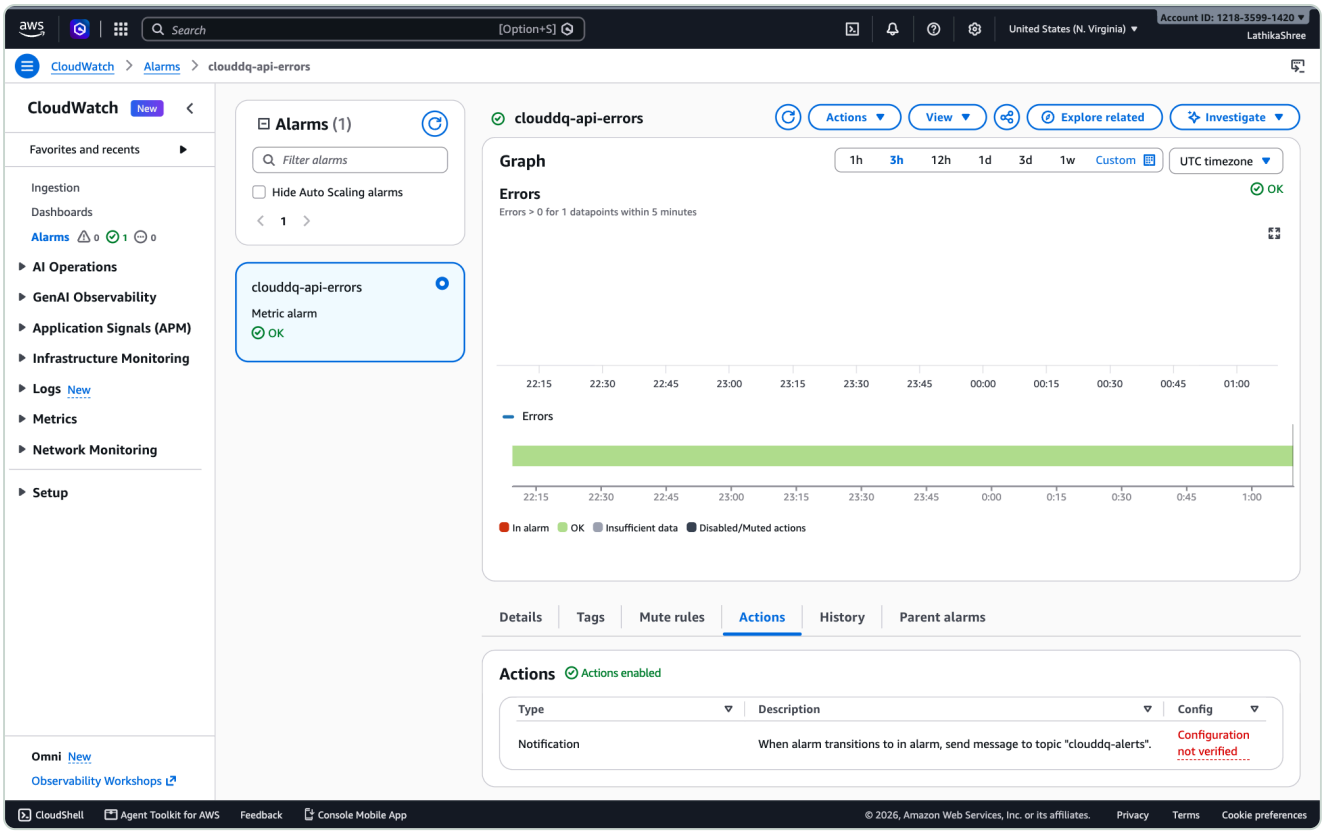
Step 11: Edit alarm — step 3 — Name stays clouddq-api-errors. Choose Next.



Step 12: Edit alarm — step 4 — Preview the change and choose Update alarm.



Step 13: Alarm updated — Back on the alarm page.



Step 14: Actions tab — Notification to clouddq-alerts when the alarm goes In alarm. 'Configuration not verified' disappears once an email subscription is confirmed.

10. Feature Output Screenshots

Captured from the live deployment https://main.d3ffgcfvtof2u.amplifyapp.com/ with the demo Data User and Business User accounts.

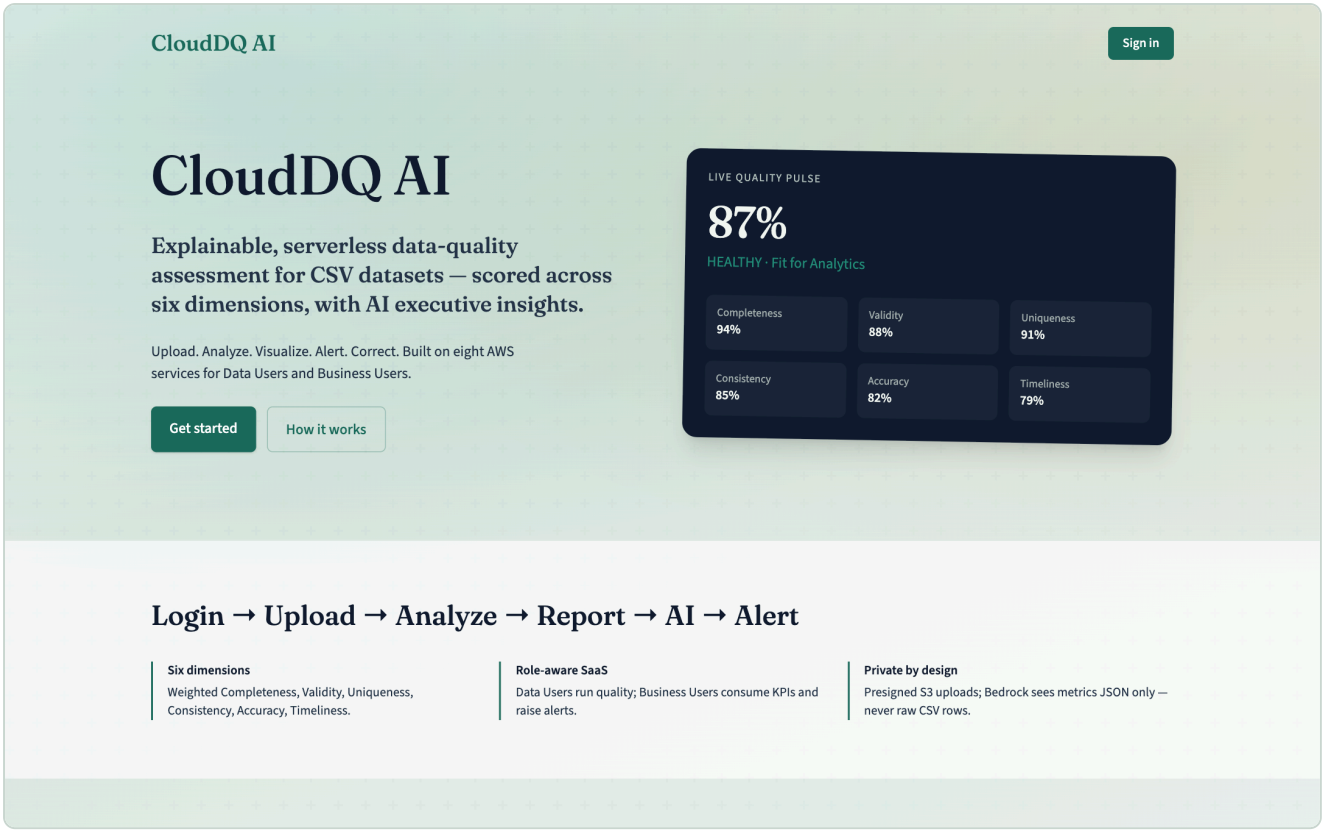


Fig. 1: Landing page — Public landing page describing the platform and the six dimensions.

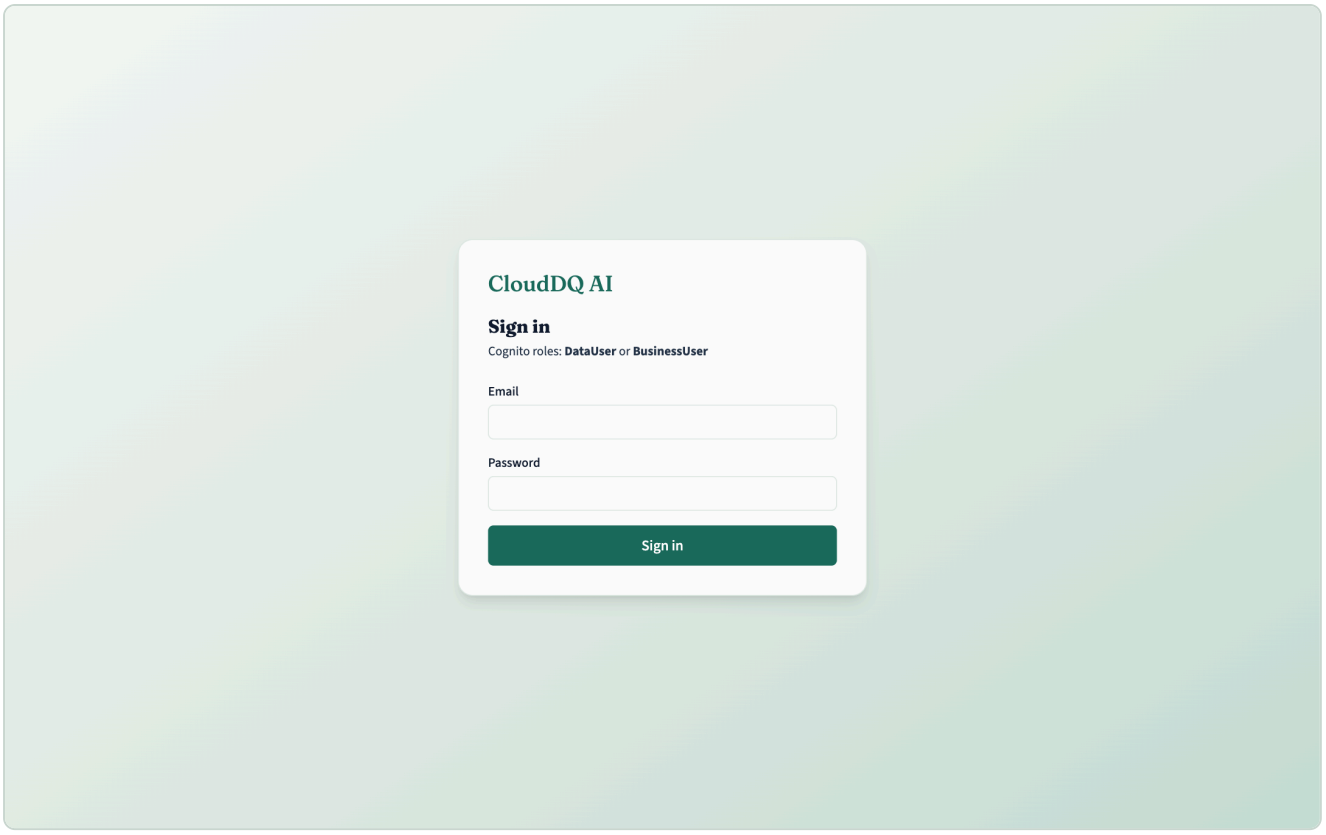


Fig. 2: Cognito sign-in — Email and password sign-in against the Amazon Cognito user pool.

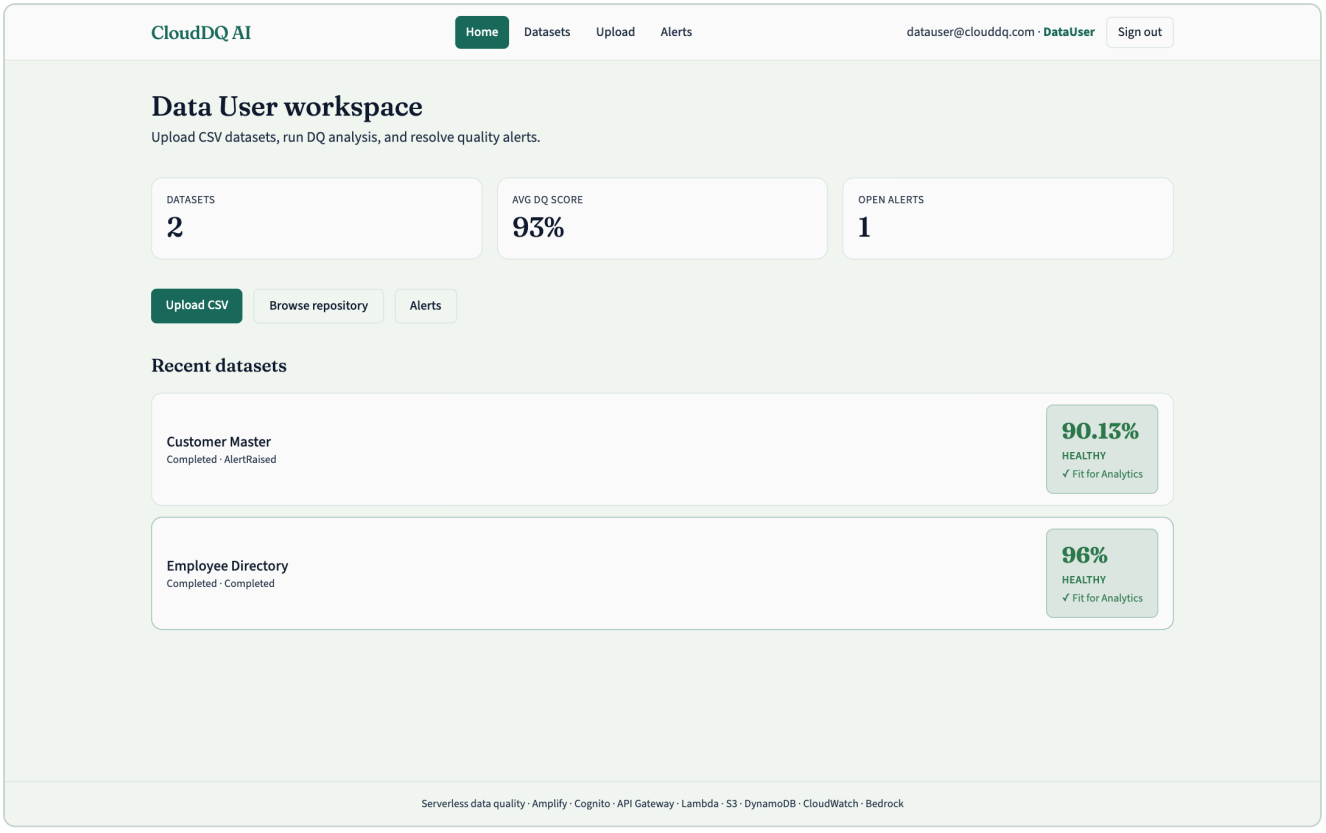


Fig. 3: Data User workspace — Role-aware home with dataset and alert counts for the Data User.

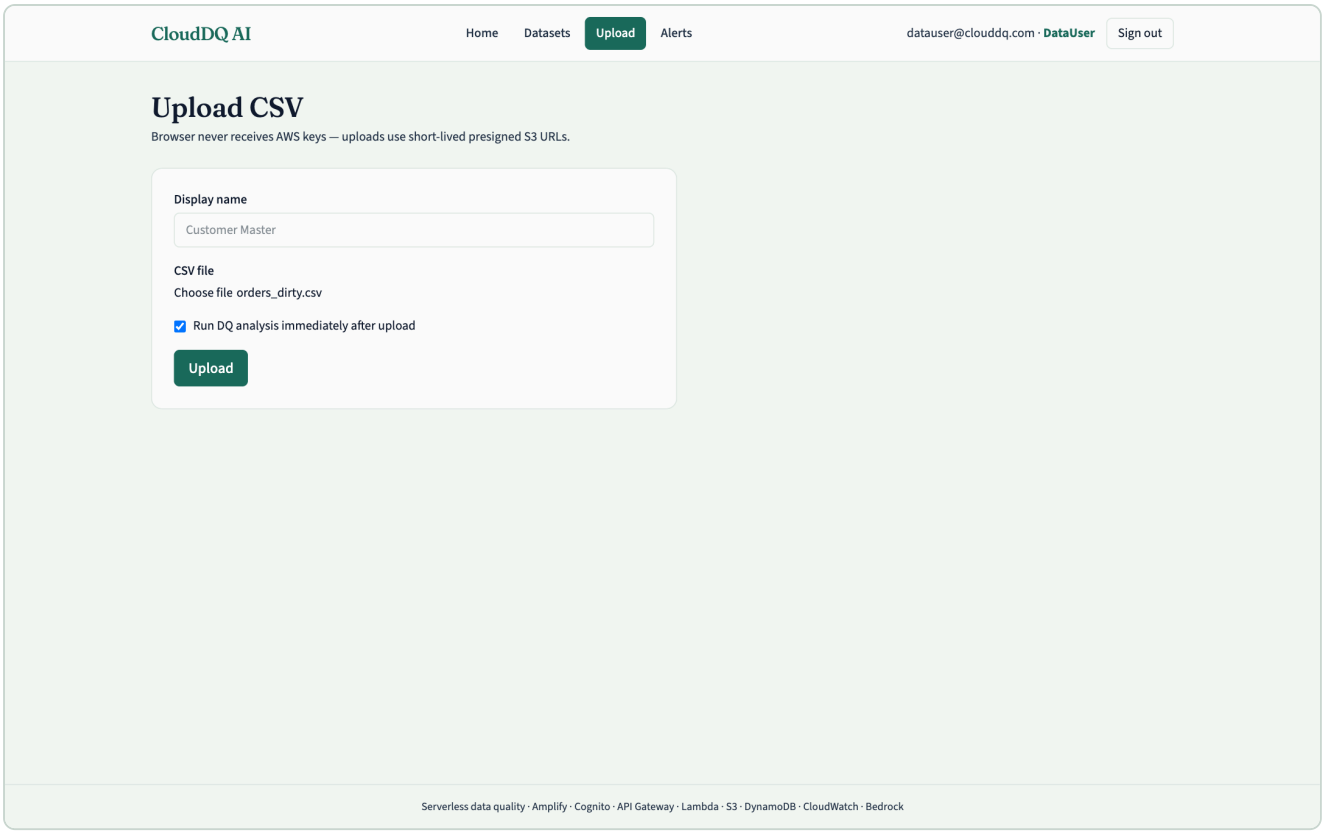


Fig. 4: CSV upload — Upload form; the file goes straight to S3 through a presigned URL.

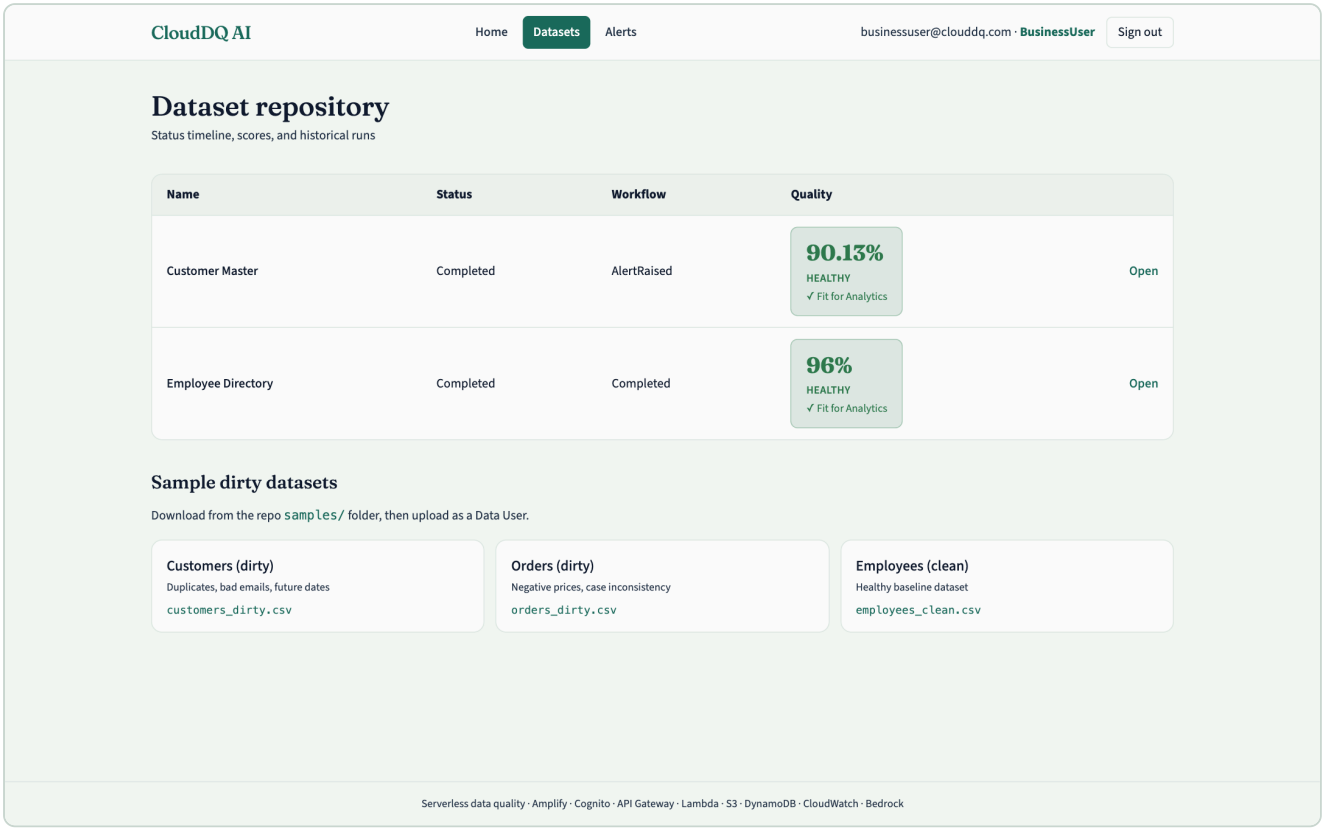


Fig. 5: Dataset repository — All datasets with DQ score, health and workflow status.

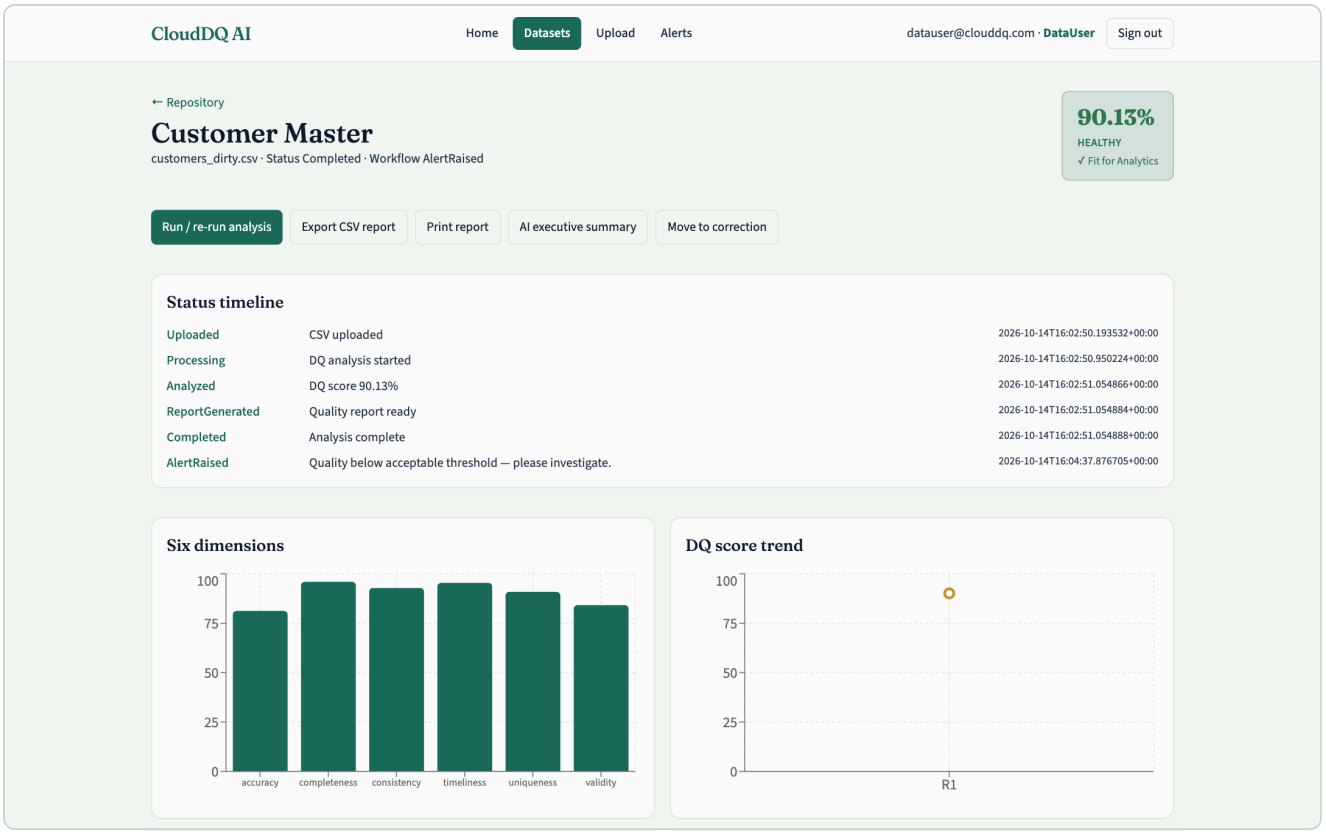


Fig. 6: Quality report — Score 90.13% Healthy, actions, status timeline, dimensions and trend.

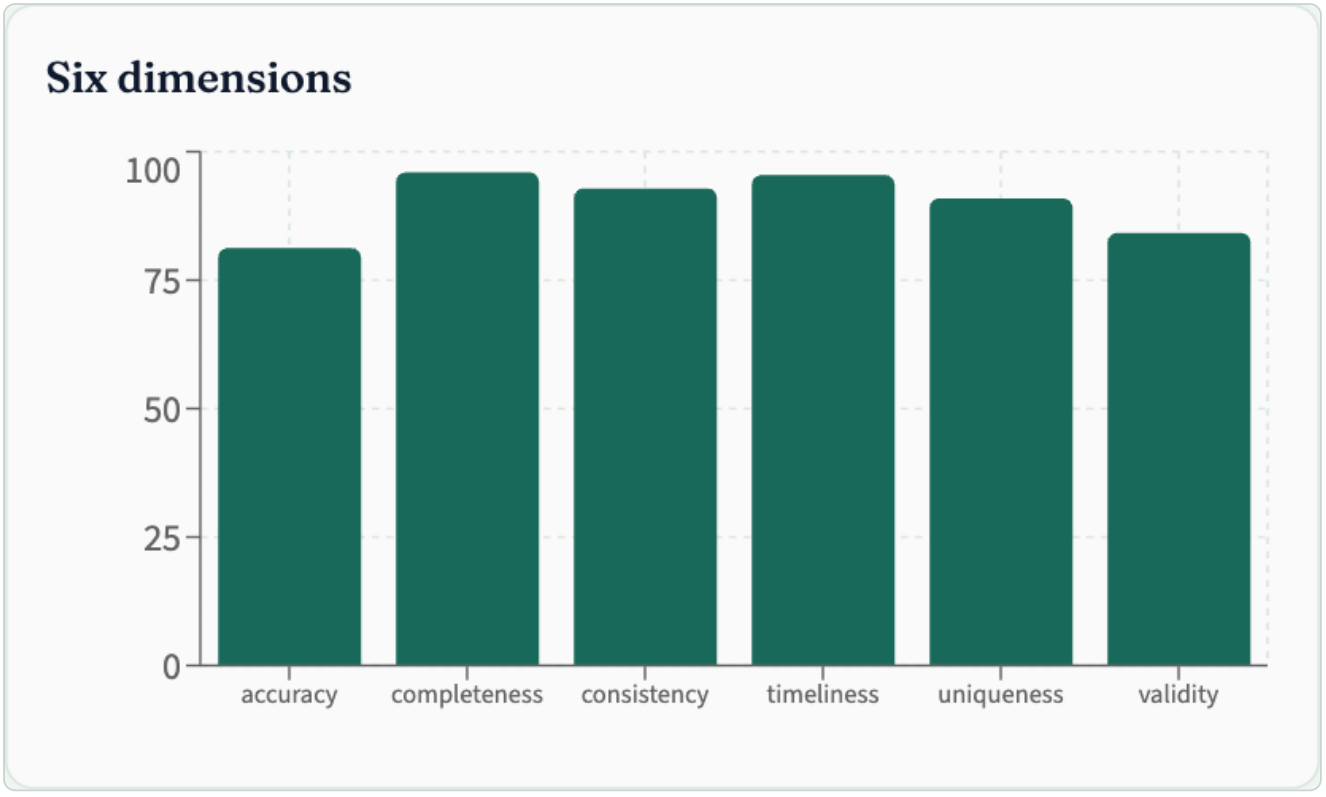


Fig. 7: Six-dimension scores — Accuracy, completeness, consistency, timeliness, uniqueness, validity.



Fig. 8: Column quality heatmap — Per-column quality from null rates and type checks.

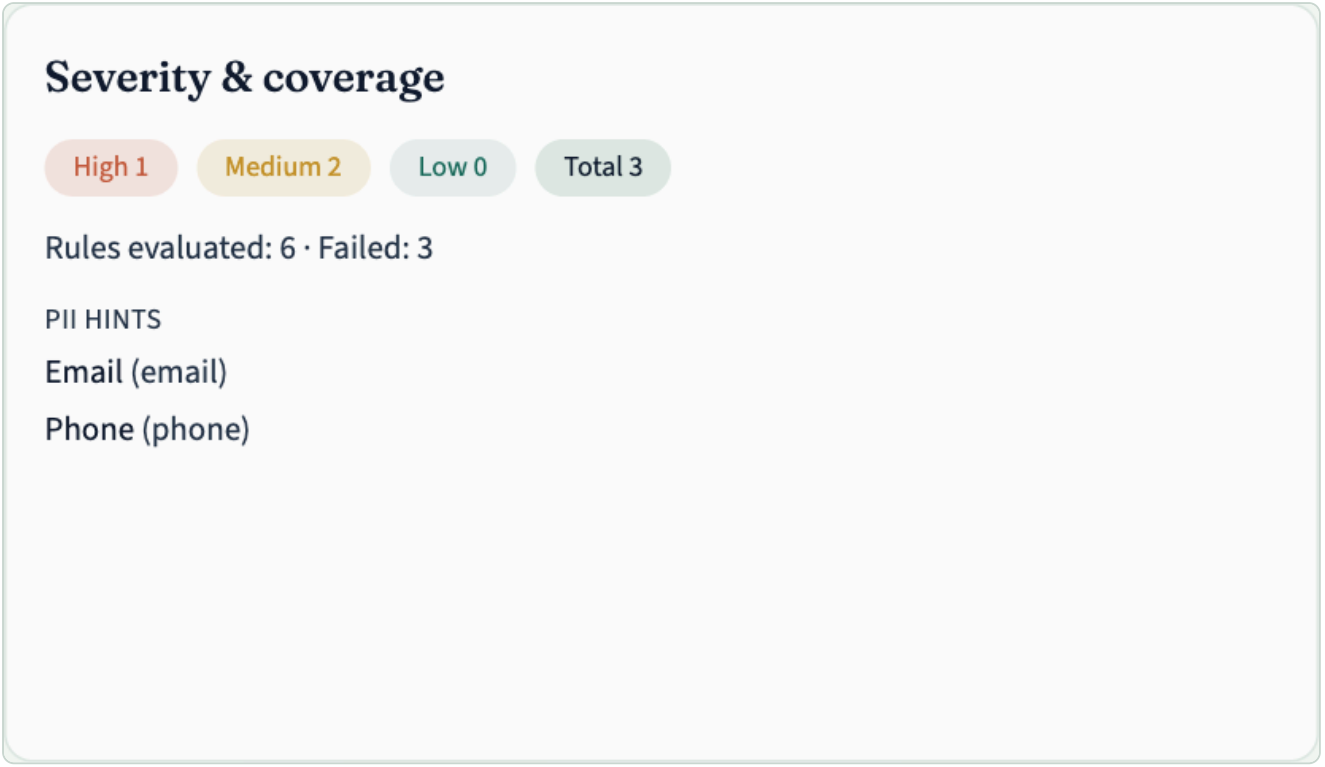


Fig. 9: Severity & rule coverage — Issue counts by severity and how many rules passed.



Fig. 10: Personal data detection (Amazon Comprehend) — Comprehend flags Full_Name as NAME and Email as EMAIL with 100% confidence.

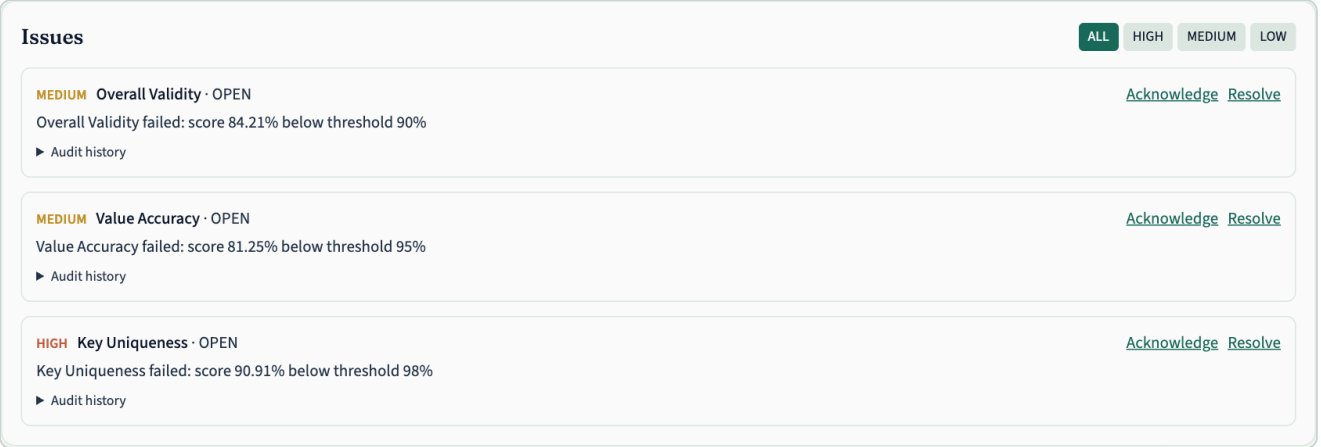


Fig. 11: Issues with lifecycle actions — Rule failures with severity, acknowledge / resolve and audit history.

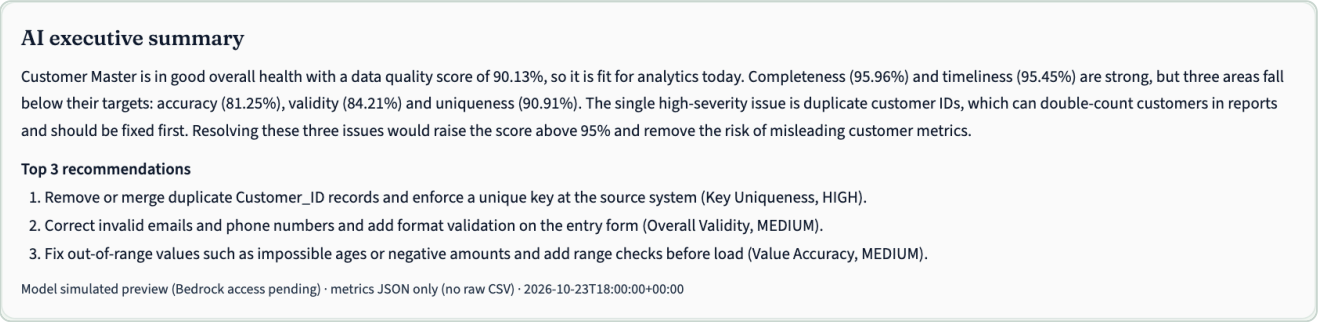


Fig. 12: AI executive summary — Executive summary and top-3 recommendations from metrics JSON only (simulated preview while Bedrock access is pending).



Fig. 13: Soft correction — Data User re-uploads a corrected CSV to re-validate.

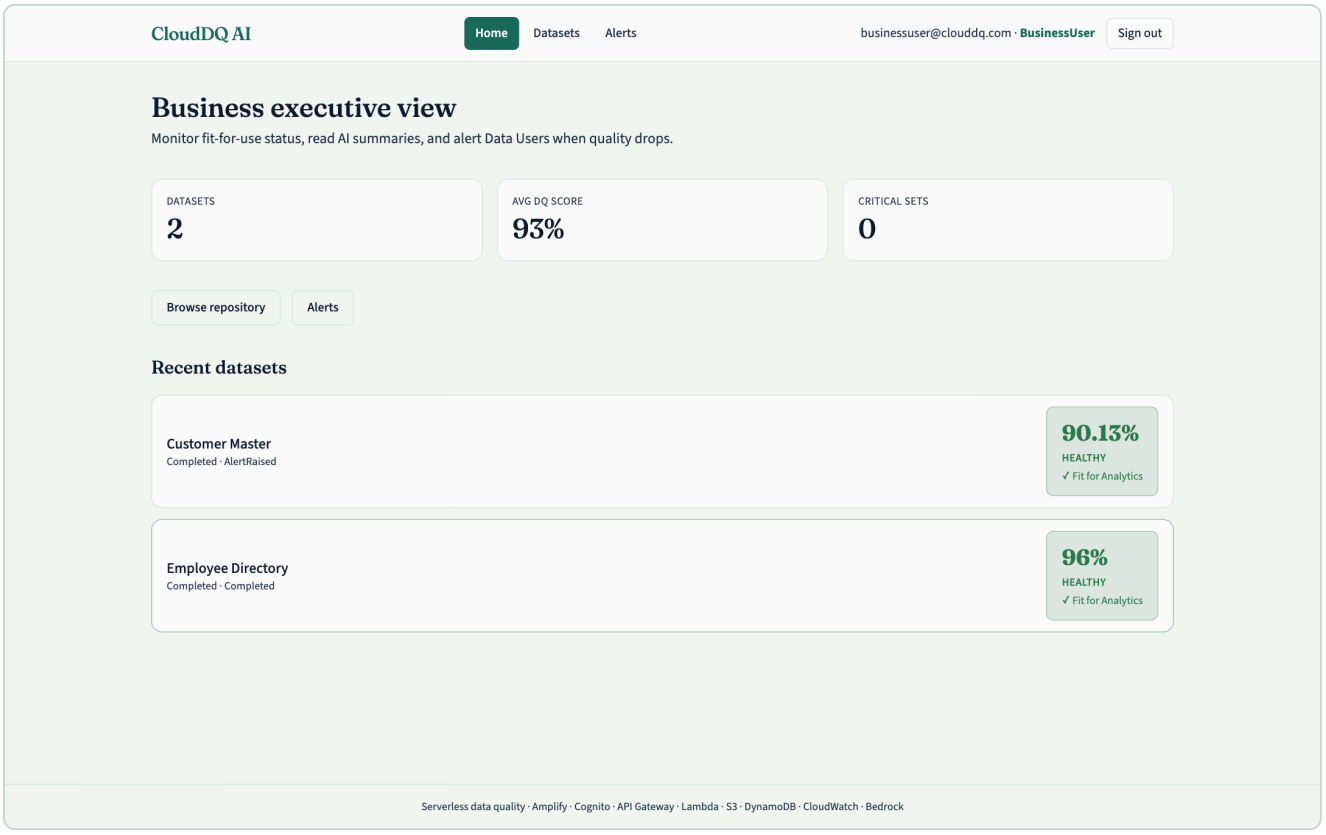


Fig. 14: Business User workspace — Business User view — no upload, read results and raise alerts.



Fig. 15: Send quality alert — Business User sends an alert with a note to the Data User.

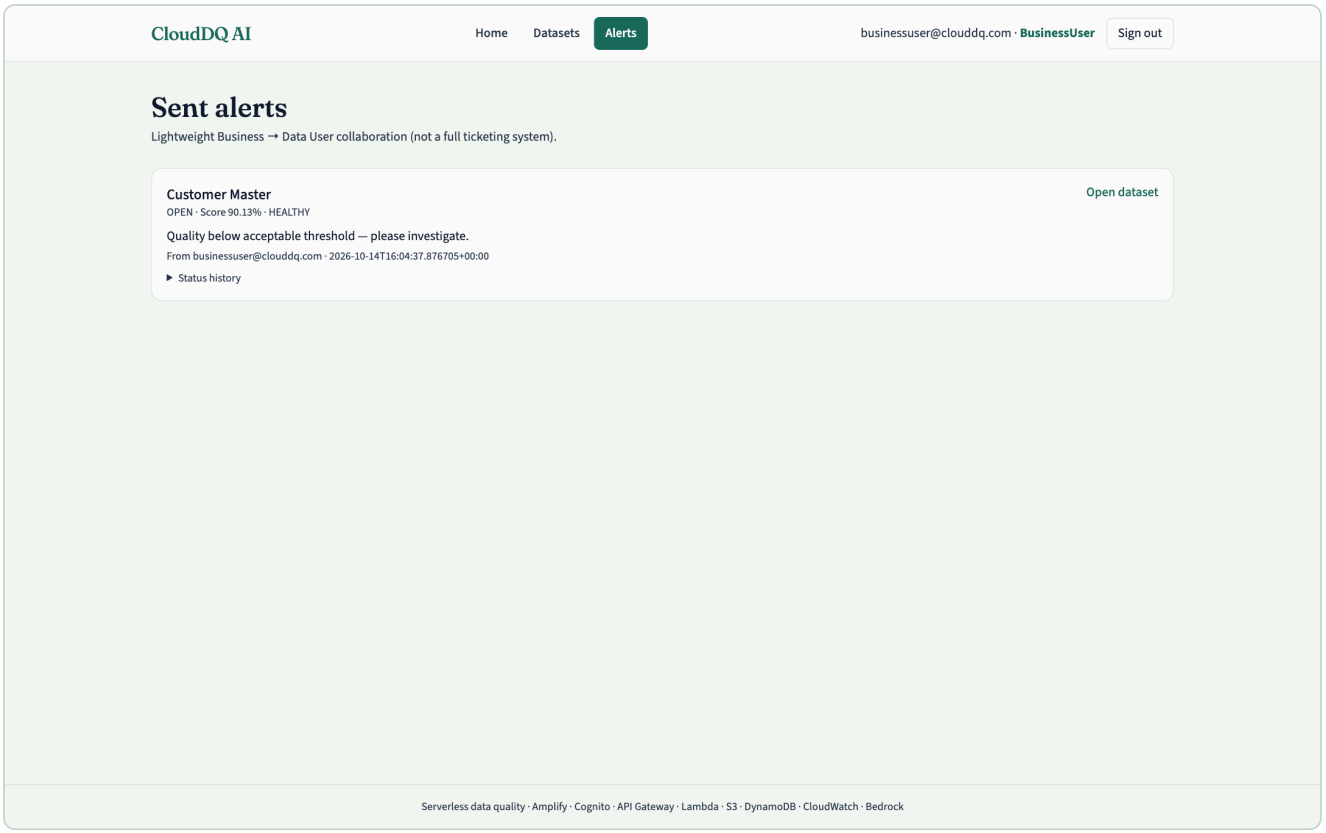


Fig. 16: Sent alerts — Alert status OPEN with score and health, plus status history.

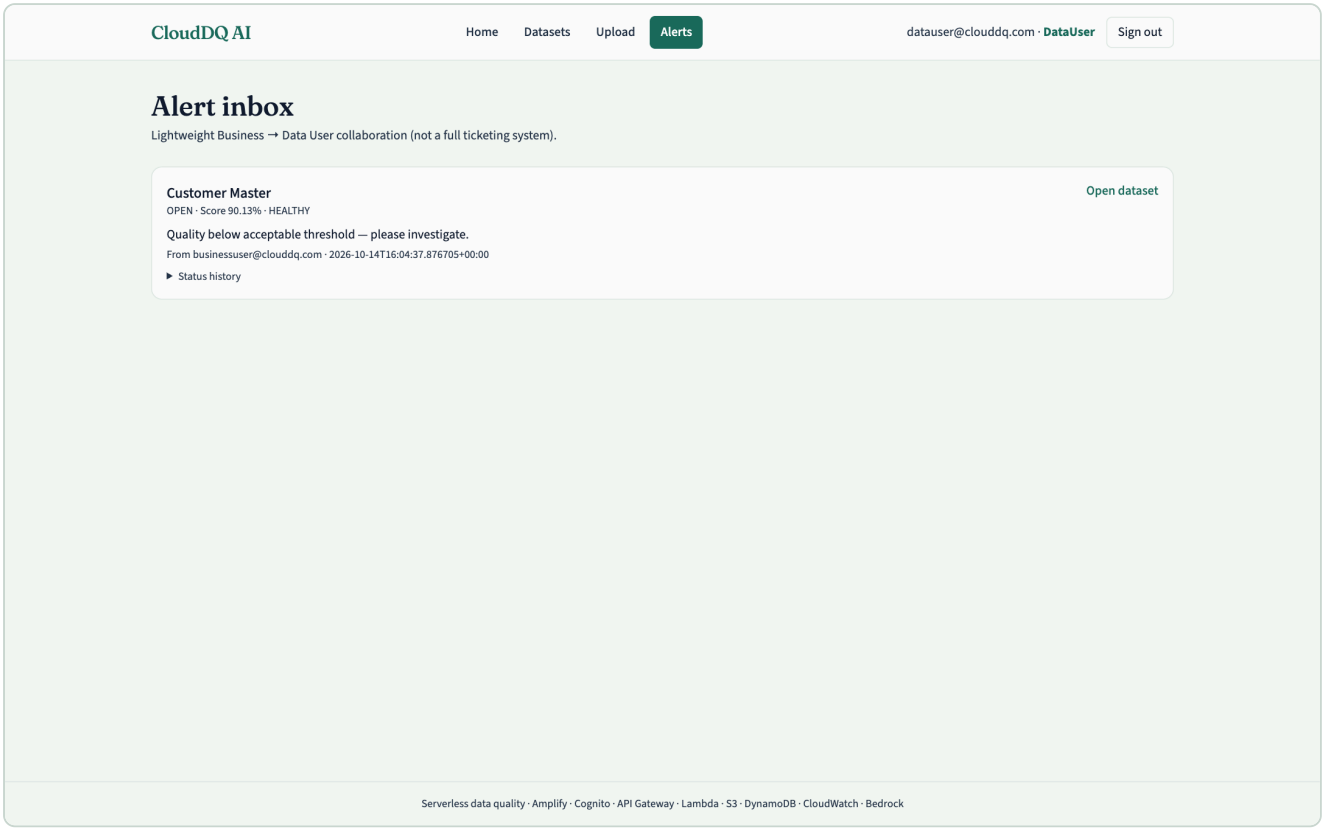


Fig. 17: Data User alert inbox — Data User receives the alert and starts the correction workflow.

11. Results, Challenges and Future Scope

Results

- Sample dataset Customer Master (customers_dirty.csv) scored 90.13% — HEALTHY, fit for analytics.
- Dimension scores: completeness 95.96, validity 84.21, uniqueness 90.91, consistency 92.86, accuracy 81.25, timeliness 95.45.
- Three open issues raised: Key Uniqueness (HIGH), Overall Validity (MEDIUM), Value Accuracy (MEDIUM).
- Employee Directory (employees_clean.csv) scored 96.0% — Comprehend flagged Full_Name (NAME) and Email (EMAIL) at 100% confidence.
- Business User alert moved the dataset workflow to AlertRaised, appeared in the Data User inbox and was published to SNS.
- Requests without a token are rejected by API Gateway with 401, and wrong-role actions by Lambda with 403.

Challenges and how they were handled

- **Bedrock access on a new account.** The account returned NOT_AUTHORIZED for Bedrock models with a quota of 0. The Converse integration and IAM are in place; the app falls back to a deterministic summary from the same metrics until AWS lifts the restriction (support case).
- **Monorepo build on Amplify.** The app root is the repository root, so the build spec runs cd frontend and publishes frontend/dist.
- **Greedy proxy resource in API Gateway.** The /{proxy+} resource needs the greedy path parameter and the authorizer on ANY only, never on the OPTIONS preflight.
- **Cognito first sign-in.** Admin-created users start in FORCE_CHANGE_PASSWORD; passwords were set as permanent to match the in-app login.

Future scope

- S3 event notifications → automatic analysis on upload; Step Functions for the analysis workflow.
- Per-user SNS subscriptions or Amazon SES templated emails, and Comprehend-driven masking of PII columns.
- AWS Glue Data Quality and Athena for very large datasets and SQL exploration.
- Bedrock Guardrails and Knowledge Bases for safer, rule-aware explanations.
- Infrastructure as code (SAM / CDK) with a CI/CD pipeline and multiple environments.

12. References

1. Amazon Web Services, “AWS Amplify Hosting User Guide.” https://docs.aws.amazon.com/amplify/latest/userguide/
2. Amazon Web Services, “Amazon Cognito Developer Guide — User pools.” https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools.html
3. Amazon Web Services, “Use Amazon Cognito user pools as authorizer for a REST API.” https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-integrate-with-cognito.html
4. Amazon Web Services, “Lambda proxy integrations in API Gateway.” https://docs.aws.amazon.com/apigateway/latest/developerguide/set-up-lambda-proxy-integrations.html
5. Amazon Web Services, “AWS Lambda Developer Guide — Building with Python.” https://docs.aws.amazon.com/lambda/latest/dg/lambda-python.html
6. Amazon Web Services, “Uploading objects with presigned URLs.” https://docs.aws.amazon.com/AmazonS3/latest/userguide/PresignedUrlUploadObject.html
7. Amazon Web Services, “Best practices for DynamoDB — single-table design.” https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-general-nosql-design.html
8. Amazon Web Services, “Amazon Bedrock — Converse API.” https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html
9. Amazon Web Services, “Amazon Nova User Guide.” https://docs.aws.amazon.com/nova/latest/userguide/
10. Amazon Web Services, “Amazon Comprehend — Detecting PII entities.” https://docs.aws.amazon.com/comprehend/latest/dg/how-pii.html
11. Amazon Web Services, “Amazon SNS Developer Guide — Email notifications.” https://docs.aws.amazon.com/sns/latest/dg/sns-email-notifications.html
12. Amazon Web Services, “Using Amazon CloudWatch alarms.” https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html
13. Amazon Web Services, “Security best practices in IAM.” https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html
14. DAMA International, DAMA-DMBOK: Data Management Body of Knowledge, 2nd ed. Technics Publications, 2017.
15. ISO/IEC 25012:2008, Software engineering — Software product Quality Requirements and Evaluation (SQuaRE) — Data quality model.
16. R. Y. Wang and D. M. Strong, “Beyond accuracy: What data quality means to data consumers,” Journal of Management Information Systems, 12(4), 5–33, 1996.
17. React documentation. https://react.dev/ · Vite documentation. https://vite.dev/ · Recharts. https://recharts.org/

Appendix — Complete Module Code

Every source file of the project, grouped by module.

1. M1 — Authentication & Role-Based Access
    - `backend/src/auth/__init__.py`
    - `frontend/src/lib/cognito.ts`
    - `frontend/src/auth/AuthContext.tsx`
    - `frontend/src/pages/LoginPage.tsx`
2. M2 — Frontend SaaS Application
    - `frontend/src/main.tsx`
    - `frontend/src/App.tsx`
    - `frontend/src/config.ts`
    - `frontend/src/types.ts`
    - `frontend/src/components/AppShell.tsx`
    - `frontend/src/components/Badges.tsx`
    - `frontend/src/pages/LandingPage.tsx`
    - `frontend/src/pages/HomePage.tsx`
    - `frontend/src/index.css`
    - `frontend/index.html`
    - `frontend/vite.config.ts`
    - `frontend/package.json`
3. M3 — Dataset Upload & Repository
    - `backend/src/datasets/__init__.py`
    - `backend/src/datasets/service.py`
    - `backend/src/store/__init__.py`
    - `backend/src/store/s3.py`
    - `frontend/src/lib/api.ts`
    - `frontend/src/pages/UploadPage.tsx`
    - `frontend/src/pages/DatasetsPage.tsx`
4. M4 — Data Quality Engine
    - `backend/src/dq_engine/__init__.py`
    - `backend/src/dq_engine/profiler.py`
    - `backend/src/dq_engine/dimensions.py`
    - `backend/src/dq_engine/rules.py`
    - `backend/src/dq_engine/anomalies.py`
    - `backend/src/dq_engine/engine.py`
    - `backend/src/ai/comprehend.py`
    - `backend/tests/test_dq_engine.py`
5. M5 — Dashboard, Reports & Analytics
    - `frontend/src/pages/DatasetDetailPage.tsx`
6. M6 — AI Executive Summary (Amazon Bedrock)

- backend/src/ai/__init__.py
- backend/src/ai/bedrock.py

7. M7 — Alerts & Soft Correction Workflow
    - backend/src/alerts/__init__.py
    - backend/src/alerts/service.py
    - backend/src/store/sns.py
    - frontend/src/pages/AlertsPage.tsx
8. M8 — API, Data Store & Cloud Deployment
    - backend/src/handler.py
    - backend/src/http_util.py
    - backend/src/__init__.py
    - backend/src/store/dynamo.py
    - backend/src/requirements.txt
    - backend/template.yaml
    - backend/local_server.py
    - frontend/amplify.yml

M1 — Authentication & Role-Based Access

backend/src/auth/__init__.py 52 lines

```python
"""Cognito JWT claims and role helpers."""
from __future__ import annotations

from typing import Literal

Role = Literal["DataUser", "BusinessUser", "Unknown"]


def claims_from_event(event: dict) -> dict:
    auth = event.get("requestContext", {}).get("authorizer", {})
    claims = auth.get("claims") or auth.get("jwt", {}).get("claims") or {}
    return claims


def user_from_event(event: dict) -> dict:
    claims = claims_from_event(event)
    groups_raw = claims.get("cognito:groups") or claims.get("cognito:groups".lower()) or ""
    if isinstance(groups_raw, list):
        groups = groups_raw
    elif isinstance(groups_raw, str):
        # API Gateway may deliver groups as a JSON-like string or comma-separated
        cleaned = groups_raw.strip("[]").replace('"', "").replace("'", "")
        groups = [g.strip() for g in cleaned.split(",") if g.strip()]
    else:
        groups = []

    role: Role = "Unknown"
    if "DataUser" in groups:
        role = "DataUser"
    elif "BusinessUser" in groups:
        role = "BusinessUser"

    return {
        "sub": claims.get("sub") or claims.get("username") or "anonymous",
        "email": claims.get("email") or claims.get("cognito:username") or "",
        "username": claims.get("cognito:username") or claims.get("email") or "",
        "groups": groups,
        "role": role,
    }


```

```python
def require_role(user: dict, *allowed: str) -> None:
    if user.get("role") not in allowed:
        raise PermissionError(f"Role {user.get('role')} not permitted; requires one of {allowed}")


def is_data_user(user: dict) -> bool:
    return user.get("role") == "DataUser"


def is_business_user(user: dict) -> bool:
    return user.get("role") == "BusinessUser"
```

frontend/src/lib/cognito.ts 129 lines

```typescript
import {
  AuthenticationDetails,
  CognitoUser,
  CognitoUserAttribute,
  CognitoUserPool,
  CognitoUserSession,
} from 'amazon-cognito-identity-js'
import { config, type Role } from '../config'

export interface AuthUser {
  email: string
  sub: string
  role: Role
  token: string
}

function pool(): CognitoUserPool | null {
  if (!config.userPoolId || !config.userPoolClientId) return null
  return new CognitoUserPool({
    UserPoolId: config.userPoolId,
    ClientId: config.userPoolClientId,
  })
}

function groupsFromSession(session: CognitoUserSession): string[] {
  const payload = session.getIdToken().decodePayload()
  const groups = payload['cognito:groups']
  if (Array.isArray(groups)) return groups as string[]
  if (typeof groups === 'string') return [groups]
  return []
}

function roleFromGroups(groups: string[]): Role {
  if (groups.includes('DataUser')) return 'DataUser'
  if (groups.includes('BusinessUser')) return 'BusinessUser'
  return 'DataUser'
}

export function login(email: string, password: string): Promise<AuthUser> {
  if (config.devAuth) {
    const role: Role = email.toLowerCase().includes('business') ? 'BusinessUser' : 'DataUser'
    const user: AuthUser = {
      email,
      sub: `dev-${role.toLowerCase()}`,
      role,
      token: 'dev-token',
    }
    localStorage.setItem('clouddq_dev_user', JSON.stringify(user))
    return Promise.resolve(user)
  }

  const userPool = pool()
  if (!userPool) {
    return Promise.reject(new Error('Cognito is not configured. Set VITE_COGNITO_* or
VITE_DEV_AUTH=1'))
  }

  const cognitoUser = new CognitoUser({ Username: email, Pool: userPool })
  const authDetails = new AuthenticationDetails({ Username: email, Password: password })

  return new Promise((resolve, reject) => {
    cognitoUser.authenticateUser(authDetails, {
      onSuccess: (session) => {
        const groups = groupsFromSession(session)
```

```
        resolve({
          email,
          sub: session.getIdToken().payload.sub as string,
          role: roleFromGroups(groups),
          token: session.getIdToken().getJwtToken(),
        })
      },
      onFailure: (err) => reject(err),
      newPasswordRequired: () => {
        reject(new Error('New password required — set permanent password in Cognito console'))
      },
    })
  })
}

export function logout(): void {
  localStorage.removeItem('clouddq_dev_user')
  const userPool = pool()
  const current = userPool?.getCurrentUser()
  current?.signOut()
}

export function getCurrentUser(): Promise<AuthUser | null> {
  if (config.devAuth) {
    const raw = localStorage.getItem('clouddq_dev_user')
    return Promise.resolve(raw ? (JSON.parse(raw) as AuthUser) : null)
  }

  const userPool = pool()
  if (!userPool) return Promise.resolve(null)
  const cognitoUser = userPool.getCurrentUser()
  if (!cognitoUser) return Promise.resolve(null)

  return new Promise((resolve) => {
    cognitoUser.getSession((err: Error | null, session: CognitoUserSession | null) => {
      if (err || !session?.isValid()) {
        resolve(null)
        return
      }
      const email =
        (session.getIdToken().payload.email as string) ||
        cognitoUser.getUsername()
      resolve({
        email,
        sub: session.getIdToken().payload.sub as string,
        role: roleFromGroups(groupsFromSession(session)),
        token: session.getIdToken().getJwtToken(),
      })
    })
  })
}

/** Optional signup helper for demo provisioning scripts. */
export function signUp(email: string, password: string): Promise<void> {
  const userPool = pool()
  if (!userPool) return Promise.reject(new Error('Cognito not configured'))
  return new Promise((resolve, reject) => {
    userPool.signUp(
      email,
      password,
      [new CognitoUserAttribute({ Name: 'email', Value: email })],
      [],
      (err) => (err ? reject(err) : resolve()),
    )
  })
}
```

frontend/src/auth/AuthContext.tsx 60 lines

```
import {
  createContext,
  useCallback,
  useContext,
  useEffect,
  useMemo,
  useState,
  type ReactNode,
} from 'react'
```

```tsx
import { getCurrentUser, login as cognitoLogin, logout as cognitoLogout, type AuthUser } from '../lib/cognito'
import { setApiAuth } from '../lib/api'

interface AuthContextValue {
  user: AuthUser | null
  loading: boolean
  login: (email: string, password: string) => Promise<void>
  logout: () => void
}

const AuthContext = createContext<AuthContextValue | undefined>(undefined)

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<AuthUser | null>(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    getCurrentUser()
      .then((u) => {
        if (u) {
          setUser(u)
          setApiAuth({ token: u.token, role: u.role, sub: u.sub, email: u.email })
        }
      })
      .finally(() => setLoading(false))
  }, [])

  const login = useCallback(async (email: string, password: string) => {
    const u = await cognitoLogin(email, password)
    setApiAuth({ token: u.token, role: u.role, sub: u.sub, email: u.email })
    setUser(u)
  }, [])

  const logout = useCallback(() => {
    cognitoLogout()
    setUser(null)
  }, [])

  const value = useMemo(
    () => ({ user, loading, login, logout }),
    [user, loading, login, logout],
  )

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>
}

export function useAuth() {
  const ctx = useContext(AuthContext)
  if (!ctx) throw new Error('useAuth must be used within AuthProvider')
  return ctx
}
```

frontend/src/pages/LoginPage.tsx 79 lines

```tsx
import { useState, type FormEvent } from 'react'
import { Link, Navigate, useNavigate } from 'react-router-dom'
import { useAuth } from '../auth/AuthContext'
import { config } from '../config'

export function LoginPage() {
  const { user, login } = useAuth()
  const navigate = useNavigate()
  const [email, setEmail] = useState(config.devAuth ? 'data@example.com' : '')
  const [password, setPassword] = useState(config.devAuth ? 'Password1' : '')
  const [error, setError] = useState('')
  const [busy, setBusy] = useState(false)

  if (user) return <Navigate to="/app" replace />

  async function onSubmit(e: FormEvent) {
    e.preventDefault()
    setBusy(true)
    setError('')
    try {
      await login(email, password)
      navigate('/app')
    } catch (err) {
```

```
      setError(err instanceof Error ? err.message : 'Login failed')
    } finally {
      setBusy(false)
    }
  }

  return (
    <div className="min-h-screen flex items-center justify-center px-4"
      style={{ background: 'linear-gradient(145deg, #f3f7f4, #dce8e1 60%, #c5ddd3)' }}>
      <div className="w-full max-w-md bg-white/90 backdrop-blur rounded-2xl shadow-lg border border-mist p-8">
        <Link to="/" className="font-display text-2xl text-sea">CloudDQ AI</Link>
        <h1 className="mt-4 text-xl font-semibold">Sign in</h1>
        <p className="text-sm text-ink-soft mt-1">
          Cognito roles: <strong>DataUser</strong> or <strong>BusinessUser</strong>
        </p>

        {config.devAuth && (
          <div className="mt-4 rounded-lg bg-amber/10 border border-amber/30 text-sm p-3 text-ink-soft">
            Dev auth is on. Use any password. Email containing <code>business</code> → BusinessUser; otherwise DataUser.
          </div>
        )}

        <form onSubmit={onSubmit} className="mt-6 space-y-4">
          <label className="block text-sm">
            <span className="font-medium">Email</span>
            <input
              type="email"
              required
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              className="mt-1 w-full rounded-md border border-mist px-3 py-2 focus:outline-none focus:ring-2 focus:ring-sea/40"
            />
          </label>
          <label className="block text-sm">
            <span className="font-medium">Password</span>
            <input
              type="password"
              required
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              className="mt-1 w-full rounded-md border border-mist px-3 py-2 focus:outline-none focus:ring-2 focus:ring-sea/40"
            />
          </label>
          {error && <p className="text-sm text-coral">{error}</p>}
          <button
            type="submit"
            disabled={busy}
            className="w-full rounded-md bg-sea text-white py-2.5 font-medium hover:bg-sea-bright disabled:opacity-60"
          >
            {busy ? 'Signing in…' : 'Sign in'}
          </button>
        </form>
      </div>
    </div>
  )
}
```

M2 — Frontend SaaS Application

frontend/src/main.tsx 13 lines

```tsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

frontend/src/App.tsx 46 lines

```tsx
import { Navigate, Outlet, Route, Routes } from 'react-router-dom'
import { AuthProvider, useAuth } from './auth/AuthContext'
import { AppShell } from './components/AppShell'
import { LandingPage } from './pages/LandingPage'
import { LoginPage } from './pages/LoginPage'
import { HomePage } from './pages/HomePage'
import { DatasetsPage } from './pages/DatasetsPage'
import { UploadPage } from './pages/UploadPage'
import { DatasetDetailPage } from './pages/DatasetDetailPage'
import { AlertsPage } from './pages/AlertsPage'

function Protected() {
  const { user, loading } = useAuth()
  if (loading) return <div className="p-8 text-ink-soft">Loading…</div>
  if (!user) return <Navigate to="/login" replace />
  return <Outlet />
}

function DataUserOnly() {
  const { user } = useAuth()
  if (user?.role !== 'DataUser') return <Navigate to="/app" replace />
  return <Outlet />
}

export default function App() {
  return (
    <AuthProvider>
      <Routes>
        <Route path="/" element={<LandingPage />} />
        <Route path="/login" element={<LoginPage />} />
        <Route element={<Protected />}>
          <Route path="/app" element={<AppShell />}>
            <Route index element={<HomePage />} />
            <Route path="datasets" element={<DatasetsPage />} />
            <Route path="datasets/:id" element={<DatasetDetailPage />} />
            <Route path="alerts" element={<AlertsPage />} />
            <Route element={<DataUserOnly />}>
              <Route path="upload" element={<UploadPage />} />
            </Route>
          </Route>
        </Route>
        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    </AuthProvider>
  )
}
```

frontend/src/config.ts 11 lines

```ts
/** Runtime config — replace after `sam deploy` or set via Amplify env. */
export const config = {
  region: import.meta.env.VITE_AWS_REGION || 'us-east-1',
```

```
  userPoolId: import.meta.env.VITE_COGNITO_USER_POOL_ID || '',
  userPoolClientId: import.meta.env.VITE_COGNITO_CLIENT_ID || '',
  apiBaseUrl: import.meta.env.VITE_API_BASE_URL || 'http://127.0.0.1:3000',
  /** Local demo without Cognito — set VITE_DEV_AUTH=1 */
  devAuth: import.meta.env.VITE_DEV_AUTH === '1',
}

export type Role = 'DataUser' | 'BusinessUser'
```

frontend/src/types.ts 81 lines

```
export type Role = 'DataUser' | 'BusinessUser'

export interface Dataset {
  datasetId: string
  name: string
  filename?: string
  s3Key?: string
  ownerId?: string
  ownerEmail?: string
  status?: string
  workflowStatus?: string
  timeline?: { status: string; at: string; note?: string }[]
  createdAt?: string
  updatedAt?: string
  runHistory?: { runId: string; at: string; dq_score: number; health: string }[]
  latestScore?: number | null
  latestHealth?: string | null
  latestFitForUse?: boolean
  latestSeverity?: { high: number; medium: number; low: number; total: number }
  latestRunId?: string
  latestDimensions?: Record<string, number>
  alertId?: string | null
  aiSummary?: AiSummary | null
}

export interface AiSummary {
  runId: string
  generatedAt: string
  executive_summary: string
  recommendations: string[]
  metrics_used?: Record<string, unknown>
  model?: string
}

export interface DqRun {
  runId: string
  datasetId: string
  createdAt: string
  dq_score: number
  health: string
  fit_for_analytics: boolean
  fit_for_use: boolean
  dimensions: Record<string, number>
  heatmap: { column: string; score: number; label: string; null_rate: number; type: string }[]
  issues: Issue[]
  severity: { high: number; medium: number; low: number; total: number }
  anomalies: { type: string; message: string }[]
  null_rate: number
  record_count: number
  rule_coverage?: { rules_evaluated: number; rules_failed: number }
  pii_hints?: { column: string; type: string }[]
  profile?: { record_count: number; column_count: number; columns: string[] }
}

export interface Issue {
  issue_id: string
  rule_name: string
  dimension: string
  severity: string
  status: string
  message: string
  threshold?: number
  current_score?: number
  history?: { status: string; at: string; note?: string; by?: string }[]
  assignedTo?: string
  column?: string
}
```

```ts

export interface Alert {
  alertId: string
  datasetId: string
  datasetName?: string
  note: string
  status: string
  dqScore?: number
  health?: string
  fromEmail?: string
  toUserId?: string
  createdAt: string
  history?: { status: string; at: string; note?: string; by?: string }[]
}
```

frontend/src/components/AppShell.tsx 58 lines

```tsx
import { Link, NavLink, Outlet } from 'react-router-dom'
import { useAuth } from '../auth/AuthContext'

export function AppShell() {
  const { user, logout } = useAuth()
  const isData = user?.role === 'DataUser'

  const linkClass = ({ isActive }: { isActive: boolean }) =>
    `px-3 py-2 rounded-md text-sm font-medium transition ${
      isActive ? 'bg-sea text-white' : 'text-ink-soft hover:bg-mist'
    }`

  return (
    <div className="min-h-screen flex flex-col">
      <header className="no-print border-b border-mist bg-white/80 backdrop-blur sticky top-0 z-20">
        <div className="mx-auto max-w-6xl px-4 py-3 flex items-center justify-between gap-4">
          <Link to="/" className="font-display text-xl text-sea tracking-tight">
            CloudDQ AI
          </Link>
          <nav className="flex flex-wrap items-center gap-1">
            <NavLink to="/app" end className={linkClass}>
              Home
            </NavLink>
            <NavLink to="/app/datasets" className={linkClass}>
              Datasets
            </NavLink>
            {isData && (
              <NavLink to="/app/upload" className={linkClass}>
                Upload
              </NavLink>
            )}
            <NavLink to="/app/alerts" className={linkClass}>
              Alerts
            </NavLink>
          </nav>
          <div className="flex items-center gap-3 text-sm">
            <span className="hidden sm:inline text-ink-soft">
              {user?.email} · <strong className="text-sea">{user?.role}</strong>
            </span>
            <button
              type="button"
              onClick={logout}
              className="rounded-md border border-mist px-3 py-1.5 hover:bg-mist"
            >
              Sign out
            </button>
          </div>
        </div>
      </header>
      <main className="flex-1 mx-auto w-full max-w-6xl px-4 py-8">
        <Outlet />
      </main>
      <footer className="no-print border-t border-mist py-4 text-center text-xs text-ink-soft">
        Serverless data quality · Amplify · Cognito · API Gateway · Lambda · S3 · DynamoDB · CloudWatch · Bedrock
      </footer>
    </div>
  )
}
```

frontend/src/components/Badges.tsx 55 lines

```tsx
export function HealthBadge({
  health,
  score,
  fit,
}: {
  health?: string | null
  score?: number | null
  fit?: boolean
}) {
  const h = (health || 'UNKNOWN').toUpperCase()
  const color =
    h === 'HEALTHY'
      ? 'bg-ok/15 text-ok border-ok/30'
      : h === 'WARNING'
        ? 'bg-amber/15 text-amber border-amber/30'
        : h === 'CRITICAL'
          ? 'bg-coral/15 text-coral border-coral/30'
          : 'bg-mist text-ink-soft border-mist'

  return (
    <div className={`inline-flex flex-col gap-1 rounded-lg border px-4 py-3 ${color}`}>
      <div className="text-2xl font-display font-semibold">
        {score != null ? `${score}%` : '—'}
      </div>
      <div className="text-xs font-semibold tracking-wide uppercase">{h}</div>
      {fit != null && (
        <div className="text-xs">
          {fit ? '✓ Fit for Analytics' : '✗ Not fit for Analytics'}
        </div>
      )}
    </div>
  )
}

export function SeverityPills({
  severity,
}: {
  severity?: { high: number; medium: number; low: number; total: number } | null
}) {
  if (!severity) return null
  return (
    <div className="flex flex-wrap gap-2 text-xs">
      <span className="rounded-full bg-coral/15 text-coral px-3 py-1">
        High {severity.high}
      </span>
      <span className="rounded-full bg-amber/15 text-amber px-3 py-1">
        Medium {severity.medium}
      </span>
      <span className="rounded-full bg-sea/10 text-sea px-3 py-1">
        Low {severity.low}
      </span>
      <span className="rounded-full bg-mist px-3 py-1">Total {severity.total}</span>
    </div>
  )
}
```

frontend/src/pages/LandingPage.tsx 108 lines

```tsx
import { Link } from 'react-router-dom'
import { useAuth } from '../auth/AuthContext'

export function LandingPage() {
  const { user } = useAuth()
  return (
    <div className="min-h-screen relative overflow-hidden">
      <div
        className="absolute inset-0 -z-10"
        style={{
          background:
            'radial-gradient(ellipse 80% 60% at 20% 10%, #248f7a33, transparent), radial-
gradient(ellipse 70% 50% at 90% 20%, #c4922a22, transparent), linear-gradient(160deg, #f3f7f4 0%, #dce8e1
45%, #e8f0ec 100%)',
        }}
      />
      <div
```

```
        className="absolute inset-0 -z-10 opacity-40"
        style={{
          backgroundImage:
            'url("data:image/svg+xml,%3Csvg width=\'60\' height=\'60\' viewBox=\'0 0 60 60\'
xmlns=\'http://www.w3.org/2000/svg\'%3E%3Cg fill=\'none\' fill-rule=\'evenodd\'%3E%3Cg fill=\'%231a6b5c\'
fill-opacity=\'0.06\'%3E%3Cpath d=\'M36 34v-4h-2v4h-4v2h4v4h2v-4h4v-2h-4zm0-30V0h-2v4h-4v2h4v4h2V6h4V4h-
4zM6 34v-4H4v4H0v2h4v4h2v-4h4v-2H6zM6 4V0H4v4H0v2h4v4h2V6h4V4H6z\'/%3E%3C/g%3E%3C/g%3E%3C/svg%3E")',
        }}
      />

      <header className="mx-auto max-w-6xl px-4 py-6 flex justify-between items-center">
        <span className="font-display text-2xl text-sea">CloudDQ AI</span>
        <Link
          to={user ? '/app' : '/login'}
          className="rounded-md bg-sea px-4 py-2 text-white text-sm font-medium hover:bg-sea-bright
transition"
        >
          {user ? 'Open app' : 'Sign in'}
        </Link>
      </header>

      <section className="mx-auto max-w-6xl px-4 pt-16 pb-24 grid lg:grid-cols-2 gap-12 items-center">
        <div className="space-y-6 animate-[fadeUp_0.7s_ease]">
          <p className="font-display text-5xl sm:text-6xl leading-tight text-ink">
            CloudDQ AI
          </p>
          <h1 className="text-xl sm:text-2xl text-ink-soft font-medium max-w-lg">
            Explainable, serverless data-quality assessment for CSV datasets — scored across six
dimensions, with AI executive insights.
          </h1>
          <p className="text-ink-soft max-w-md">
            Upload. Analyze. Visualize. Alert. Correct. Built on eight AWS services for Data Users and
Business Users.
          </p>
          <div className="flex flex-wrap gap-3">
            <Link
              to="/login"
              className="rounded-md bg-sea px-5 py-3 text-white font-medium hover:bg-sea-bright
transition shadow-sm"
            >
              Get started
            </Link>
            <a
              href="#how"
              className="rounded-md border border-sea/30 px-5 py-3 text-sea font-medium hover:bg-
white/60 transition"
            >
              How it works
            </a>
          </div>
        </div>

        <div className="relative animate-[fadeIn_1s_ease]">
          <div className="rounded-2xl bg-ink text-foam p-6 shadow-xl rotate-1 hover:rotate-0
transition duration-500">
            <div className="text-xs uppercase tracking-widest text-mist/80 mb-4">Live quality
pulse</div>
            <div className="font-display text-5xl mb-2">87%</div>
            <div className="text-sea-bright mb-6">HEALTHY · Fit for Analytics</div>
            <div className="grid grid-cols-3 gap-3 text-sm">
              {[
                ['Completeness', '94'],
                ['Validity', '88'],
                ['Uniqueness', '91'],
                ['Consistency', '85'],
                ['Accuracy', '82'],
                ['Timeliness', '79'],
              ].map(([k, v]) => (
                <div key={k} className="rounded-lg bg-white/5 p-3">
                  <div className="text-mist/70 text-xs">{k}</div>
                  <div className="font-semibold">{v}%</div>
                </div>
              ))}
            </div>
          </div>
        </div>
      </section>

      <section id="how" className="bg-white/70 border-t border-mist py-16">
        <div className="mx-auto max-w-6xl px-4">
```

```
          <h2 className="font-display text-3xl mb-8">Login → Upload → Analyze → Report → AI → Alert</h2>
          <div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-6 text-sm text-ink-soft">
            {[
              ['Six dimensions', 'Weighted Completeness, Validity, Uniqueness, Consistency, Accuracy, Timeliness.'],
              ['Role-aware SaaS', 'Data Users run quality; Business Users consume KPIs and raise alerts.'],
              ['Private by design', 'Presigned S3 uploads; Bedrock sees metrics JSON only — never raw CSV rows.'],
            ].map(([t, d]) => (
              <div key={t} className="border-l-2 border-sea pl-4">
                <div className="font-semibold text-ink mb-1">{t}</div>
                <p>{d}</p>
              </div>
            ))}
          </div>
        </div>
      </section>

      <style>{`
        @keyframes fadeUp { from { opacity: 0; transform: translateY(16px); } to { opacity: 1; transform: none; } }
        @keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }
      `}</style>
    </div>
  )
}
```

frontend/src/pages/HomePage.tsx 99 lines

```tsx
import { useEffect, useState } from 'react'
import { Link } from 'react-router-dom'
import { useAuth } from '../auth/AuthContext'
import { listAlerts, listDatasets } from '../lib/api'
import type { Alert, Dataset } from '../types'
import { HealthBadge } from '../components/Badges'

export function HomePage() {
  const { user } = useAuth()
  const [datasets, setDatasets] = useState<Dataset[]>([])
  const [alerts, setAlerts] = useState<Alert[]>([])

  useEffect(() => {
    listDatasets().then(setDatasets).catch(() => setDatasets([]))
    listAlerts().then(setAlerts).catch(() => setAlerts([]))
  }, [])

  const isData = user?.role === 'DataUser'
  const scored = datasets.filter((d) => d.latestScore != null)
  const avg =
    scored.length > 0
      ? Math.round(scored.reduce((s, d) => s + (d.latestScore || 0), 0) / scored.length)
      : null
  const critical = scored.filter((d) => d.latestHealth === 'CRITICAL').length
  const openAlerts = alerts.filter((a) => a.status === 'OPEN').length

  return (
    <div className="space-y-8">
      <div>
        <h1 className="font-display text-3xl">
          {isData ? 'Data User workspace' : 'Business executive view'}
        </h1>
        <p className="text-ink-soft mt-1">
          {isData
            ? 'Upload CSV datasets, run DQ analysis, and resolve quality alerts.'
            : 'Monitor fit-for-use status, read AI summaries, and alert Data Users when quality drops.'}
        </p>
      </div>

      <div className="grid sm:grid-cols-3 gap-4">
        <Kpi label="Datasets" value={String(datasets.length)} />
        <Kpi label="Avg DQ score" value={avg != null ? `${avg}%` : '—'} />
        <Kpi
          label={isData ? 'Open alerts' : 'Critical sets'}
          value={String(isData ? openAlerts : critical)}
        />
```

```
      </div>

      <div className="flex flex-wrap gap-3">
        {isData && (
          <Link to="/app/upload" className="rounded-md bg-sea text-white px-4 py-2 text-sm font-
medium">
            Upload CSV
          </Link>
        )}
        <Link to="/app/datasets" className="rounded-md border border-mist px-4 py-2 text-sm font-medium
hover:bg-mist">
          Browse repository
        </Link>
        <Link to="/app/alerts" className="rounded-md border border-mist px-4 py-2 text-sm font-medium
hover:bg-mist">
          Alerts
        </Link>
      </div>

      <section>
        <h2 className="font-display text-xl mb-4">Recent datasets</h2>
        <div className="grid gap-3">
          {datasets.slice(0, 5).map((d) => (
            <Link
              key={d.datasetId}
              to={`/app/datasets/${d.datasetId}`}
              className="flex items-center justify-between gap-4 rounded-xl border border-mist bg-
white/80 px-4 py-3 hover:border-sea/40 transition"
            >
              <div>
                <div className="font-medium">{d.name}</div>
                <div className="text-xs text-ink-soft">
                  {d.status} · {d.workflowStatus}
                </div>
              </div>
              <HealthBadge health={d.latestHealth} score={d.latestScore} fit={d.latestFitForUse} />
            </Link>
          ))}
          {datasets.length === 0 && (
            <p className="text-ink-soft text-sm">
              No datasets yet. {isData ? 'Upload a CSV to begin.' : 'Waiting for Data User uploads.'}
            </p>
          )}
        </div>
      </section>
    </div>
  )
}

function Kpi({ label, value }: { label: string; value: string }) {
  return (
    <div className="rounded-xl border border-mist bg-white/80 p-4">
      <div className="text-xs uppercase tracking-wide text-ink-soft">{label}</div>
      <div className="font-display text-3xl mt-1">{value}</div>
    </div>
  )
}
```

frontend/src/index.css 31 lines

```
@import "tailwindcss";

@theme {
  --font-display: "Fraunces", Georgia, serif;
  --font-body: "Source Sans 3", "Segoe UI", sans-serif;
  --color-ink: #0f1c2e;
  --color-ink-soft: #243447;
  --color-foam: #f3f7f4;
  --color-mist: #dce8e1;
  --color-sea: #1a6b5c;
  --color-sea-bright: #248f7a;
  --color-coral: #c45c3e;
  --color-amber: #c4922a;
  --color-ok: #2d7a4f;
}

@layer base {
  html {
```

```
    font-family: var(--font-body);
    color: var(--color-ink);
    background: var(--color-foam);
  }
  h1, h2, h3 {
    font-family: var(--font-display);
  }
}

@media print {
  .no-print { display: none !important; }
  body { background: white; }
}
```

frontend/index.html 19 lines

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>CloudDQ AI</title>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?
family=Fraunces:opsz,wght@9..144,500;9..144,700&family=Source+Sans+3:wght@400;500;600;700&display=swap"
rel="stylesheet" />
  </head>
  <body>
    <div id="root"></div>
    <script>
      window.global = window
    </script>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

frontend/vite.config.ts 13 lines

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
  define: {
    global: 'globalThis',
  },
  server: {
    port: 5173,
  },
})
```

frontend/package.json 31 lines

```
{
  "name": "clouddq-ai-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "oxlint",
    "preview": "vite preview"
  },
  "dependencies": {
    "amazon-cognito-identity-js": "^6.3.20",
    "axios": "^1.20.0",
    "react": "^19.2.8",
    "react-dom": "^19.2.8",
    "react-router-dom": "^7.18.4",
    "recharts": "^3.10.1"
  },
```

```
  "devDependencies": {
    "@tailwindcss/vite": "^4.3.3",
    "@types/node": "^24.13.3",
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "oxlint": "^1.81.0",
    "tailwindcss": "^4.3.3",
    "typescript": "~6.0.2",
    "vite": "^8.3.0"
  }
}
```

M3 — Dataset Upload & Repository

backend/src/datasets/__init__.py 4 lines

```python
"""Dataset API package."""
from . import service

__all__ = ["service"]
```

backend/src/datasets/service.py 342 lines

```python
"""Dataset upload, registry, analyze, report, export APIs."""
from __future__ import annotations

import csv
import io
from typing import Any

from ai import comprehend
from auth import require_role
from dq_engine import run_analysis
from store import dynamo, s3


STATUS_FLOW = [
    "Uploaded",
    "Processing",
    "Analyzed",
    "ReportGenerated",
    "Completed",
]


def _append_timeline(meta: dict, status: str, note: str = "") -> list:
    timeline = list(meta.get("timeline") or [])
    timeline.append({"status": status, "at": dynamo.now_iso(), "note": note})
    return timeline


def create_upload_url(user: dict, body: dict) -> dict:
    require_role(user, "DataUser")
    filename = body.get("filename") or "dataset.csv"
    if not filename.lower().endswith(".csv"):
        raise ValueError("Only CSV files are supported")
    dataset_id = dynamo.new_id()
    key = f"datasets/{user['sub']}/{dataset_id}/{filename}"
    url = s3.presigned_put_url(key, content_type=body.get("contentType") or "text/csv")
    return {
        "datasetId": dataset_id,
        "uploadUrl": url,
        "s3Key": key,
        "expiresIn": 900,
    }


def register_dataset(user: dict, body: dict) -> dict:
    require_role(user, "DataUser")
    dataset_id = body.get("datasetId") or dynamo.new_id()
    s3_key = body.get("s3Key")
    name = body.get("name") or body.get("filename") or "Untitled Dataset"
    if not s3_key:
        raise ValueError("s3Key is required")

    now = dynamo.now_iso()
    item = {
        "PK": f"DATASET#{dataset_id}",
        "SK": "META",
        "entityType": "DATASET",
        "datasetId": dataset_id,
        "name": name,
        "filename": body.get("filename") or name,
        "s3Key": s3_key,
        "ownerId": user["sub"],
        "ownerEmail": user.get("email"),
```

```
        "status": "Uploaded",
        "workflowStatus": "Completed",
        "timeline": [{"status": "Uploaded", "at": now, "note": "CSV uploaded"}],
        "createdAt": now,
        "updatedAt": now,
        "GSI1PK": f"USER#{user['sub']}",
        "GSI1SK": f"DATASET#{now}",
        "runHistory": [],
        "latestScore": None,
        "latestHealth": None,
        "alertId": None,
        "aiSummary": None,
    }
    dynamo.put_item(item)
    return _public_dataset(item)


def list_datasets(user: dict) -> list[dict]:
    if user["role"] == "DataUser":
        items = dynamo.query_gsi1(f"USER#{user['sub']}", "DATASET#")
        # GSI returns META rows that have GSI1; filter META
        return [_public_dataset(i) for i in items if i.get("SK") == "META" or i.get("entityType") ==
"DATASET"]
    # Business users see all datasets (demo SaaS)
    items = dynamo.scan_entity("DATASET#")
    return [_public_dataset(i) for i in items if i.get("SK") == "META"]


def get_dataset(user: dict, dataset_id: str) -> dict:
    item = dynamo.get_item(f"DATASET#{dataset_id}", "META")
    if not item:
        raise LookupError("Dataset not found")
    _assert_can_view(user, item)
    return _public_dataset(item)


def analyze_dataset(user: dict, dataset_id: str, body: dict | None = None) -> dict:
    require_role(user, "DataUser")
    meta = dynamo.get_item(f"DATASET#{dataset_id}", "META")
    if not meta:
        raise LookupError("Dataset not found")
    if meta.get("ownerId") != user["sub"]:
        raise PermissionError("Only the dataset owner can run analysis")

    body = body or {}
    # Optional correction re-upload key
    if body.get("s3Key"):
        meta = dynamo.update_item(
            f"DATASET#{dataset_id}",
            "META",
            {"s3Key": body["s3Key"], "workflowStatus": "Analyzing"},
        )

    timeline = _append_timeline(meta, "Processing", "DQ analysis started")
    dynamo.update_item(
        f"DATASET#{dataset_id}",
        "META",
        {"status": "Processing", "timeline": timeline, "workflowStatus": "Analyzing"},
    )

    csv_text = s3.get_object_text(meta["s3Key"] if not body.get("s3Key") else body["s3Key"])

    # Previous run for anomalies / trends
    runs = dynamo.query_pk(f"DATASET#{dataset_id}", "RUN#")
    runs_sorted = sorted(runs, key=lambda r: r.get("SK", ""), reverse=True)
    previous = runs_sorted[0] if runs_sorted else None

    result = run_analysis(csv_text, previous_run=previous)
    result["pii_hints"] = comprehend.detect_pii_columns(result["profile"], result["pii_hints"])
    run_ts = dynamo.now_iso()
    run_id = run_ts.replace(":", "-")

    run_item = {
        "PK": f"DATASET#{dataset_id}",
        "SK": f"RUN#{run_ts}",
        "entityType": "RUN",
        "datasetId": dataset_id,
        "runId": run_id,
        "createdAt": run_ts,
        "createdBy": user["sub"],
```

```
        **{k: v for k, v in result.items()},
    }
    dynamo.put_item(run_item)

    # Persist issues as separate items
    for issue in result["issues"]:
        issue_item = {
            "PK": f"DATASET#{dataset_id}",
            "SK": f"ISSUE#{issue['issue_id']}",
            "entityType": "ISSUE",
            "datasetId": dataset_id,
            "runId": run_id,
            "assignedTo": user["sub"],
            "history": [
                {
                    "status": "OPEN",
                    "at": run_ts,
                    "note": "Issue generated after rule breach",
                    "by": "system",
                }
            ],
            **issue,
        }
        dynamo.put_item(issue_item)

    run_history = list(meta.get("runHistory") or [])
    run_history.append(
        {
            "runId": run_id,
            "at": run_ts,
            "dq_score": result["dq_score"],
            "health": result["health"],
        }
    )
    # Keep last 20 for trend without new table
    run_history = run_history[-20:]

    timeline = _append_timeline(
        {"timeline": timeline}, "Analyzed", f"DQ score {result['dq_score']}%"
    )
    timeline = _append_timeline(
        {"timeline": timeline}, "ReportGenerated", "Quality report ready"
    )
    timeline = _append_timeline({"timeline": timeline}, "Completed", "Analysis complete")

    workflow = "Completed"
    if meta.get("workflowStatus") in {"AlertRaised", "InCorrection", "Analyzing"}:
        # After correction re-run, mark resolved path if healthy enough
        if result["dq_score"] >= 60:
            workflow = "Resolved"
        else:
            workflow = "Completed"

    updated = dynamo.update_item(
        f"DATASET#{dataset_id}",
        "META",
        {
            "status": "Completed",
            "workflowStatus": workflow,
            "timeline": timeline,
            "runHistory": run_history,
            "latestScore": result["dq_score"],
            "latestHealth": result["health"],
            "latestFitForUse": result["fit_for_use"],
            "latestSeverity": result["severity"],
            "latestRunId": run_id,
            "latestDimensions": result["dimensions"],
            "aiSummary": None,  # invalidate cached summary
        },
    )
    return {
        "dataset": _public_dataset(updated),
        "run": _public_run(run_item),
    }


def list_runs(user: dict, dataset_id: str) -> list[dict]:
    meta = dynamo.get_item(f"DATASET#{dataset_id}", "META")
    if not meta:
        raise LookupError("Dataset not found")
```

```
    _assert_can_view(user, meta)
    runs = dynamo.query_pk(f"DATASET#{dataset_id}", "RUN#")
    runs_sorted = sorted(runs, key=lambda r: r.get("SK", ""))
    return [_public_run(r) for r in runs_sorted]


def get_report(user: dict, dataset_id: str) -> dict:
    meta = dynamo.get_item(f"DATASET#{dataset_id}", "META")
    if not meta:
        raise LookupError("Dataset not found")
    _assert_can_view(user, meta)
    runs = dynamo.query_pk(f"DATASET#{dataset_id}", "RUN#")
    if not runs:
        raise LookupError("No analysis runs yet")
    latest = sorted(runs, key=lambda r: r.get("SK", ""), reverse=True)[0]
    issues = dynamo.query_pk(f"DATASET#{dataset_id}", "ISSUE#")
    # Prefer issues from latest run
    latest_issues = [i for i in issues if i.get("runId") == latest.get("runId")] or issues
    return {
        "dataset": _public_dataset(meta),
        "run": _public_run(latest),
        "issues": [_public_issue(i) for i in latest_issues],
    }


def export_report_csv(user: dict, dataset_id: str) -> dict:
    report = get_report(user, dataset_id)
    run = report["run"]
    buf = io.StringIO()
    writer = csv.writer(buf)
    writer.writerow(["Metric", "Value"])
    writer.writerow(["DQ Score", run.get("dq_score")])
    writer.writerow(["Health", run.get("health")])
    writer.writerow(["Fit for Analytics", run.get("fit_for_analytics")])
    for dim, val in (run.get("dimensions") or {}).items():
        writer.writerow([f"Dimension:{dim}", val])
    writer.writerow([])
    writer.writerow(["Issue", "Severity", "Status", "Message"])
    for issue in report["issues"]:
        writer.writerow(
            [
                issue.get("rule_name"),
                issue.get("severity"),
                issue.get("status"),
                issue.get("message"),
            ]
        )
    return {
        "filename": f"{report['dataset'].get('name', dataset_id)}_dq_report.csv",
        "contentType": "text/csv",
        "content": buf.getvalue(),
    }


def list_issues(user: dict, dataset_id: str) -> list[dict]:
    meta = dynamo.get_item(f"DATASET#{dataset_id}", "META")
    if not meta:
        raise LookupError("Dataset not found")
    _assert_can_view(user, meta)
    issues = dynamo.query_pk(f"DATASET#{dataset_id}", "ISSUE#")
    return [_public_issue(i) for i in issues]


def enter_correction(user: dict, dataset_id: str, body: dict | None = None) -> dict:
    require_role(user, "DataUser")
    meta = dynamo.get_item(f"DATASET#{dataset_id}", "META")
    if not meta:
        raise LookupError("Dataset not found")
    note = (body or {}).get("note") or "Moved to correction flow"
    timeline = _append_timeline(meta, "InCorrection", note)
    updated = dynamo.update_item(
        f"DATASET#{dataset_id}",
        "META",
        {"workflowStatus": "InCorrection", "timeline": timeline},
    )
    return _public_dataset(updated)


def _assert_can_view(user: dict, meta: dict) -> None:
    if user["role"] == "BusinessUser":
```

```
        return
    if meta.get("ownerId") == user["sub"]:
        return
    raise PermissionError("Not allowed to view this dataset")


def _public_dataset(item: dict) -> dict:
    keys = [
        "datasetId",
        "name",
        "filename",
        "s3Key",
        "ownerId",
        "ownerEmail",
        "status",
        "workflowStatus",
        "timeline",
        "createdAt",
        "updatedAt",
        "runHistory",
        "latestScore",
        "latestHealth",
        "latestFitForUse",
        "latestSeverity",
        "latestRunId",
        "latestDimensions",
        "alertId",
        "aiSummary",
    ]
    return {k: item.get(k) for k in keys if k in item or True}


def _public_run(item: dict) -> dict:
    omit = {"PK", "SK", "entityType"}
    return {k: v for k, v in item.items() if k not in omit}


def _public_issue(item: dict) -> dict:
    omit = {"PK", "SK", "entityType"}
    return {k: v for k, v in item.items() if k not in omit}
```

backend/src/store/__init__.py — 4 lines

```
"""Store package."""
from . import dynamo, s3

__all__ = ["dynamo", "s3"]
```

backend/src/store/s3.py — 42 lines

```
"""S3 helpers — private CSV storage + presigned uploads."""
from __future__ import annotations

import os
from typing import BinaryIO

import boto3

BUCKET_NAME = os.environ.get("BUCKET_NAME", "")
_s3 = None


def _client():
    global _s3
    if _s3 is None:
        _s3 = boto3.client("s3")
    return _s3


def presigned_put_url(key: str, content_type: str = "text/csv", expires: int = 900) -> str:
    return _client().generate_presigned_url(
        "put_object",
        Params={
            "Bucket": BUCKET_NAME,
            "Key": key,
            "ContentType": content_type,
```

```
        },
        ExpiresIn=expires,
    )


def get_object_bytes(key: str) -> bytes:
    resp = _client().get_object(Bucket=BUCKET_NAME, Key=key)
    return resp["Body"].read()


def get_object_text(key: str) -> str:
    return get_object_bytes(key).decode("utf-8-sig")


def put_object(key: str, body: bytes | BinaryIO, content_type: str = "text/csv") -> None:
    _client().put_object(Bucket=BUCKET_NAME, Key=key, Body=body, ContentType=content_type)
```

frontend/src/lib/api.ts 142 lines

```
import axios from 'axios'
import { config } from '../config'
import type { Alert, AiSummary, Dataset, DqRun, Issue } from '../types'

let authToken = ''
let devRole = 'DataUser'
let devSub = 'dev-user'
let devEmail = 'dev@example.com'

export function setApiAuth(opts: {
  token: string
  role?: string
  sub?: string
  email?: string
}) {
  authToken = opts.token
  if (opts.role) devRole = opts.role
  if (opts.sub) devSub = opts.sub
  if (opts.email) devEmail = opts.email
}

const client = axios.create({
  baseURL: config.apiBaseUrl,
  timeout: 120000,
})

client.interceptors.request.use((req) => {
  req.headers = req.headers || {}
  if (authToken) {
    req.headers.Authorization = `Bearer ${authToken}`
  }
  if (config.devAuth) {
    req.headers['X-Dev-Role'] = devRole
    req.headers['X-Dev-Sub'] = devSub
    req.headers['X-Dev-Email'] = devEmail
  }
  return req
})

export async function getUploadUrl(filename: string) {
  const { data } = await client.post<{
    datasetId: string
    uploadUrl: string
    s3Key: string
  }>('/datasets/upload-url', { filename, contentType: 'text/csv' })
  return data
}

export async function uploadToS3(uploadUrl: string, file: File) {
  await axios.put(uploadUrl, file, {
    headers: { 'Content-Type': 'text/csv' },
  })
}

export async function registerDataset(payload: {
  datasetId: string
  s3Key: string
  name: string
  filename: string
```

```ts
}) {
  const { data } = await client.post<Dataset>('/datasets', payload)
  return data
}

export async function listDatasets() {
  const { data } = await client.get<{ datasets: Dataset[] }>('/datasets')
  return data.datasets
}

export async function getDataset(id: string) {
  const { data } = await client.get<Dataset>(`/datasets/${id}`)
  return data
}

export async function analyzeDataset(id: string, body?: { s3Key?: string }) {
  const { data } = await client.post<{ dataset: Dataset; run: DqRun }>(
    `/datasets/${id}/analyze`,
    body || {},
  )
  return data
}

export async function getRuns(id: string) {
  const { data } = await client.get<{ runs: DqRun[] }>(`/datasets/${id}/runs`)
  return data.runs
}

export async function getReport(id: string) {
  const { data } = await client.get<{
    dataset: Dataset
    run: DqRun
    issues: Issue[]
  }>(`/datasets/${id}/report`)
  return data
}

export async function exportReport(id: string) {
  const { data } = await client.get<{
    filename: string
    contentType: string
    content: string
  }>(`/datasets/${id}/export`)
  return data
}

export async function getAiSummary(id: string) {
  const { data } = await client.post<AiSummary>(`/datasets/${id}/summary`, {})
  return data
}

export async function listIssues(id: string) {
  const { data } = await client.get<{ issues: Issue[] }>(`/datasets/${id}/issues`)
  return data.issues
}

export async function enterCorrection(id: string, note?: string) {
  const { data } = await client.post<Dataset>(`/datasets/${id}/correction`, { note })
  return data
}

export async function createAlert(datasetId: string, note: string, force = false) {
  const { data } = await client.post<Alert>('/alerts', { datasetId, note, force })
  return data
}

export async function listAlerts() {
  const { data } = await client.get<{ alerts: Alert[] }>('/alerts')
  return data.alerts
}

export async function issueAction(
  issueId: string,
  action: 'assign' | 'acknowledge' | 'resolve' | 'reopen',
  datasetId: string,
  note?: string,
) {
  const { data } = await client.post<Issue>(`/issues/${issueId}/${action}`, {
    datasetId,
    note,
```

```
  })
  return data
}
```

frontend/src/pages/UploadPage.tsx 108 lines

```tsx
import { useState, type FormEvent } from 'react'
import { useNavigate } from 'react-router-dom'
import { analyzeDataset, getUploadUrl, registerDataset, uploadToS3 } from '../lib/api'
import { config } from '../config'

export function UploadPage() {
  const navigate = useNavigate()
  const [file, setFile] = useState<File [] null>(null)
  const [name, setName] = useState('')
  const [busy, setBusy] = useState(false)
  const [step, setStep] = useState('')
  const [error, setError] = useState('')
  const [autoAnalyze, setAutoAnalyze] = useState(true)

  async function onSubmit(e: FormEvent) {
    e.preventDefault()
    if (!file) return
    setBusy(true)
    setError('')
    try {
      setStep('Requesting presigned upload URL…')
      const { datasetId, uploadUrl, s3Key } = await getUploadUrl(file.name)

      if (config.devAuth && uploadUrl.includes('amazonaws.com') === false) {
        // Local mock path handled by backend mock if present
      }

      setStep('Uploading CSV directly to S3…')
      try {
        await uploadToS3(uploadUrl, file)
      } catch (uploadErr) {
        if (!config.devAuth) throw uploadErr
        // In pure local mock without S3, registration may still work if backend stores inline — skip
        console.warn('S3 upload failed in dev; continuing if backend allows', uploadErr)
      }

      setStep('Registering dataset…')
      const ds = await registerDataset({
        datasetId,
        s3Key,
        name: name || file.name.replace(/\.csv$/i, ''),
        filename: file.name,
      })

      if (autoAnalyze) {
        setStep('Running DQ analysis…')
        await analyzeDataset(ds.datasetId)
      }

      navigate(`/app/datasets/${ds.datasetId}`)
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Upload failed')
    } finally {
      setBusy(false)
      setStep('')
    }
  }

  return (
    <div className="max-w-xl space-y-6">
      <div>
        <h1 className="font-display text-3xl">Upload CSV</h1>
        <p className="text-ink-soft text-sm mt-1">
          Browser never receives AWS keys — uploads use short-lived presigned S3 URLs.
        </p>
      </div>

      <form onSubmit={onSubmit} className="rounded-xl border border-mist bg-white/80 p-6 space-y-4">
        <label className="block text-sm">
          <span className="font-medium">Display name</span>
          <input
            value={name}
```

```
            onChange={(e) => setName(e.target.value)}
            placeholder="Customer Master"
            className="mt-1 w-full rounded-md border border-mist px-3 py-2"
          />
        </label>
        <label className="block text-sm">
          <span className="font-medium">CSV file</span>
          <input
            type="file"
            accept=".csv,text/csv"
            required
            onChange={(e) => setFile(e.target.files?.[0] || null)}
            className="mt-1 w-full text-sm"
          />
        </label>
        <label className="flex items-center gap-2 text-sm">
          <input
            type="checkbox"
            checked={autoAnalyze}
            onChange={(e) => setAutoAnalyze(e.target.checked)}
          />
          Run DQ analysis immediately after upload
        </label>
        {step && <p className="text-sm text-sea">{step}</p>}
        {error && <p className="text-sm text-coral">{error}</p>}
        <button
          type="submit"
          disabled={busy || !file}
          className="rounded-md bg-sea text-white px-4 py-2 font-medium disabled:opacity-60"
        >
          {busy ? 'Working…' : 'Upload'}
        </button>
      </form>
    </div>
  )
}
```

frontend/src/pages/DatasetsPage.tsx — 96 lines

```
import { useEffect, useState } from 'react'
import { Link } from 'react-router-dom'
import { listDatasets } from '../lib/api'
import type { Dataset } from '../types'
import { HealthBadge } from '../components/Badges'
import { useAuth } from '../auth/AuthContext'

const SAMPLES = [
  { name: 'Customers (dirty)', file: 'customers_dirty.csv', hint: 'Duplicates, bad emails, future dates' },
  { name: 'Orders (dirty)', file: 'orders_dirty.csv', hint: 'Negative prices, case inconsistency' },
  { name: 'Employees (clean)', file: 'employees_clean.csv', hint: 'Healthy baseline dataset' },
]

export function DatasetsPage() {
  const { user } = useAuth()
  const [datasets, setDatasets] = useState<Dataset[]>([])
  const [error, setError] = useState('')

  useEffect(() => {
    listDatasets()
      .then(setDatasets)
      .catch((e) => setError(e instanceof Error ? e.message : 'Failed to load'))
  }, [])

  return (
    <div className="space-y-8">
      <div>
        <div>
          <h1 className="font-display text-3xl">Dataset repository</h1>
          <p className="text-ink-soft text-sm mt-1">Status timeline, scores, and historical runs</p>
        </div>
        {user?.role === 'DataUser' && (
          <Link to="/app/upload" className="rounded-md bg-sea text-white px-4 py-2 text-sm font-medium">
            Upload new
          </Link>
        )}
      </div>
```

```
      {error && <p className="text-coral text-sm">{error}</p>}

      <div className="overflow-x-auto rounded-xl border border-mist bg-white/80">
        <table className="w-full text-sm">
          <thead className="bg-mist/50 text-left">
            <tr>
              <th className="px-4 py-3">Name</th>
              <th className="px-4 py-3">Status</th>
              <th className="px-4 py-3">Workflow</th>
              <th className="px-4 py-3">Quality</th>
              <th className="px-4 py-3"></th>
            </tr>
          </thead>
          <tbody>
            {datasets.map((d) => (
              <tr key={d.datasetId} className="border-t border-mist">
                <td className="px-4 py-3 font-medium">{d.name}</td>
                <td className="px-4 py-3">{d.status}</td>
                <td className="px-4 py-3">{d.workflowStatus}</td>
                <td className="px-4 py-3">
                  <HealthBadge health={d.latestHealth} score={d.latestScore} fit={d.latestFitForUse} />
                </td>
                <td className="px-4 py-3 text-right">
                  <Link to={`/app/datasets/${d.datasetId}`} className="text-sea font-medium
hover:underline">
                    Open
                  </Link>
                </td>
              </tr>
            ))}
            {datasets.length === 0 && (
              <tr>
                <td colSpan={5} className="px-4 py-8 text-center text-ink-soft">
                  No datasets registered yet.
                </td>
              </tr>
            )}
          </tbody>
        </table>
      </div>

      <section>
        <h2 className="font-display text-xl mb-3">Sample dirty datasets</h2>
        <p className="text-sm text-ink-soft mb-4">
          Download from the repo <code className="text-sea">samples/</code> folder, then upload as a
Data User.
        </p>
        <div className="grid sm:grid-cols-3 gap-3">
          {SAMPLES.map((s) => (
            <div key={s.file} className="rounded-xl border border-mist bg-white/80 p-4">
              <div className="font-medium">{s.name}</div>
              <div className="text-xs text-ink-soft mt-1">{s.hint}</div>
              <div className="text-xs mt-2 font-mono text-sea">{s.file}</div>
            </div>
          ))}
        </div>
      </section>
    </div>
  )
}
```

M4 — Data Quality Engine

backend/src/dq_engine/__init__.py 4 lines

```python
"""Data quality engine package."""
from .engine import run_analysis

__all__ = ["run_analysis"]
```

backend/src/dq_engine/profiler.py 97 lines

```python
"""CSV dataset profiling."""
from __future__ import annotations

import csv
import io
import re
from collections import Counter
from typing import Any


EMAIL_RE = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")
PHONE_RE = re.compile(r"^[\d\s\-\+\(\)]{7,20}$")
DATE_RE = re.compile(
    r"^(\d{4}-\d{2}-\d{2}|\d{2}/\d{2}/\d{4}|\d{2}-\d{2}-\d{4})$"
)
INT_RE = re.compile(r"^-?\d+$")
FLOAT_RE = re.compile(r"^-?\d+(\.\d+)?$")


def _is_empty(value: Any) -> bool:
    if value is None:
        return True
    s = str(value).strip()
    return s == "" or s.lower() in {"null", "none", "na", "n/a", "-"}


def infer_column_type(name: str, samples: list[str]) -> str:
    lower = name.lower()
    if "email" in lower:
        return "email"
    if "phone" in lower or "mobile" in lower:
        return "phone"
    if any(k in lower for k in ("date", "created", "updated", "timestamp", "dob")):
        return "date"
    if any(k in lower for k in ("id", "key", "code")):
        return "id"
    if any(k in lower for k in ("age", "amount", "price", "qty", "quantity", "score", "count")):
        return "numeric"
    if any(k in lower for k in ("gender", "sex", "status", "country", "state", "city")):
        return "categorical"

    non_empty = [s for s in samples if not _is_empty(s)]
    if not non_empty:
        return "string"
    email_hits = sum(1 for s in non_empty if EMAIL_RE.match(s))
    if email_hits / len(non_empty) > 0.6:
        return "email"
    date_hits = sum(1 for s in non_empty if DATE_RE.match(s))
    if date_hits / len(non_empty) > 0.6:
        return "date"
    num_hits = sum(1 for s in non_empty if FLOAT_RE.match(s))
    if num_hits / len(non_empty) > 0.8:
        return "numeric"
    return "string"


def profile_csv(text: str) -> dict[str, Any]:
    reader = csv.DictReader(io.StringIO(text))
    columns = reader.fieldnames or []
    rows: list[dict[str, str]] = []
    for row in reader:
        rows.append({k: (v if v is not None else "") for k, v in row.items()})

```

```
    record_count = len(rows)
    column_profiles: list[dict[str, Any]] = []

    for col in columns:
        values = [row.get(col, "") for row in rows]
        empty_count = sum(1 for v in values if _is_empty(v))
        non_empty = [v.strip() for v in values if not _is_empty(v)]
        unique_count = len(set(v.lower() for v in non_empty))
        samples = non_empty[:50]
        col_type = infer_column_type(col, samples)
        value_counts = Counter(v.lower() for v in non_empty)
        top_values = [{"value": k, "count": c} for k, c in value_counts.most_common(5)]

        null_rate = (empty_count / record_count * 100) if record_count else 0.0
        column_profiles.append(
            {
                "name": col,
                "type": col_type,
                "null_count": empty_count,
                "null_rate": round(null_rate, 2),
                "unique_count": unique_count,
                "non_empty_count": len(non_empty),
                "top_values": top_values,
                "sample_values": samples[:10],
            }
        )

    return {
        "record_count": record_count,
        "column_count": len(columns),
        "columns": columns,
        "column_profiles": column_profiles,
        "rows": rows,  # kept in-memory for scoring only; never sent to Bedrock
    }
```

backend/src/dq_engine/dimensions.py 276 lines

```
"""Six-dimension DQ scoring + overall health."""
from __future__ import annotations

import re
from collections import Counter, defaultdict
from datetime import datetime, timezone
from typing import Any

WEIGHTS = {
    "completeness": 0.25,
    "validity": 0.20,
    "uniqueness": 0.15,
    "consistency": 0.15,
    "accuracy": 0.15,
    "timeliness": 0.10,
}

EMAIL_RE = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")
PHONE_RE = re.compile(r"^[\d\s\-\+\(\)]{7,20}$")
DATE_RE = re.compile(r"^(\d{4}-\d{2}-\d{2}|\d{2}/\d{2}/\d{4}|\d{2}-\d{2}-\d{4})$")
FLOAT_RE = re.compile(r"^-?\d+(\.\d+)?$")


def _empty(v: Any) -> bool:
    if v is None:
        return True
    s = str(v).strip()
    return s == "" or s.lower() in {"null", "none", "na", "n/a", "-"}


def _parse_date(s: str) -> datetime | None:
    for fmt in ("%Y-%m-%d", "%d/%m/%Y", "%m/%d/%Y", "%d-%m-%Y"):
        try:
            return datetime.strptime(s.strip(), fmt).replace(tzinfo=timezone.utc)
        except ValueError:
            continue
    return None


def score_completeness(profile: dict) -> tuple[float, dict]:
    total = profile["record_count"] * max(profile["column_count"], 1)
```

```
    if total == 0:
        return 100.0, {"null_cells": 0, "total_cells": 0}
    null_cells = sum(c["null_count"] for c in profile["column_profiles"])
    score = max(0.0, 100.0 - (null_cells / total * 100))
    return round(score, 2), {"null_cells": null_cells, "total_cells": total}


def score_validity(profile: dict, rows: list[dict]) -> tuple[float, dict]:
    if not rows:
        return 100.0, {"invalid": 0, "checked": 0}
    invalid = 0
    checked = 0
    for col in profile["column_profiles"]:
        name, ctype = col["name"], col["type"]
        for row in rows:
            v = row.get(name, "")
            if _empty(v):
                continue
            checked += 1
            s = str(v).strip()
            ok = True
            if ctype == "email":
                ok = bool(EMAIL_RE.match(s))
            elif ctype == "phone":
                ok = bool(PHONE_RE.match(s))
            elif ctype == "date":
                ok = bool(DATE_RE.match(s))
            elif ctype == "numeric":
                ok = bool(FLOAT_RE.match(s))
            if not ok:
                invalid += 1
    if checked == 0:
        return 100.0, {"invalid": 0, "checked": 0}
    score = max(0.0, 100.0 - (invalid / checked * 100))
    return round(score, 2), {"invalid": invalid, "checked": checked}


def score_uniqueness(profile: dict, rows: list[dict]) -> tuple[float, dict]:
    if not rows:
        return 100.0, {"duplicate_keys": 0}
    # Prefer ID-like columns as keys
    id_cols = [c["name"] for c in profile["column_profiles"] if c["type"] == "id"]
    if not id_cols and profile["columns"]:
        id_cols = [profile["columns"][0]]

    duplicate_keys = 0
    for col in id_cols:
        values = [str(r.get(col, "")).strip().lower() for r in rows if not _empty(r.get(col))]
        counts = Counter(values)
        duplicate_keys += sum(c - 1 for c in counts.values() if c > 1)

    total = len(rows) * max(len(id_cols), 1)
    rate = (duplicate_keys / total * 100) if total else 0
    score = max(0.0, 100.0 - rate)
    return round(score, 2), {"duplicate_keys": duplicate_keys, "id_columns": id_cols}


def score_consistency(profile: dict, rows: list[dict]) -> tuple[float, dict]:
    if not rows:
        return 100.0, {"inconsistent_groups": 0}
    inconsistent = 0
    checked_groups = 0
    for col in profile["column_profiles"]:
        if col["type"] not in {"categorical", "string"}:
            continue
        name = col["name"]
        groups: dict[str, set[str]] = defaultdict(set)
        for row in rows:
            v = row.get(name, "")
            if _empty(v):
                continue
            s = str(v).strip()
            groups[s.lower()].add(s)
        for variants in groups.values():
            checked_groups += 1
            if len(variants) > 1:
                inconsistent += 1
    if checked_groups == 0:
        return 100.0, {"inconsistent_groups": 0}
    score = max(0.0, 100.0 - (inconsistent / checked_groups * 100))
```

```
    return round(score, 2), {"inconsistent_groups": inconsistent, "checked_groups": checked_groups}


def score_accuracy(profile: dict, rows: list[dict]) -> tuple[float, dict]:
    if not rows:
        return 100.0, {"out_of_range": 0}
    out = 0
    checked = 0
    for col in profile["column_profiles"]:
        name, ctype = col["name"], col["type"]
        lower = name.lower()
        for row in rows:
            v = row.get(name, "")
            if _empty(v):
                continue
            s = str(v).strip()
            if ctype == "numeric" or "age" in lower:
                if not FLOAT_RE.match(s):
                    continue
                checked += 1
                num = float(s)
                if "age" in lower and (num < 0 or num > 120):
                    out += 1
                elif "amount" in lower or "price" in lower:
                    if num < 0:
                        out += 1
                elif num < -1e12 or num > 1e12:
                    out += 1
            elif ctype == "email":
                checked += 1
                if not EMAIL_RE.match(s):
                    out += 1
    if checked == 0:
        return 100.0, {"out_of_range": 0, "checked": 0}
    score = max(0.0, 100.0 - (out / checked * 100))
    return round(score, 2), {"out_of_range": out, "checked": checked}


def score_timeliness(profile: dict, rows: list[dict]) -> tuple[float, dict]:
    date_cols = [c["name"] for c in profile["column_profiles"] if c["type"] == "date"]
    if not date_cols or not rows:
        return 90.0, {"freshness_delay_days": 0, "future_dates": 0, "note": "no date columns"}

    now = datetime.now(timezone.utc)
    future = 0
    latest: datetime | None = None
    dated = 0
    for col in date_cols:
        for row in rows:
            v = row.get(col, "")
            if _empty(v):
                continue
            dt = _parse_date(str(v))
            if not dt:
                continue
            dated += 1
            if dt > now:
                future += 1
            if latest is None or dt > latest:
                latest = dt

    delay_days = 0
    if latest:
        delay_days = max(0, (now.date() - latest.date()).days)

    # Penalize future dates and staleness (>7 days)
    future_penalty = (future / dated * 50) if dated else 0
    stale_penalty = min(40.0, delay_days * 2) if delay_days > 7 else 0
    score = max(0.0, 100.0 - future_penalty - stale_penalty)
    return round(score, 2), {
        "freshness_delay_days": delay_days,
        "future_dates": future,
        "latest_date": latest.date().isoformat() if latest else None,
    }


def classify_health(score: float) -> dict[str, Any]:
    if score >= 80:
        status = "HEALTHY"
        fit = True
```

```
    elif score >= 60:
        status = "WARNING"
        fit = False
    else:
        status = "CRITICAL"
        fit = False
    return {
        "health": status,
        "fit_for_analytics": fit,
        "fit_for_use": fit,
    }


def compute_dimensions(profile: dict) -> dict[str, Any]:
    rows = profile.get("rows") or []
    completeness, c_detail = score_completeness(profile)
    validity, v_detail = score_validity(profile, rows)
    uniqueness, u_detail = score_uniqueness(profile, rows)
    consistency, k_detail = score_consistency(profile, rows)
    accuracy, a_detail = score_accuracy(profile, rows)
    timeliness, t_detail = score_timeliness(profile, rows)

    dimensions = {
        "completeness": completeness,
        "validity": validity,
        "uniqueness": uniqueness,
        "consistency": consistency,
        "accuracy": accuracy,
        "timeliness": timeliness,
    }
    overall = round(
        sum(dimensions[k] * WEIGHTS[k] for k in WEIGHTS),
        2,
    )
    health = classify_health(overall)

    # Column heatmap: blend null rate + validity proxy
    heatmap = []
    for col in profile["column_profiles"]:
        col_score = max(0.0, 100.0 - col["null_rate"])
        if col["type"] in {"email", "phone", "date", "numeric"}:
            # slight penalty already in null; keep simple
            pass
        label = "High Quality"
        if col_score < 60:
            label = "Critical"
        elif col_score < 80:
            label = "Warning"
        elif col_score < 90:
            label = "Good"
        heatmap.append(
            {
                "column": col["name"],
                "score": round(col_score, 2),
                "label": label,
                "null_rate": col["null_rate"],
                "type": col["type"],
            }
        )

    return {
        "dq_score": overall,
        "dimensions": dimensions,
        "weights": WEIGHTS,
        "details": {
            "completeness": c_detail,
            "validity": v_detail,
            "uniqueness": u_detail,
            "consistency": k_detail,
            "accuracy": a_detail,
            "timeliness": t_detail,
        },
        "heatmap": heatmap,
        **health,
    }
```

backend/src/dq_engine/rules.py 117 lines

```python
"""Rule evaluation → issues with severity."""
from __future__ import annotations

import uuid
from typing import Any


DEFAULT_RULES = [
    {
        "rule_id": "completeness_overall",
        "name": "Overall Completeness",
        "dimension": "completeness",
        "threshold": 95,
        "severity": "HIGH",
    },
    {
        "rule_id": "validity_overall",
        "name": "Overall Validity",
        "dimension": "validity",
        "threshold": 90,
        "severity": "MEDIUM",
    },
    {
        "rule_id": "uniqueness_overall",
        "name": "Key Uniqueness",
        "dimension": "uniqueness",
        "threshold": 98,
        "severity": "HIGH",
    },
    {
        "rule_id": "consistency_overall",
        "name": "Format Consistency",
        "dimension": "consistency",
        "threshold": 90,
        "severity": "LOW",
    },
    {
        "rule_id": "accuracy_overall",
        "name": "Value Accuracy",
        "dimension": "accuracy",
        "threshold": 95,
        "severity": "MEDIUM",
    },
    {
        "rule_id": "timeliness_overall",
        "name": "Data Timeliness",
        "dimension": "timeliness",
        "threshold": 85,
        "severity": "MEDIUM",
    },
]


def evaluate_rules(score_result: dict, profile: dict) -> list[dict[str, Any]]:
    issues: list[dict[str, Any]] = []
    dimensions = score_result["dimensions"]

    for rule in DEFAULT_RULES:
        dim = rule["dimension"]
        current = dimensions.get(dim, 100)
        passed = current >= rule["threshold"]
        if not passed:
            issues.append(
                {
                    "issue_id": str(uuid.uuid4()),
                    "rule_id": rule["rule_id"],
                    "rule_name": rule["name"],
                    "dimension": dim,
                    "threshold": rule["threshold"],
                    "current_score": current,
                    "result": "FAIL",
                    "severity": rule["severity"],
                    "status": "OPEN",
                    "message": (
                        f"{rule['name']} failed: score {current}% "
                        f"below threshold {rule['threshold']}%"
                    ),
```

```
                }
            )

    # Column-level completeness issues for critical columns
    for col in profile.get("column_profiles", []):
        if col["null_rate"] > 20:
            severity = "HIGH" if col["null_rate"] > 40 else "MEDIUM"
            issues.append(
                {
                    "issue_id": str(uuid.uuid4()),
                    "rule_id": f"col_completeness_{col['name']}",
                    "rule_name": f"{col['name']} Completeness",
                    "dimension": "completeness",
                    "threshold": 80,
                    "current_score": round(100 - col["null_rate"], 2),
                    "result": "FAIL",
                    "severity": severity,
                    "status": "OPEN",
                    "column": col["name"],
                    "message": (
                        f"Column {col['name']} has {col['null_rate']}% missing values"
                    ),
                }
            )

    # Severity breakdown
    return issues


def severity_breakdown(issues: list[dict]) -> dict[str, int]:
    counts = {"HIGH": 0, "MEDIUM": 0, "LOW": 0, "CRITICAL": 0, "WARNING": 0, "INFO": 0}
    for issue in issues:
        sev = issue.get("severity", "INFO")
        counts[sev] = counts.get(sev, 0) + 1
    return {
        "high": counts.get("HIGH", 0) + counts.get("CRITICAL", 0),
        "medium": counts.get("MEDIUM", 0) + counts.get("WARNING", 0),
        "low": counts.get("LOW", 0) + counts.get("INFO", 0),
        "total": len(issues),
    }
```

backend/src/dq_engine/anomalies.py 83 lines

```
"""Simple previous-vs-current anomaly detection (no ML)."""
from __future__ import annotations

from typing import Any


DEFAULT_MARGINS = {
    "null_rate_spike_pp": 10.0,  # percentage points
    "volume_drop_pct": 20.0,
    "freshness_delay_days": 3,
    "dimension_drop_pp": 10.0,
}


def detect_anomalies(current: dict, previous: dict | None, margins: dict | None = None) ->
list[dict[str, Any]]:
    if not previous:
        return []

    m = {**DEFAULT_MARGINS, **(margins or {})}
    anomalies: list[dict[str, Any]] = []

    curr_null = current.get("null_rate", 0)
    prev_null = previous.get("null_rate", 0)
    null_delta = curr_null - prev_null
    if null_delta > m["null_rate_spike_pp"]:
        anomalies.append(
            {
                "type": "null_rate_spike",
                "message": f"Null-rate spike: {prev_null}% → {curr_null}% (+{round(null_delta, 1)}
pp)",
                "previous": prev_null,
                "current": curr_null,
                "change": round(null_delta, 2),
            }
```

```
        )

    curr_vol = current.get("record_count", 0)
    prev_vol = previous.get("record_count", 0) or 1
    vol_drop = (prev_vol - curr_vol) / prev_vol * 100
    if vol_drop > m["volume_drop_pct"]:
        anomalies.append(
            {
                "type": "volume_drop",
                "message": f"Record/volume drop: {prev_vol} → {curr_vol} (-{round(vol_drop, 1)}%)",
                "previous": prev_vol,
                "current": curr_vol,
                "change": round(vol_drop, 2),
            }
        )

    curr_fresh = current.get("freshness_delay_days", 0)
    prev_fresh = previous.get("freshness_delay_days", 0)
    fresh_delta = curr_fresh - prev_fresh
    if fresh_delta > m["freshness_delay_days"]:
        anomalies.append(
            {
                "type": "freshness_delay",
                "message": f"Freshness delay increased: {prev_fresh}d → {curr_fresh}d",
                "previous": prev_fresh,
                "current": curr_fresh,
                "change": fresh_delta,
            }
        )

    curr_dims = current.get("dimensions") or {}
    prev_dims = previous.get("dimensions") or {}
    for dim, curr_val in curr_dims.items():
        prev_val = prev_dims.get(dim)
        if prev_val is None:
            continue
        drop = prev_val - curr_val
        if drop > m["dimension_drop_pp"]:
            anomalies.append(
                {
                    "type": "dimension_score_drop",
                    "dimension": dim,
                    "message": f"{dim.title()} score drop: {prev_val}% → {curr_val}% (-{round(drop, 1)}
pp)",
                    "previous": prev_val,
                    "current": curr_val,
                    "change": round(drop, 2),
                }
            )

    return anomalies
```

backend/src/dq_engine/engine.py 93 lines

```
"""DQ engine orchestration: S3 → profile → score → issues → anomalies."""
from __future__ import annotations

from typing import Any

from .anomalies import detect_anomalies
from .dimensions import compute_dimensions
from .profiler import profile_csv
from .rules import evaluate_rules, severity_breakdown


def run_analysis(csv_text: str, previous_run: dict | None = None) -> dict[str, Any]:
    profile = profile_csv(csv_text)
    # Strip rows from persisted profile later; keep for scoring
    score = compute_dimensions(profile)
    issues = evaluate_rules(score, profile)
    severity = severity_breakdown(issues)

    null_cells = score["details"]["completeness"].get("null_cells", 0)
    total_cells = score["details"]["completeness"].get("total_cells", 1) or 1
    null_rate = round(null_cells / total_cells * 100, 2)

    current_signals = {
        "null_rate": null_rate,
```

```
        "record_count": profile["record_count"],
        "freshness_delay_days": score["details"]["timeliness"].get("freshness_delay_days", 0),
        "dimensions": score["dimensions"],
        "dq_score": score["dq_score"],
    }

    previous_signals = None
    if previous_run:
        previous_signals = {
            "null_rate": previous_run.get("null_rate", 0),
            "record_count": previous_run.get("record_count", 0),
            "freshness_delay_days": previous_run.get("freshness_delay_days", 0),
            "dimensions": previous_run.get("dimensions") or {},
            "dq_score": previous_run.get("dq_score", 0),
        }

    anomalies = detect_anomalies(current_signals, previous_signals)

    # Persistable profile (no raw rows)
    persist_profile = {
        "record_count": profile["record_count"],
        "column_count": profile["column_count"],
        "columns": profile["columns"],
        "column_profiles": [
            {k: v for k, v in c.items() if k != "sample_values" or True}
            for c in profile["column_profiles"]
        ],
    }
    # trim sample_values for storage size
    for c in persist_profile["column_profiles"]:
        c["sample_values"] = c.get("sample_values", [])[:5]

    return {
        "dq_score": score["dq_score"],
        "health": score["health"],
        "fit_for_analytics": score["fit_for_analytics"],
        "fit_for_use": score["fit_for_use"],
        "dimensions": score["dimensions"],
        "weights": score["weights"],
        "details": score["details"],
        "heatmap": score["heatmap"],
        "issues": issues,
        "severity": severity,
        "anomalies": anomalies,
        "null_rate": null_rate,
        "record_count": profile["record_count"],
        "freshness_delay_days": current_signals["freshness_delay_days"],
        "profile": persist_profile,
        "rule_coverage": {
            "rules_evaluated": 6 + sum(
                1 for c in profile["column_profiles"] if c["null_rate"] > 20
            ),
            "rules_failed": len(issues),
        },
        "pii_hints": _pii_hints(profile),
    }


def _pii_hints(profile: dict) -> list[dict]:
    hints = []
    for col in profile["column_profiles"]:
        lower = col["name"].lower()
        if col["type"] == "email" or "email" in lower:
            hints.append({"column": col["name"], "type": "email"})
        elif col["type"] == "phone" or "phone" in lower:
            hints.append({"column": col["name"], "type": "phone"})
        elif any(k in lower for k in ("ssn", "passport", "aadhaar", "pan")):
            hints.append({"column": col["name"], "type": "identifier"})
    return hints
```

backend/src/ai/comprehend.py 51 lines

```
"""Amazon Comprehend PII detection on a small sample of each column's values."""
from __future__ import annotations

import logging
import os

```

```python
import boto3

logger = logging.getLogger("clouddq")

PII_MIN_SCORE = float(os.environ.get("PII_MIN_SCORE", "0.8"))
SAMPLE_VALUES = 10
MAX_COLUMNS = 25
# Entity types that are too generic to mark a column as personal data on their own.
IGNORED_TYPES = {"DATE_TIME", "AGE", "URL"}


def _client():
    return boto3.client("comprehend")


def detect_pii_columns(profile: dict, fallback: list[dict]) -> list[dict]:
    """Return one hint per column whose sample values Comprehend classifies as PII.

    Falls back to the name-based hints if Comprehend is unavailable.
    """
    try:
        client = _client()
        hints = []
        for col in profile["column_profiles"][:MAX_COLUMNS]:
            samples = [str(v).strip() for v in (col.get("sample_values") or []) if str(v).strip()]
            if not samples:
                continue
            text = f"{col['name']}: " + "; ".join(samples[:SAMPLE_VALUES])
            entities = client.detect_pii_entities(Text=text[:4000], LanguageCode="en")["Entities"]
            found: dict[str, float] = {}
            for ent in entities:
                if ent["Score"] >= PII_MIN_SCORE and ent["Type"] not in IGNORED_TYPES:
                    found[ent["Type"]] = max(found.get(ent["Type"], 0), ent["Score"])
            if found:
                top = max(found, key=found.get)
                hints.append({
                    "column": col["name"],
                    "type": f"{top} · Amazon Comprehend {round(found[top] * 100)}%",
                    "entities": sorted(found),
                    "source": "comprehend",
                })
        return hints
    except Exception as exc:  # noqa: BLE001 — keep analysis working without Comprehend
        logger.warning("comprehend unavailable: %s", exc)
        return fallback
```

backend/tests/test_dq_engine.py 54 lines

```python
"""Unit tests for DQ engine (no AWS required)."""
from __future__ import annotations

import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))

from dq_engine.engine import run_analysis  # noqa: E402
from dq_engine.dimensions import classify_health  # noqa: E402


SAMPLES = Path(__file__).resolve().parents[2] / "samples"


def test_dirty_customers_has_issues():
    text = (SAMPLES / "customers_dirty.csv").read_text()
    result = run_analysis(text)
    assert 0 <= result["dq_score"] <= 100
    assert result["health"] in {"HEALTHY", "WARNING", "CRITICAL"}
    assert "completeness" in result["dimensions"]
    assert len(result["issues"]) >= 1
    assert len(result["heatmap"]) >= 1


def test_clean_employees_high_score():
    text = (SAMPLES / "employees_clean.csv").read_text()
    result = run_analysis(text)
    assert result["dq_score"] >= 80
```

```
    assert result["fit_for_use"] is True


def test_anomaly_on_volume_drop():
    text = (SAMPLES / "customers_dirty.csv").read_text()
    first = run_analysis(text)
    previous = {**first, "record_count": first["record_count"] * 3, "null_rate": 0}
    second = run_analysis(text, previous_run=previous)
    types = {a["type"] for a in second["anomalies"]}
    assert "volume_drop" in types


def test_health_bands():
    assert classify_health(87)["health"] == "HEALTHY"
    assert classify_health(70)["health"] == "WARNING"
    assert classify_health(40)["health"] == "CRITICAL"


if __name__ == "__main__":
    test_dirty_customers_has_issues()
    test_clean_employees_high_score()
    test_anomaly_on_volume_drop()
    test_health_bands()
    print("All DQ engine tests passed")
```

M5 — Dashboard, Reports & Analytics

frontend/src/pages/DatasetDetailPage.tsx 463 lines

```tsx
import { useEffect, useMemo, useState } from 'react'
import { Link, useParams } from 'react-router-dom'
import {
  CartesianGrid,
  Line,
  LineChart,
  ResponsiveContainer,
  Tooltip,
  XAxis,
  YAxis,
  Bar,
  BarChart,
} from 'recharts'
import { useAuth } from '../auth/AuthContext'
import { HealthBadge, SeverityPills } from '../components/Badges'
import {
  analyzeDataset,
  createAlert,
  enterCorrection,
  exportReport,
  getAiSummary,
  getDataset,
  getReport,
  getUploadUrl,
  issueAction,
  uploadToS3,
} from '../lib/api'
import type { AiSummary, Dataset, DqRun, Issue } from '../types'

export function DatasetDetailPage() {
  const { id = '' } = useParams()
  const { user } = useAuth()
  const [dataset, setDataset] = useState<Dataset | null>(null)
  const [run, setRun] = useState<DqRun | null>(null)
  const [issues, setIssues] = useState<Issue[]>([])
  const [summary, setSummary] = useState<AiSummary | null>(null)
  const [error, setError] = useState('')
  const [busy, setBusy] = useState('')
  const [alertNote, setAlertNote] = useState('Quality below acceptable threshold — please investigate.')
  const [sevFilter, setSevFilter] = useState<'ALL' | 'HIGH' | 'MEDIUM' | 'LOW'>('ALL')

  async function refresh() {
    const ds = await getDataset(id)
    setDataset(ds)
    if (ds.latestRunId) {
      try {
        const report = await getReport(id)
        setRun(report.run)
        setIssues(report.issues)
      } catch {
        setRun(null)
        setIssues([])
      }
    }
    if (ds.aiSummary) setSummary(ds.aiSummary)
  }

  useEffect(() => {
    refresh().catch((e) => setError(e instanceof Error ? e.message : 'Load failed'))
  }, [id])

  const trend = useMemo(
    () =>
      (dataset?.runHistory || []).map((r, i) => ({
        name: `R${i + 1}`,
        score: r.dq_score,
      })),
    [dataset],
  )

  const dimData = useMemo(() => {
```

```
    const dims = run?.dimensions || dataset?.latestDimensions || {}
    return Object.entries(dims).map(([name, score]) => ({ name, score }))
  }, [run, dataset])

  const filteredIssues = issues.filter(
    (i) => sevFilter === 'ALL' || i.severity === sevFilter,
  )

  async function runAnalyze() {
    setBusy('Analyzing…')
    setError('')
    try {
      const res = await analyzeDataset(id)
      setDataset(res.dataset)
      setRun(res.run)
      setIssues(res.run.issues || [])
      setSummary(null)
    } catch (e) {
      setError(e instanceof Error ? e.message : 'Analyze failed')
    } finally {
      setBusy('')
    }
  }

  async function onExport() {
    const file = await exportReport(id)
    const blob = new Blob([file.content], { type: file.contentType })
    const url = URL.createObjectURL(blob)
    const a = document.createElement('a')
    a.href = url
    a.download = file.filename
    a.click()
    URL.revokeObjectURL(url)
  }

  async function onAiSummary() {
    setBusy('Generating AI summary…')
    try {
      const s = await getAiSummary(id)
      setSummary(s)
      await refresh()
    } catch (e) {
      setError(e instanceof Error ? e.message : 'AI summary failed')
    } finally {
      setBusy('')
    }
  }

  async function onAlert() {
    setBusy('Sending alert…')
    try {
      await createAlert(id, alertNote, true)
      await refresh()
    } catch (e) {
      setError(e instanceof Error ? e.message : 'Alert failed')
    } finally {
      setBusy('')
    }
  }

  async function onCorrection() {
    setBusy('Entering correction…')
    try {
      await enterCorrection(id, 'Data user started correction flow')
      await refresh()
    } catch (e) {
      setError(e instanceof Error ? e.message : 'Failed')
    } finally {
      setBusy('')
    }
  }

  async function onReupload(file: File) {
    setBusy('Re-uploading corrected CSV…')
    try {
      const { uploadUrl, s3Key } = await getUploadUrl(file.name)
      await uploadToS3(uploadUrl, file)
      // Keep same dataset id — update via analyze with new key
      await analyzeDataset(id, { s3Key })
      await refresh()
```

```
    } catch (e) {
      setError(e instanceof Error ? e.message : 'Re-upload failed')
    } finally {
      setBusy('')
    }
  }

  async function onIssue(issue: Issue, action: 'acknowledge' | 'resolve' | 'reopen' | 'assign') {
    setBusy(`${action}…`)
    try {
      await issueAction(issue.issue_id, action, id)
      await refresh()
    } catch (e) {
      setError(e instanceof Error ? e.message : 'Action failed')
    } finally {
      setBusy('')
    }
  }

  if (!dataset) {
    return <p className="text-ink-soft">{error || 'Loading…'}</p>
  }

  const isData = user?.role === 'DataUser'
  const isBiz = user?.role === 'BusinessUser'

  return (
    <div className="space-y-8">
      <div className="flex flex-wrap items-start justify-between gap-4">
        <div>
          <Link to="/app/datasets" className="text-sm text-sea hover:underline">
            ← Repository
          </Link>
          <h1 className="font-display text-3xl mt-1">{dataset.name}</h1>
          <p className="text-sm text-ink-soft">
            {dataset.filename} · Status {dataset.status} · Workflow {dataset.workflowStatus}
          </p>
        </div>
        <HealthBadge
          health={run?.health || dataset.latestHealth}
          score={run?.dq_score ?? dataset.latestScore}
          fit={run?.fit_for_use ?? dataset.latestFitForUse}
        />
      </div>

      {error && <p className="text-coral text-sm">{error}</p>}
      {busy && <p className="text-sea text-sm">{busy}</p>}

      <div className="no-print flex flex-wrap gap-2">
        {isData && (
          <button type="button" onClick={runAnalyze} className="rounded-md bg-sea text-white px-3 py-2
text-sm">
            Run / re-run analysis
          </button>
        )}
        <button type="button" onClick={onExport} className="rounded-md border border-mist px-3 py-2
text-sm">
          Export CSV report
        </button>
        <button type="button" onClick={() => window.print()} className="rounded-md border border-mist
px-3 py-2 text-sm">
          Print report
        </button>
        <button type="button" onClick={onAiSummary} className="rounded-md border border-mist px-3 py-2
text-sm">
          AI executive summary
        </button>
        {isData && (
          <button type="button" onClick={onCorrection} className="rounded-md border border-mist px-3
py-2 text-sm">
            Move to correction
          </button>
        )}
      </div>

      {/* Timeline */}
      <section className="rounded-xl border border-mist bg-white/80 p-4">
        <h2 className="font-display text-lg mb-3">Status timeline</h2>
        <ol className="space-y-2 text-sm">
          {(dataset.timeline || []).map((t, i) => (
```

```
            <li key={i} className="flex gap-3">
              <span className="text-sea font-medium w-36 shrink-0">{t.status}</span>
              <span className="text-ink-soft">{t.note}</span>
              <span className="ml-auto text-xs text-ink-soft">{t.at}</span>
            </li>
          ))}
        </ol>
      </section>

      {/* Dimension charts */}
      {dimData.length > 0 && (
        <section className="grid lg:grid-cols-2 gap-4">
          <div className="rounded-xl border border-mist bg-white/80 p-4">
            <h2 className="font-display text-lg mb-3">Six dimensions</h2>
            <div className="h-64">
              <ResponsiveContainer width="100%" height="100%">
                <BarChart data={dimData}>
                  <CartesianGrid strokeDasharray="3 3" stroke="#dce8e1" />
                  <XAxis dataKey="name" tick={{ fontSize: 11 }} />
                  <YAxis domain={[0, 100]} />
                  <Tooltip />
                  <Bar dataKey="score" fill="#1a6b5c" radius={[4, 4, 0, 0]} />
                </BarChart>
              </ResponsiveContainer>
            </div>
          </div>
          <div className="rounded-xl border border-mist bg-white/80 p-4">
            <h2 className="font-display text-lg mb-3">DQ score trend</h2>
            <div className="h-64">
              {trend.length > 0 ? (
                <ResponsiveContainer width="100%" height="100%">
                  <LineChart data={trend}>
                    <CartesianGrid strokeDasharray="3 3" stroke="#dce8e1" />
                    <XAxis dataKey="name" />
                    <YAxis domain={[0, 100]} />
                    <Tooltip />
                    <Line type="monotone" dataKey="score" stroke="#c4922a" strokeWidth={3} dot={{ r: 5
}} />
                  </LineChart>
                </ResponsiveContainer>
              ) : (
                <p className="text-sm text-ink-soft">Run analysis more than once to see trends.</p>
              )}
            </div>
          </div>
        </section>
      )}

      {/* Heatmap + severity */}
      {run && (
        <section className="grid lg:grid-cols-2 gap-4">
          <div className="rounded-xl border border-mist bg-white/80 p-4">
            <h2 className="font-display text-lg mb-3">Column quality heatmap</h2>
            <div className="space-y-2">
              {run.heatmap.map((h) => (
                <div key={h.column} className="flex items-center gap-3 text-sm">
                  <span className="w-36 truncate font-medium">{h.column}</span>
                  <div className="flex-1 h-3 rounded-full bg-mist overflow-hidden">
                    <div
                      className="h-full rounded-full transition-all"
                      style={{
                        width: `${h.score}%`,
                        background:
                          h.score >= 90 ? '#2d7a4f' : h.score >= 80 ? '#1a6b5c' : h.score >= 60 ?
'#c4922a' : '#c45c3e',
                      }}
                    />
                  </div>
                  <span className="w-20 text-right text-xs">{h.label}</span>
                </div>
              ))}
            </div>
          </div>
          <div className="rounded-xl border border-mist bg-white/80 p-4 space-y-3">
            <h2 className="font-display text-lg">Severity & coverage</h2>
            <SeverityPills severity={run.severity} />
            <p className="text-sm text-ink-soft">
              Rules evaluated: {run.rule_coverage?.rules_evaluated ?? '—'} · Failed:{' '}
              {run.rule_coverage?.rules_failed ?? '—'}
            </p>
```

```
            {run.pii_hints && run.pii_hints.length > 0 && (
              <div>
                <div className="text-xs uppercase tracking-wide text-ink-soft mb-1">PII hints</div>
                <ul className="text-sm space-y-1">
                  {run.pii_hints.map((p) => (
                    <li key={p.column}>
                      {p.column} <span className="text-ink-soft">({p.type})</span>
                    </li>
                  ))}
                </ul>
              </div>
            )}
            {run.anomalies?.length > 0 && (
              <div>
                <div className="text-xs uppercase tracking-wide text-ink-soft mb-1">Anomalies</div>
                <ul className="text-sm space-y-1">
                  {run.anomalies.map((a, i) => (
                    <li key={i} className="text-amber">
                      {a.message}
                    </li>
                  ))}
                </ul>
              </div>
            )}
          </div>
        </section>
      )}

      {/* AI summary */}
      <section className="rounded-xl border border-mist bg-white/80 p-4">
        <h2 className="font-display text-lg mb-2">AI executive summary</h2>
        {summary ? (
          <div className="space-y-3 text-sm">
            <p className="leading-relaxed">{summary.executive_summary}</p>
            <div>
              <div className="font-semibold mb-1">Top 3 recommendations</div>
              <ol className="list-decimal pl-5 space-y-1">
                {summary.recommendations.map((r, i) => (
                  <li key={i}>{r}</li>
                ))}
              </ol>
            </div>
            <p className="text-xs text-ink-soft">
              Model {summary.model} · metrics JSON only (no raw CSV) · {summary.generatedAt}
            </p>
          </div>
        ) : (
          <p className="text-sm text-ink-soft">Generate a Bedrock summary from aggregated quality
metrics.</p>
        )}
      </section>

      {/* Business alert */}
      {isBiz && (
        <section className="no-print rounded-xl border border-coral/30 bg-coral/5 p-4 space-y-3">
          <h2 className="font-display text-lg">Send quality alert</h2>
          <textarea
            value={alertNote}
            onChange={(e) => setAlertNote(e.target.value)}
            className="w-full rounded-md border border-mist px-3 py-2 text-sm"
            rows={3}
          />
          <button type="button" onClick={onAlert} className="rounded-md bg-coral text-white px-4 py-2
text-sm">
            Alert Data User
          </button>
        </section>
      )}

      {/* Correction re-upload */}
      {isData && (dataset.workflowStatus === 'InCorrection' || dataset.workflowStatus ===
'AlertRaised') && (
        <section className="no-print rounded-xl border border-amber/30 bg-amber/5 p-4 space-y-3">
          <h2 className="font-display text-lg">Soft correction — re-upload CSV</h2>
          <input
            type="file"
            accept=".csv"
            onChange={(e) => {
              const f = e.target.files?.[0]
              if (f) void onReupload(f)
```

```
382              }}
383            />
384          </section>
385        )}
386
387        {/* Issues */}
388        <section className="rounded-xl border border-mist bg-white/80 p-4">
389          <div className="flex flex-wrap items-center justify-between gap-3 mb-3">
390            <h2 className="font-display text-lg">Issues</h2>
391            <div className="flex gap-1 text-xs">
392              {(['ALL', 'HIGH', 'MEDIUM', 'LOW'] as const).map((s) => (
393                <button
394                  key={s}
395                  type="button"
396                  onClick={() => setSevFilter(s)}
397                  className={`px-2 py-1 rounded ${sevFilter === s ? 'bg-sea text-white' : 'bg-mist'}`}
398                >
399                  {s}
400                </button>
401              ))}
402            </div>
403          </div>
404          <div className="space-y-3">
405            {filteredIssues.map((issue) => (
406              <div key={issue.issue_id} className="border border-mist rounded-lg p-3 text-sm">
407                <div className="flex flex-wrap justify-between gap-2">
408                  <div>
409                    <span
410                      className={`text-xs font-semibold mr-2 ${
411                        issue.severity === 'HIGH'
412                          ? 'text-coral'
413                          : issue.severity === 'MEDIUM'
414                            ? 'text-amber'
415                            : 'text-sea'
416                      }`}
417                    >
418                      {issue.severity}
419                    </span>
420                    <span className="font-medium">{issue.rule_name}</span>
421                    <span className="text-ink-soft"> · {issue.status}</span>
422                  </div>
423                  {isData && (
424                    <div className="no-print flex gap-2">
425                      {issue.status === 'OPEN' && (
426                        <button type="button" className="text-sea underline" onClick={() =>
onIssue(issue, 'acknowledge')}>
427                          Acknowledge
428                        </button>
429                      )}
430                      {issue.status !== 'RESOLVED' && (
431                        <button type="button" className="text-sea underline" onClick={() =>
onIssue(issue, 'resolve')}>
432                          Resolve
433                        </button>
434                      )}
435                      {issue.status === 'RESOLVED' && (
436                        <button type="button" className="text-sea underline" onClick={() =>
onIssue(issue, 'reopen')}>
437                          Reopen
438                        </button>
439                      )}
440                    </div>
441                  )}
442                </div>
443                <p className="mt-1 text-ink-soft">{issue.message}</p>
444                {issue.history && issue.history.length > 0 && (
445                  <details className="mt-2 text-xs text-ink-soft">
446                    <summary>Audit history</summary>
447                    <ul className="mt-1 space-y-1 pl-2">
448                      {issue.history.map((h, i) => (
449                        <li key={i}>
450                          {h.status} — {h.note} ({h.at})
451                        </li>
452                      ))}
453                    </ul>
454                  </details>
455                )}
456              </div>
457            ))}
458            {filteredIssues.length === 0 && <p className="text-ink-soft text-sm">No issues for this
```

```
filter.</p>}
 459          </div>
 460        </section>
 461      </div>
 462    )
 463  }
```

M6 — AI Executive Summary (Amazon Bedrock)

backend/src/ai/__init__.py 4 lines

```python
"""AI package."""
from . import bedrock

__all__ = ["bedrock"]
```

backend/src/ai/bedrock.py 175 lines

```python
"""Amazon Bedrock bridge — metrics JSON only, never raw CSV rows."""
from __future__ import annotations

import json
import os
from typing import Any

import boto3

from store import dynamo

BEDROCK_MODEL_ID = os.environ.get("BEDROCK_MODEL_ID", "amazon.nova-lite-v1:0")
# Comma-separated models tried in order after BEDROCK_MODEL_ID.
BEDROCK_FALLBACK_MODEL_IDS = [
    m.strip() for m in os.environ.get("BEDROCK_FALLBACK_MODEL_IDS", "").split(",") if m.strip()
]
BEDROCK_REGION = os.environ.get("BEDROCK_REGION") or None


def _bedrock():
    return boto3.client("bedrock-runtime", region_name=BEDROCK_REGION)


def _model_ids() -> list[str]:
    return list(dict.fromkeys([BEDROCK_MODEL_ID, *BEDROCK_FALLBACK_MODEL_IDS]))


def build_metrics_payload(run: dict, meta: dict) -> dict[str, Any]:
    """Aggregated metrics only — no raw CSV rows."""
    severity = run.get("severity") or {}
    return {
        "dataset_name": meta.get("name"),
        "dq_score": run.get("dq_score"),
        "health": run.get("health"),
        "fit_for_analytics": run.get("fit_for_analytics"),
        "dimensions": run.get("dimensions"),
        "issues": {
            "high": severity.get("high", 0),
            "medium": severity.get("medium", 0),
            "low": severity.get("low", 0),
            "total": severity.get("total", 0),
        },
        "anomalies": [a.get("message") or a.get("type") for a in (run.get("anomalies") or [])],
        "rule_coverage": run.get("rule_coverage"),
        "record_count": run.get("record_count"),
        "null_rate": run.get("null_rate"),
        "top_issue_messages": [
            i.get("message") for i in (run.get("issues") or [])[:5]
        ],
    }


def generate_summary(user: dict, dataset_id: str) -> dict:
    meta = dynamo.get_item(f"DATASET#{dataset_id}", "META")
    if not meta:
        raise LookupError("Dataset not found")

    # Return cached summary if present and still valid for latest run
    cached = meta.get("aiSummary")
    if cached and cached.get("runId") == meta.get("latestRunId") and not cached.get("offline"):
        return cached

    runs = dynamo.query_pk(f"DATASET#{dataset_id}", "RUN#")
```

```
    if not runs:
        raise LookupError("No analysis runs yet")
    latest = sorted(runs, key=lambda r: r.get("SK", ""), reverse=True)[0]
    metrics = build_metrics_payload(latest, meta)

    prompt = (
        "You are a data-quality advisor for business stakeholders. "
        "Given ONLY the aggregated metrics JSON below (no raw data rows), write:\n"
        "1) A short executive summary (3-5 sentences) in plain language.\n"
        "2) Exactly 3 prioritized recommended fixes.\n"
        "Respond in JSON with keys: executive_summary (string), recommendations (array of 3
strings).\n\n"
        f"METRICS_JSON:\n{json.dumps(metrics, indent=2)}"
    )

    summary_text, recommendations, model_used = _invoke_bedrock(prompt, metrics)

    result = {
        "runId": latest.get("runId"),
        "generatedAt": dynamo.now_iso(),
        "executive_summary": summary_text,
        "recommendations": recommendations,
        "metrics_used": metrics,  # transparency: what Bedrock saw
        "model": model_used or "offline-template",
        "offline": model_used is None,
    }
    dynamo.update_item(
        f"DATASET#{dataset_id}",
        "META",
        {"aiSummary": result},
    )
    return result


def _invoke_bedrock(prompt: str, metrics: dict) -> tuple[str, list[str], str | None]:
    """Call Bedrock Converse on each configured model; fall back to a template if all fail."""
    client = _bedrock()
    last_error = "no model configured"
    for model_id in _model_ids():
        try:
            resp = client.converse(
                modelId=model_id,
                messages=[{"role": "user", "content": [{"text": prompt}]}],
                inferenceConfig={"maxTokens": 800, "temperature": 0.2},
            )
        except Exception as exc:  # noqa: BLE001 — try the next model, then fall back
            last_error = f"{model_id}: {exc}"
            continue
        text = ""
        for block in resp.get("output", {}).get("message", {}).get("content", []):
            if "text" in block:
                text += block["text"]
        parsed = _parse_ai_json(text)
        if parsed:
            return parsed["executive_summary"], parsed["recommendations"][:3], model_id
        # If model returned prose, wrap it
        return text.strip(), _fallback_recommendations(metrics), model_id
    summary, recs = _fallback_summary(metrics, last_error)
    return summary, recs, None


def _parse_ai_json(text: str) -> dict | None:
    text = text.strip()
    # Extract the JSON object from fences, preambles or reasoning text
    start = text.find("{")
    end = text.rfind("}")
    if start >= 0 and end > start:
        text = text[start : end + 1]
    try:
        data = json.loads(text)
        if "executive_summary" in data and "recommendations" in data:
            return data
    except json.JSONDecodeError:
        return None
    return None


def _fallback_summary(metrics: dict, error: str) -> tuple[str, list[str]]:
    score = metrics.get("dq_score")
    health = metrics.get("health")
```

```
    dims = metrics.get("dimensions") or {}
    worst = sorted(dims.items(), key=lambda x: x[1])[:3]
    summary = (
        f"Dataset quality is currently in the {health} range with an overall DQ score of {score}%. "
        f"The weakest dimensions are "
        + ", ".join(f"{k} ({v}%)" for k, v in worst)
        + ". "
        f"There are {metrics.get('issues', {}).get('high', 0)} high-severity and "
        f"{metrics.get('issues', {}).get('medium', 0)} medium-severity issues. "
        "Addressing the top rule failures should improve fitness for analytics. "
        f"(Generated offline — Bedrock unavailable: {error[:120]})"
    )
    return summary, _fallback_recommendations(metrics)


def _fallback_recommendations(metrics: dict) -> list[str]:
    recs = []
    msgs = metrics.get("top_issue_messages") or []
    for m in msgs[:3]:
        recs.append(f"Investigate and remediate: {m}")
    anomalies = metrics.get("anomalies") or []
    for a in anomalies:
        if len(recs) >= 3:
            break
        recs.append(f"Resolve anomaly: {a}")
    while len(recs) < 3:
        defaults = [
            "Investigate missing required identifiers and reduce null rates.",
            "Standardize formats (email, dates, categorical case) for consistency.",
            "Refresh stale records and eliminate future-dated values.",
        ]
        recs.append(defaults[len(recs)])
    return recs[:3]
```

M7 — Alerts & Soft Correction Workflow

backend/src/alerts/__init__.py 4 lines

```python
"""Alerts package."""
from . import service

__all__ = ["service"]
```

backend/src/alerts/service.py 268 lines

```python
"""Alerts and issue lifecycle (soft correction workflow)."""
from __future__ import annotations

from auth import require_role
from store import dynamo, sns


def create_alert(user: dict, body: dict) -> dict:
    require_role(user, "BusinessUser")
    dataset_id = body.get("datasetId")
    if not dataset_id:
        raise ValueError("datasetId is required")
    meta = dynamo.get_item(f"DATASET#{dataset_id}", "META")
    if not meta:
        raise LookupError("Dataset not found")

    score = meta.get("latestScore")
    health = meta.get("latestHealth")
    if health not in {"WARNING", "CRITICAL"} and (score is None or score >= 80):
        # Allow override with force, but default gate for low/critical
        if not body.get("force"):
            raise ValueError("Alerts are intended for Warning/Critical datasets")

    alert_id = dynamo.new_id()
    now = dynamo.now_iso()
    data_user_id = meta.get("ownerId")
    item = {
        "PK": f"ALERT#{alert_id}",
        "SK": "META",
        "entityType": "ALERT",
        "alertId": alert_id,
        "datasetId": dataset_id,
        "datasetName": meta.get("name"),
        "fromUserId": user["sub"],
        "fromEmail": user.get("email"),
        "toUserId": data_user_id,
        "note": body.get("note") or "Quality below acceptable threshold",
        "status": "OPEN",
        "dqScore": score,
        "health": health,
        "createdAt": now,
        "updatedAt": now,
        "history": [
            {
                "status": "OPEN",
                "at": now,
                "note": body.get("note") or "Business alert raised",
                "by": user.get("email") or user["sub"],
            }
        ],
        "GSI1PK": f"USER#{data_user_id}",
        "GSI1SK": f"ALERT#{now}",
    }
    dynamo.put_item(item)

    timeline = list(meta.get("timeline") or [])
    timeline.append({"status": "AlertRaised", "at": now, "note": item["note"]})
    dynamo.update_item(
        f"DATASET#{dataset_id}",
        "META",
        {
            "workflowStatus": "AlertRaised",
            "alertId": alert_id,
```

```
            "timeline": timeline,
        },
    )
    item["notified"] = sns.publish_alert(item)
    return _public_alert(item)


def list_alerts(user: dict) -> list[dict]:
    if user["role"] == "DataUser":
        items = dynamo.query_gsi1(f"USER#{user['sub']}", "ALERT#")
        return [_public_alert(i) for i in items]
    # Business: scan alerts they created (demo-scale)
    items = dynamo.scan_entity("ALERT#")
    mine = [
        i
        for i in items
        if i.get("SK") == "META" and i.get("fromUserId") == user["sub"]
    ]
    return [_public_alert(i) for i in mine]


def get_alert(user: dict, alert_id: str) -> dict:
    item = dynamo.get_item(f"ALERT#{alert_id}", "META")
    if not item:
        raise LookupError("Alert not found")
    if user["role"] == "DataUser" and item.get("toUserId") != user["sub"]:
        raise PermissionError("Not your alert")
    if user["role"] == "BusinessUser" and item.get("fromUserId") != user["sub"]:
        raise PermissionError("Not your alert")
    return _public_alert(item)


def _append_issue_history(issue: dict, status: str, user: dict, note: str) -> list:
    history = list(issue.get("history") or [])
    history.append(
        {
            "status": status,
            "at": dynamo.now_iso(),
            "note": note,
            "by": user.get("email") or user["sub"],
        }
    )
    return history


def assign_issue(user: dict, issue_id: str, body: dict) -> dict:
    require_role(user, "DataUser")
    dataset_id = body.get("datasetId")
    if not dataset_id:
        raise ValueError("datasetId is required")
    issue = dynamo.get_item(f"DATASET#{dataset_id}", f"ISSUE#{issue_id}")
    if not issue:
        raise LookupError("Issue not found")
    assignee = body.get("assignedTo") or user["sub"]
    history = _append_issue_history(issue, "ASSIGNED", user, body.get("note") or "Assigned")
    updated = dynamo.update_item(
        f"DATASET#{dataset_id}",
        f"ISSUE#{issue_id}",
        {"status": "ASSIGNED", "assignedTo": assignee, "history": history},
    )
    return _public_issue(updated)


def acknowledge_issue(user: dict, issue_id: str, body: dict) -> dict:
    require_role(user, "DataUser")
    dataset_id = body.get("datasetId")
    if not dataset_id:
        raise ValueError("datasetId is required")
    issue = dynamo.get_item(f"DATASET#{dataset_id}", f"ISSUE#{issue_id}")
    if not issue:
        raise LookupError("Issue not found")
    history = _append_issue_history(
        issue, "IN_PROGRESS", user, body.get("note") or "Acknowledged — investigating"
    )
    updated = dynamo.update_item(
        f"DATASET#{dataset_id}",
        f"ISSUE#{issue_id}",
        {"status": "IN_PROGRESS", "history": history},
    )
    # Move dataset into correction if alert exists
```

```
    meta = dynamo.get_item(f"DATASET#{dataset_id}", "META")
    if meta and meta.get("workflowStatus") == "AlertRaised":
        timeline = list(meta.get("timeline") or [])
        timeline.append(
            {
                "status": "InCorrection",
                "at": dynamo.now_iso(),
                "note": "Data user acknowledged alert",
            }
        )
        dynamo.update_item(
            f"DATASET#{dataset_id}",
            "META",
            {"workflowStatus": "InCorrection", "timeline": timeline},
        )
    return _public_issue(updated)


def resolve_issue(user: dict, issue_id: str, body: dict) -> dict:
    require_role(user, "DataUser")
    dataset_id = body.get("datasetId")
    if not dataset_id:
        raise ValueError("datasetId is required")
    issue = dynamo.get_item(f"DATASET#{dataset_id}", f"ISSUE#{issue_id}")
    if not issue:
        raise LookupError("Issue not found")

    # Re-validation: latest run must pass the related rule/dimension
    meta = dynamo.get_item(f"DATASET#{dataset_id}", "META")
    if not meta:
        raise LookupError("Dataset not found")
    dims = meta.get("latestDimensions") or {}
    dimension = issue.get("dimension")
    threshold = issue.get("threshold", 0)
    current = dims.get(dimension)
    if current is None:
        raise ValueError("No latest analysis to re-validate against — re-run analysis first")
    if current < threshold:
        raise ValueError(
            f"Re-validation failed: {dimension} is {current}% (threshold {threshold}%). "
            "Re-run analysis after fixing data."
        )

    history = _append_issue_history(
        issue,
        "RESOLVED",
        user,
        body.get("note") or f"Re-validation passed ({dimension}={current}%)",
    )
    updated = dynamo.update_item(
        f"DATASET#{dataset_id}",
        f"ISSUE#{issue_id}",
        {"status": "RESOLVED", "history": history, "resolvedAt": dynamo.now_iso()},
    )

    # Close alert if all HIGH issues resolved
    if meta.get("alertId"):
        issues = dynamo.query_pk(f"DATASET#{dataset_id}", "ISSUE#")
        open_high = [
            i
            for i in issues
            if i.get("severity") in {"HIGH", "CRITICAL"} and i.get("status") != "RESOLVED"
            and i.get("issue_id") != issue_id
        ]
        # account for current issue now resolved
        if not open_high:
            alert = dynamo.get_item(f"ALERT#{meta['alertId']}", "META")
            if alert:
                ah = list(alert.get("history") or [])
                ah.append(
                    {
                        "status": "RESOLVED",
                        "at": dynamo.now_iso(),
                        "note": "Issues resolved after re-validation",
                        "by": user.get("email") or user["sub"],
                    }
                )
                dynamo.update_item(
                    f"ALERT#{meta['alertId']}",
                    "META",
```

```
                    {"status": "RESOLVED", "history": ah},
                )
            timeline = list(meta.get("timeline") or [])
            timeline.append(
                {
                    "status": "Resolved",
                    "at": dynamo.now_iso(),
                    "note": "Alert resolved",
                }
            )
            dynamo.update_item(
                f"DATASET#{dataset_id}",
                "META",
                {"workflowStatus": "Resolved", "timeline": timeline},
            )

    return _public_issue(updated)


def reopen_issue(user: dict, issue_id: str, body: dict) -> dict:
    dataset_id = body.get("datasetId")
    if not dataset_id:
        raise ValueError("datasetId is required")
    issue = dynamo.get_item(f"DATASET#{dataset_id}", f"ISSUE#{issue_id}")
    if not issue:
        raise LookupError("Issue not found")
    history = _append_issue_history(
        issue, "OPEN", user, body.get("note") or "Issue reopened"
    )
    updated = dynamo.update_item(
        f"DATASET#{dataset_id}",
        f"ISSUE#{issue_id}",
        {"status": "OPEN", "history": history},
    )
    return _public_issue(updated)


def _public_alert(item: dict) -> dict:
    omit = {"PK", "SK", "entityType", "GSI1PK", "GSI1SK"}
    return {k: v for k, v in item.items() if k not in omit}


def _public_issue(item: dict) -> dict:
    omit = {"PK", "SK", "entityType"}
    return {k: v for k, v in item.items() if k not in omit}
```

backend/src/store/sns.py — 38 lines

```
"""Amazon SNS notifications for business alerts."""
from __future__ import annotations

import logging
import os

import boto3

logger = logging.getLogger("clouddq")

ALERT_TOPIC_ARN = os.environ.get("ALERT_TOPIC_ARN", "")
APP_URL = os.environ.get("APP_URL", "").rstrip("/")


def publish_alert(alert: dict) -> bool:
    """Email subscribers of the alert topic. Never fails the API call."""
    if not ALERT_TOPIC_ARN:
        return False
    link = f"{APP_URL}/app/datasets/{alert['datasetId']}" if APP_URL else ""
    message = "\n".join(line for line in [
        f"A Business User raised a data quality alert on '{alert.get('datasetName')}'.",
        "",
        f"DQ score: {alert.get('dqScore')}%  ·  Health: {alert.get('health')}",
        f"Note: {alert.get('note')}",
        f"From: {alert.get('fromEmail')}",
        "",
        f"Open the dataset: {link}" if link else "",
    ] if line is not None)
    try:
        boto3.client("sns").publish(
```

```
            TopicArn=ALERT_TOPIC_ARN,
            Subject=f"CloudDQ alert: {alert.get('datasetName')}"[:100],
            Message=message,
        )
        return True
    except Exception as exc:  # noqa: BLE001
        logger.warning("sns publish failed: %s", exc)
        return False
```

frontend/src/pages/AlertsPage.tsx 67 lines

```
import { useEffect, useState } from 'react'
import { Link } from 'react-router-dom'
import { useAuth } from '../auth/AuthContext'
import { listAlerts } from '../lib/api'
import type { Alert } from '../types'

export function AlertsPage() {
  const { user } = useAuth()
  const [alerts, setAlerts] = useState<Alert[]>([])
  const [error, setError] = useState('')

  useEffect(() => {
    listAlerts()
      .then(setAlerts)
      .catch((e) => setError(e instanceof Error ? e.message : 'Failed'))
  }, [])

  return (
    <div className="space-y-6">
      <div>
        <h1 className="font-display text-3xl">
          {user?.role === 'DataUser' ? 'Alert inbox' : 'Sent alerts'}
        </h1>
        <p className="text-ink-soft text-sm mt-1">
          Lightweight Business → Data User collaboration (not a full ticketing system).
        </p>
      </div>
      {error && <p className="text-coral text-sm">{error}</p>}
      <div className="space-y-3">
        {alerts.map((a) => (
          <div key={a.alertId} className="rounded-xl border border-mist bg-white/80 p-4">
            <div className="flex flex-wrap justify-between gap-2">
              <div>
                <div className="font-medium">{a.datasetName || a.datasetId}</div>
                <div className="text-xs text-ink-soft">
                  {a.status} · Score {a.dqScore ?? '–'}% · {a.health}
                </div>
              </div>
              <Link to={`/app/datasets/${a.datasetId}`} className="text-sea text-sm font-medium
hover:underline">
                Open dataset
              </Link>
            </div>
            <p className="text-sm mt-2">{a.note}</p>
            <p className="text-xs text-ink-soft mt-1">
              From {a.fromEmail || 'business'} · {a.createdAt}
            </p>
            {a.history && (
              <details className="mt-2 text-xs text-ink-soft">
                <summary>Status history</summary>
                <ul className="mt-1 pl-2 space-y-1">
                  {a.history.map((h, i) => (
                    <li key={i}>
                      {h.status} — {h.note} ({h.at})
                    </li>
                  ))}
                </ul>
              </details>
            )}
          </div>
        ))}
        {alerts.length === 0 && (
          <p className="text-ink-soft text-sm">No alerts yet.</p>
        )}
      </div>
    </div>
```

```
  )
}
```

M8 — API, Data Store & Cloud Deployment

backend/src/handler.py 140 lines

```python
"""
CloudDQ AI — single Lambda entry point.
Routes REST API requests from API Gateway to domain services.
"""
from __future__ import annotations

import logging
import os
import re
import traceback

from auth import user_from_event
from http_util import err, ok, parse_body, path_params
from datasets import service as datasets
from alerts import service as alerts
from ai import bedrock

LOG_LEVEL = os.environ.get("LOG_LEVEL", "INFO")
logger = logging.getLogger("clouddq")
logger.setLevel(LOG_LEVEL)


def lambda_handler(event, context):
    method = (event.get("httpMethod") or event.get("requestContext", {}).get("http", {}).get("method")
or "GET").upper()
    # Support both REST (proxy+) and HTTP API path styles
    path = event.get("path") or event.get("rawPath") or "/"
    # Strip stage prefix e.g. /prod
    path = re.sub(r"^/prod", "", path) or "/"

    logger.info("request %s %s", method, path)

    if method == "OPTIONS":
        return ok({"ok": True})

    try:
        user = user_from_event(event)
        # Local/dev authorizer bypass for SAM local without Cognito
        if user["role"] == "Unknown" and os.environ.get("ALLOW_DEV_AUTH") == "1":
            headers = {k.lower(): v for k, v in (event.get("headers") or {}).items()}
            role = headers.get("x-dev-role", "DataUser")
            user = {
                "sub": headers.get("x-dev-sub", "dev-user"),
                "email": headers.get("x-dev-email", "dev@example.com"),
                "username": "dev",
                "groups": [role],
                "role": role if role in {"DataUser", "BusinessUser"} else "DataUser",
            }

        body = parse_body(event)
        params = path_params(event)

        # Health
        if method == "GET" and path in {"/", "/health"}:
            return ok({"service": "CloudDQ AI", "status": "ok"})

        # Datasets
        if method == "POST" and path == "/datasets/upload-url":
            return ok(datasets.create_upload_url(user, body))

        if method == "POST" and path == "/datasets":
            return ok(datasets.register_dataset(user, body), 201)

        if method == "GET" and path == "/datasets":
            return ok({"datasets": datasets.list_datasets(user)})

        m = re.match(r"^/datasets/([^/]+)$", path)
        if m and method == "GET":
            return ok(datasets.get_dataset(user, m.group(1)))

        m = re.match(r"^/datasets/([^/]+)/analyze$", path)
        if m and method == "POST":
```

```
            return ok(datasets.analyze_dataset(user, m.group(1), body))

        m = re.match(r"^/datasets/([^/]+)/runs$", path)
        if m and method == "GET":
            return ok({"runs": datasets.list_runs(user, m.group(1))})

        m = re.match(r"^/datasets/([^/]+)/report$", path)
        if m and method == "GET":
            return ok(datasets.get_report(user, m.group(1)))

        m = re.match(r"^/datasets/([^/]+)/export$", path)
        if m and method == "GET":
            return ok(datasets.export_report_csv(user, m.group(1)))

        m = re.match(r"^/datasets/([^/]+)/summary$", path)
        if m and method == "POST":
            return ok(bedrock.generate_summary(user, m.group(1)))

        m = re.match(r"^/datasets/([^/]+)/issues$", path)
        if m and method == "GET":
            return ok({"issues": datasets.list_issues(user, m.group(1))})

        m = re.match(r"^/datasets/([^/]+)/correction$", path)
        if m and method == "POST":
            return ok(datasets.enter_correction(user, m.group(1), body))

        # Alerts
        if method == "POST" and path == "/alerts":
            return ok(alerts.create_alert(user, body), 201)

        if method == "GET" and path == "/alerts":
            return ok({"alerts": alerts.list_alerts(user)})

        m = re.match(r"^/alerts/([^/]+)$", path)
        if m and method == "GET":
            return ok(alerts.get_alert(user, m.group(1)))

        # Issues lifecycle — /issues/{id}/action with datasetId in body
        m = re.match(r"^/issues/([^/]+)/(assign|acknowledge|resolve|reopen)$", path)
        if m and method == "POST":
            issue_id, action = m.group(1), m.group(2)
            fn = {
                "assign": alerts.assign_issue,
                "acknowledge": alerts.acknowledge_issue,
                "resolve": alerts.resolve_issue,
                "reopen": alerts.reopen_issue,
            }[action]
            return ok(fn(user, issue_id, body))

        # Fallback: proxy path params style
        proxy = (params.get("proxy") or "").lstrip("/")
        if proxy:
            # Re-dispatch with reconstructed path
            event = {**event, "path": "/" + proxy, "rawPath": "/" + proxy}
            event.pop("pathParameters", None)
            return lambda_handler(event, context)

        return err("Not found", 404, path=path, method=method)

    except PermissionError as e:
        logger.warning("forbidden: %s", e)
        return err(str(e), 403)
    except LookupError as e:
        return err(str(e), 404)
    except ValueError as e:
        return err(str(e), 400)
    except Exception as e:  # noqa: BLE001
        logger.error("unhandled: %s\n%s", e, traceback.format_exc())
        return err("Internal server error", 500, detail=str(e))
```

backend/src/http_util.py 52 lines

```
"""Shared HTTP helpers for the single Lambda router."""
from __future__ import annotations

import json
from typing import Any

```

```python

def response(status: int, body: Any, headers: dict | None = None) -> dict:
    base = {
        "Access-Control-Allow-Origin": "*",
        "Access-Control-Allow-Headers": "Content-Type,Authorization,X-Amz-Date,X-Api-Key,X-Amz-Security-Token",
        "Access-Control-Allow-Methods": "GET,POST,PUT,PATCH,DELETE,OPTIONS",
        "Content-Type": "application/json",
    }
    if headers:
        base.update(headers)
    return {
        "statusCode": status,
        "headers": base,
        "body": json.dumps(body, default=str),
    }


def ok(body: Any, status: int = 200) -> dict:
    return response(status, body)


def err(message: str, status: int = 400, **extra: Any) -> dict:
    payload = {"error": message, **extra}
    return response(status, payload)


def parse_body(event: dict) -> dict:
    raw = event.get("body") or "{}"
    if event.get("isBase64Encoded"):
        import base64

        raw = base64.b64decode(raw).decode("utf-8")
    if isinstance(raw, dict):
        return raw
    try:
        return json.loads(raw) if raw else {}
    except json.JSONDecodeError:
        return {}


def path_params(event: dict) -> dict:
    return event.get("pathParameters") or {}


def query_params(event: dict) -> dict:
    return event.get("queryStringParameters") or {}
```

backend/src/__init__.py 1 lines

```python
# Empty init so Lambda package imports resolve as top-level modules
```

backend/src/store/dynamo.py 139 lines

```python
"""DynamoDB single-table access layer."""
from __future__ import annotations

import os
import uuid
from datetime import datetime, timezone
from decimal import Decimal
from typing import Any

import boto3
from boto3.dynamodb.conditions import Key

TABLE_NAME = os.environ.get("TABLE_NAME", "CloudDQ")

_dynamodb = None
_table = None


def _table_resource():
    global _dynamodb, _table
    if _table is None:
        _dynamodb = boto3.resource("dynamodb")
```

```
        _table = _dynamodb.Table(TABLE_NAME)
    return _table


def now_iso() -> str:
    return datetime.now(timezone.utc).isoformat()


def new_id(prefix: str = "") -> str:
    uid = str(uuid.uuid4())
    return f"{prefix}{uid}" if prefix else uid


def to_dynamo(obj: Any) -> Any:
    """Convert floats to Decimal for DynamoDB."""
    if isinstance(obj, float):
        return Decimal(str(round(obj, 4)))
    if isinstance(obj, dict):
        return {k: to_dynamo(v) for k, v in obj.items()}
    if isinstance(obj, list):
        return [to_dynamo(v) for v in obj]
    return obj


def from_dynamo(obj: Any) -> Any:
    """Convert Decimal back to float/int for JSON."""
    if isinstance(obj, Decimal):
        if obj % 1 == 0:
            return int(obj)
        return float(obj)
    if isinstance(obj, dict):
        return {k: from_dynamo(v) for k, v in obj.items()}
    if isinstance(obj, list):
        return [from_dynamo(v) for v in obj]
    return obj


def put_item(item: dict) -> dict:
    table = _table_resource()
    cleaned = to_dynamo(item)
    table.put_item(Item=cleaned)
    return from_dynamo(cleaned)


def get_item(pk: str, sk: str) -> dict | None:
    table = _table_resource()
    resp = table.get_item(Key={"PK": pk, "SK": sk})
    item = resp.get("Item")
    return from_dynamo(item) if item else None


def update_item(pk: str, sk: str, updates: dict) -> dict:
    table = _table_resource()
    expr_parts = []
    names = {}
    values = {}
    for i, (k, v) in enumerate(updates.items()):
        nk = f"#k{i}"
        nv = f":v{i}"
        expr_parts.append(f"{nk} = {nv}")
        names[nk] = k
        values[nv] = to_dynamo(v)
    values[":updatedAt"] = now_iso()
    names["#updatedAt"] = "updatedAt"
    expr_parts.append("#updatedAt = :updatedAt")

    resp = table.update_item(
        Key={"PK": pk, "SK": sk},
        UpdateExpression="SET " + ", ".join(expr_parts),
        ExpressionAttributeNames=names,
        ExpressionAttributeValues=values,
        ReturnValues="ALL_NEW",
    )
    return from_dynamo(resp["Attributes"])


def query_pk(pk: str, sk_prefix: str | None = None) -> list[dict]:
    table = _table_resource()
    if sk_prefix:
        resp = table.query(
```

```
            KeyConditionExpression=Key("PK").eq(pk) & Key("SK").begins_with(sk_prefix)
        )
    else:
        resp = table.query(KeyConditionExpression=Key("PK").eq(pk))
    return [from_dynamo(i) for i in resp.get("Items", [])]


def query_gsi1(gsi1pk: str, sk_prefix: str | None = None) -> list[dict]:
    table = _table_resource()
    if sk_prefix:
        resp = table.query(
            IndexName="GSI1",
            KeyConditionExpression=Key("GSI1PK").eq(gsi1pk)
            & Key("GSI1SK").begins_with(sk_prefix),
        )
    else:
        resp = table.query(
            IndexName="GSI1",
            KeyConditionExpression=Key("GSI1PK").eq(gsi1pk),
        )
    return [from_dynamo(i) for i in resp.get("Items", [])]


def scan_entity(entity_prefix: str) -> list[dict]:
    """Scan for items whose PK starts with entity_prefix (demo-scale)."""
    table = _table_resource()
    items: list[dict] = []
    kwargs: dict[str, Any] = {}
    while True:
        resp = table.scan(**kwargs)
        for item in resp.get("Items", []):
            if str(item.get("PK", "")).startswith(entity_prefix):
                items.append(from_dynamo(item))
        if "LastEvaluatedKey" not in resp:
            break
        kwargs["ExclusiveStartKey"] = resp["LastEvaluatedKey"]
    return items
```

backend/src/requirements.txt — 2 lines

```
boto3>=1.34.0
botocore>=1.34.0
```

backend/template.yaml — 171 lines

```yaml
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Description: CloudDQ AI — serverless data quality platform (8 AWS services only)

Globals:
  Function:
    Timeout: 120
    MemorySize: 1024
    Runtime: python3.12
    Architectures:
      - x86_64
    Environment:
      Variables:
        TABLE_NAME: !Ref CloudDQTable
        BUCKET_NAME: !Ref DatasetsBucket
        BEDROCK_MODEL_ID: !Ref BedrockModelId
        USER_POOL_ID: !Ref ExistingUserPoolId
        LOG_LEVEL: INFO

Parameters:
  BedrockModelId:
    Type: String
    Default: amazon.nova-lite-v1:0
    Description: Bedrock model ID for executive summaries
  CorsOrigin:
    Type: String
    Default: '*'
    Description: Allowed CORS origin for API Gateway
  ExistingUserPoolId:
    Type: String
    Description: Cognito user pool created in the console (groups DataUser and BusinessUser)
```

```yaml
  ExistingUserPoolClientId:
    Type: String
    Description: Public app client ID for that user pool

Resources:
  # ── S3 (private CSV storage) ──────────────────────────
  DatasetsBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketEncryption:
        ServerSideEncryptionConfiguration:
          - ServerSideEncryptionByDefault:
              SSEAlgorithm: AES256
      PublicAccessBlockConfiguration:
        BlockPublicAcls: true
        BlockPublicPolicy: true
        IgnorePublicAcls: true
        RestrictPublicBuckets: true
      CorsConfiguration:
        CorsRules:
          - AllowedHeaders:
              - '*'
            AllowedMethods:
              - PUT
              - GET
              - HEAD
            AllowedOrigins:
              - !Ref CorsOrigin
            MaxAge: 3600

  # ── DynamoDB (single-table) ───────────────────────────
  CloudDQTable:
    Type: AWS::DynamoDB::Table
    Properties:
      TableName: CloudDQ
      BillingMode: PAY_PER_REQUEST
      AttributeDefinitions:
        - AttributeName: PK
          AttributeType: S
        - AttributeName: SK
          AttributeType: S
        - AttributeName: GSI1PK
          AttributeType: S
        - AttributeName: GSI1SK
          AttributeType: S
      KeySchema:
        - AttributeName: PK
          KeyType: HASH
        - AttributeName: SK
          KeyType: RANGE
      GlobalSecondaryIndexes:
        - IndexName: GSI1
          KeySchema:
            - AttributeName: GSI1PK
              KeyType: HASH
            - AttributeName: GSI1SK
              KeyType: RANGE
          Projection:
            ProjectionType: ALL

  # ── Lambda + API Gateway ──────────────────────────────
  ApiFunction:
    Type: AWS::Serverless::Function
    Properties:
      FunctionName: clouddq-api
      CodeUri: src/
      Handler: handler.lambda_handler
      Description: CloudDQ AI single Lambda — APIs, DQ engine, alerts, Bedrock bridge
      Policies:
        - DynamoDBCrudPolicy:
            TableName: !Ref CloudDQTable
        - S3CrudPolicy:
            BucketName: !Ref DatasetsBucket
        - Statement:
            - Effect: Allow
              Action:
                - bedrock:InvokeModel
                - bedrock:Converse
              Resource: '*'
            - Effect: Allow
```

```
              Action:
                - logs:CreateLogGroup
                - logs:CreateLogStream
                - logs:PutLogEvents
              Resource: '*'
      Events:
        ProxyApi:
          Type: Api
          Properties:
            Path: /{proxy+}
            Method: ANY
            RestApiId: !Ref CloudDQApi
        RootApi:
          Type: Api
          Properties:
            Path: /
            Method: ANY
            RestApiId: !Ref CloudDQApi

  CloudDQApi:
    Type: AWS::Serverless::Api
    Properties:
      Name: clouddq-api
      StageName: prod
      Cors:
        AllowMethods: "'GET,POST,PUT,PATCH,DELETE,OPTIONS'"
        AllowHeaders: "'Content-Type,Authorization,X-Amz-Date,X-Api-Key,X-Amz-Security-Token'"
        AllowOrigin: !Sub "'${CorsOrigin}'"
      Auth:
        DefaultAuthorizer: CognitoAuthorizer
        AddDefaultAuthorizerToCorsPreflight: false
        Authorizers:
          CognitoAuthorizer:
            UserPoolArn: !Sub arn:aws:cognito-
idp:${AWS::Region}:${AWS::AccountId}:userpool/${ExistingUserPoolId}

  ApiLogGroup:
    Type: AWS::Logs::LogGroup
    Properties:
      LogGroupName: !Sub /aws/lambda/${ApiFunction}
      RetentionInDays: 14

Outputs:
  ApiUrl:
    Description: API Gateway endpoint URL
    Value: !Sub https://${CloudDQApi}.execute-api.${AWS::Region}.amazonaws.com/prod
  UserPoolId:
    Description: Cognito User Pool ID
    Value: !Ref ExistingUserPoolId
  UserPoolClientId:
    Description: Cognito App Client ID
    Value: !Ref ExistingUserPoolClientId
  BucketName:
    Description: Private S3 bucket for CSVs
    Value: !Ref DatasetsBucket
  TableName:
    Description: DynamoDB table name
    Value: !Ref CloudDQTable
  Region:
    Description: AWS Region
    Value: !Ref AWS::Region
```

backend/local_server.py 234 lines

```
"""
Local development API — same Lambda handler, filesystem-backed S3 + DynamoDB.
Run:  cd backend && python3 local_server.py
"""
from __future__ import annotations

import json
import os
import sys
import threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from urllib.parse import unquote, urlparse

```

```python
# Ensure src/ is on path
ROOT = Path(__file__).resolve().parent
SRC = ROOT / "src"
sys.path.insert(0, str(SRC))

DATA = ROOT / ".localdata"
S3_DIR = DATA / "s3"
DDB_FILE = DATA / "dynamodb.json"
DATA.mkdir(exist_ok=True)
S3_DIR.mkdir(exist_ok=True)

os.environ.setdefault("TABLE_NAME", "CloudDQ")
os.environ.setdefault("BUCKET_NAME", "local-clouddq")
os.environ.setdefault("ALLOW_DEV_AUTH", "1")
os.environ.setdefault("BEDROCK_MODEL_ID", "amazon.nova-lite-v1:0")
os.environ.setdefault("LOG_LEVEL", "INFO")
os.environ.setdefault("LOCAL_MODE", "1")

# ── Patch store modules before importing handler ──────────────
import store.dynamo as dynamo_mod  # noqa: E402
import store.s3 as s3_mod  # noqa: E402


_lock = threading.Lock()


def _load_db() -> dict:
    if DDB_FILE.exists():
        return json.loads(DDB_FILE.read_text())
    return {"items": {}}


def _save_db(db: dict) -> None:
    DDB_FILE.write_text(json.dumps(db, indent=2, default=str))


def _key(pk: str, sk: str) -> str:
    return f"{pk}||{sk}"


def local_put_item(item: dict) -> dict:
    with _lock:
        db = _load_db()
        cleaned = dynamo_mod.to_dynamo(item)
        # Convert Decimal back for JSON file
        cleaned = dynamo_mod.from_dynamo(cleaned)
        db["items"][_key(item["PK"], item["SK"])] = cleaned
        _save_db(db)
        return cleaned


def local_get_item(pk: str, sk: str) -> dict | None:
    with _lock:
        db = _load_db()
        return db["items"].get(_key(pk, sk))


def local_update_item(pk: str, sk: str, updates: dict) -> dict:
    with _lock:
        db = _load_db()
        k = _key(pk, sk)
        item = dict(db["items"].get(k) or {"PK": pk, "SK": sk})
        item.update(updates)
        item["updatedAt"] = dynamo_mod.now_iso()
        db["items"][k] = item
        _save_db(db)
        return item


def local_query_pk(pk: str, sk_prefix: str | None = None) -> list[dict]:
    with _lock:
        db = _load_db()
        out = []
        for item in db["items"].values():
            if item.get("PK") != pk:
                continue
            if sk_prefix and not str(item.get("SK", "")).startswith(sk_prefix):
                continue
            out.append(item)
        return out
```

```
 95
 96
 97 def local_query_gsi1(gsi1pk: str, sk_prefix: str | None = None) -> list[dict]:
 98     with _lock:
 99         db = _load_db()
100         out = []
101         for item in db["items"].values():
102             if item.get("GSI1PK") != gsi1pk:
103                 continue
104             if sk_prefix and not str(item.get("GSI1SK", "")).startswith(sk_prefix):
105                 continue
106             out.append(item)
107         return out
108
109
110 def local_scan_entity(entity_prefix: str) -> list[dict]:
111     with _lock:
112         db = _load_db()
113         return [i for i in db["items"].values() if str(i.get("PK", "")).startswith(entity_prefix)]
114
115
116 dynamo_mod.put_item = local_put_item
117 dynamo_mod.get_item = local_get_item
118 dynamo_mod.update_item = local_update_item
119 dynamo_mod.query_pk = local_query_pk
120 dynamo_mod.query_gsi1 = local_query_gsi1
121 dynamo_mod.scan_entity = local_scan_entity
122
123
124 def local_presigned(key: str, content_type: str = "text/csv", expires: int = 900) -> str:
125     # Point frontend PUT to local server
126     return f"http://127.0.0.1:3000/_local_s3/{key}?contentType={content_type}"
127
128
129 def local_get_bytes(key: str) -> bytes:
130     path = S3_DIR / key
131     if not path.exists():
132         raise FileNotFoundError(f"S3 object not found: {key}")
133     return path.read_bytes()
134
135
136 def local_get_text(key: str) -> str:
137     return local_get_bytes(key).decode("utf-8-sig")
138
139
140 def local_put_object(key: str, body, content_type: str = "text/csv") -> None:
141     path = S3_DIR / key
142     path.parent.mkdir(parents=True, exist_ok=True)
143     data = body if isinstance(body, (bytes, bytearray)) else body.read()
144     path.write_bytes(data)
145
146
147 s3_mod.presigned_put_url = local_presigned
148 s3_mod.get_object_bytes = local_get_bytes
149 s3_mod.get_object_text = local_get_text
150 s3_mod.put_object = local_put_object
151
152 from handler import lambda_handler  # noqa: E402
153
154
155 class Handler(BaseHTTPRequestHandler):
156     def _cors(self):
157         self.send_header("Access-Control-Allow-Origin", "*")
158         self.send_header(
159             "Access-Control-Allow-Headers",
160             "Content-Type,Authorization,X-Amz-Date,X-Api-Key,X-Amz-Security-Token,X-Dev-Role,X-Dev-
Sub,X-Dev-Email",
161         )
162         self.send_header("Access-Control-Allow-Methods", "GET,POST,PUT,PATCH,DELETE,OPTIONS")
163
164     def do_OPTIONS(self):
165         self.send_response(204)
166         self._cors()
167         self.end_headers()
168
169     def do_PUT(self):
170         parsed = urlparse(self.path)
171         if parsed.path.startswith("/_local_s3/"):
172             key = unquote(parsed.path[len("/_local_s3/") :])
173             length = int(self.headers.get("Content-Length", 0))
```

```
            body = self.rfile.read(length)
            local_put_object(key, body)
            self.send_response(200)
            self._cors()
            self.end_headers()
            self.wfile.write(b"OK")
            return
        self.send_response(404)
        self.end_headers()

    def _handle(self):
        parsed = urlparse(self.path)
        length = int(self.headers.get("Content-Length", 0))
        body = self.rfile.read(length).decode("utf-8") if length else None
        headers = {k: v for k, v in self.headers.items()}
        event = {
            "httpMethod": self.command,
            "path": parsed.path,
            "headers": headers,
            "body": body,
            "isBase64Encoded": False,
            "pathParameters": {"proxy": parsed.path.lstrip("/")},
            "requestContext": {"authorizer": {"claims": {}}},
        }
        result = lambda_handler(event, None)
        self.send_response(result.get("statusCode", 200))
        self._cors()
        for k, v in (result.get("headers") or {}).items():
            if k.lower() == "content-type" or k.lower().startswith("access-control"):
                self.send_header(k, v)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write((result.get("body") or "").encode("utf-8"))

    def do_GET(self):
        self._handle()

    def do_POST(self):
        self._handle()

    def do_PATCH(self):
        self._handle()

    def do_DELETE(self):
        self._handle()

    def log_message(self, fmt, *args):
        sys.stderr.write("%s - %s\n" % (self.address_string(), fmt % args))


def main():
    port = int(os.environ.get("PORT", "3000"))
    server = ThreadingHTTPServer(("127.0.0.1", port), Handler)
    print(f"CloudDQ local API on http://127.0.0.1:{port}", flush=True)
    print(f"Data dir: {DATA}", flush=True)
    print("Dev auth: send X-Dev-Role: DataUser|BusinessUser", flush=True)
    server.serve_forever()


if __name__ == "__main__":
    main()
```

frontend/amplify.yml 16 lines

```yaml
version: 1
frontend:
  phases:
    preBuild:
      commands:
        - npm ci
    build:
      commands:
        - npm run build
  artifacts:
    baseDirectory: dist
    files:
      - '**/*'
  cache:
```

```
    paths:
      - node_modules/**/*
```